



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Inteligencia Artificial

Predicción de Cadenas de Ataque en Infraestructuras Kubernetes mediante Graph Neural Networks

Trabajo fin de estudio presentado por:	Milton Obed Rayo Viana
Tipo de trabajo:	Desarrollo de software
Director/a:	Pedro León Millán
Fecha:	Septiembre de 2026

Resumen

Las herramientas de análisis estático de Kubernetes detectan configuraciones inseguras aisladas, pero no si se encadenan en una ruta de ataque viable. El objetivo es identificar automáticamente esas cadenas a partir de manifiestos estáticos, sin acceso a un clúster activo. kubescan combina un clasificador Random Forest (Capa 1), una red de atención sobre grafos (*Graph Attention Network*, GAT) (Capa 2) y pesos de *ensemble* optimizados por Algoritmo Genético (Capa 3).

El corpus —2.827 manifiestos de cinco fuentes públicas complementarias— y su protocolo de particionado antifuga son la contribución metodológica central. Los grafos de clúster derivados incorporan cinco tipos de arista de seguridad. El particionado, consciente de grupos y de familias de plantilla, elimina la fuga entre familias casi duplicadas.

La Capa 1 alcanza $F1 \text{ macro} = 0,9946$ y área bajo la curva ROC (AUC) = 0,9986, sin falsos negativos en test. La Capa 2 logra $F1 \text{ macro} = 0,803 \pm 0,137$ y $\text{Precisión}@5 = 0,720 \pm 0,160$ en validación cruzada. Sobre el test independiente (86 grafos, 5 cadenas reales) el sistema alcanza $\text{Precisión}@1 = 1,00$, $\text{Precisión}@3 = 1,00$ y $\text{Precisión}@5 = 0,80$, sin falsos positivos en los tres clústeres limpios ($FPR_{\text{clean}} = 0 \%$). La restricción de viabilidad estructural es decisiva: sin ella $P@1$ y $P@3$ caen a 0,00 y 0,33. Las etiquetas derivan de reglas sobre las mismas características que observan los modelos: las cifras miden separabilidad del criterio, no validación externa. Tres expertos otorgan 88,3 de media en la escala de usabilidad del sistema (SUS), «excelente».

El análisis estático de manifiestos basta para situar las cadenas de ataque en cabeza del *ranking*. kubescan es viable como control preventivo integrable en flujos de integración y entrega continua (CI/CD), antes de que las configuraciones de riesgo lleguen a producción.

Palabras clave: Kubernetes, Graph Neural Networks, Random Forest, cadenas de ataque, infraestructura como código.

Abstract

Static analysis tools for Kubernetes detect isolated insecure settings, but not whether they chain into a viable attack path. The objective of this work is to identify such chains automatically from static manifests, without access to a live cluster. kubescan combines a Random Forest classifier (Layer 1), a Graph Attention Network (GAT) (Layer 2) and ensemble weights optimised by a Genetic Algorithm (Layer 3).

The corpus assembled —2,827 manifests from five complementary public sources— and its anti-leakage partitioning protocol are the central methodological contribution. The cluster graphs derived from it carry five types of security edge. Partitioning is group- and template-family-aware, which removes the leakage between near-duplicate families.

Layer 1 achieves macro-F1=0.9946 and an area under the ROC curve (AUC)=0.9986, with no false negatives on test. Layer 2 achieves macro-F1=0.803 $\pm$ 0.137 and Precision@5=0.720 $\pm$ 0.160 under cross-validation. On the independent test set (86 graphs, 5 real chains) the system achieves Precision@1=1.00, Precision@3=1.00 and Precision@5=0.80, with no false positives on the three clean clusters (FPR_clean=0 %). The structural feasibility constraint is decisive: without it P@1 and P@3 fall to 0.00 and 0.33. Graph labels are derived from rules defined over the same features the models observe, so the figures measure separability of that criterion, not external validation. Three experts award a mean System Usability Scale (SUS) score of 88.3, an “excellent” rating.

Static analysis of manifests therefore suffices to place attack chains at the top of the risk ranking. kubescan is viable as a preventive control integrable into Continuous Integration / Continuous Delivery (CI/CD) pipelines, before risky configurations reach production.

Keywords: Kubernetes, Graph Neural Networks, Random Forest, attack chains, infrastructure as code.

Índice de contenidos

Resumen	I
Abstract	II
Índice de figuras	VII
Índice de tablas	VIII
Índice de ecuaciones	IX
1. Introducción	1
1.1. Motivación	1
1.2. Planteamiento del trabajo	2
1.3. Estructura del trabajo	3
2. Contexto y estado del arte	5
2.1. Contexto del problema: seguridad en infraestructuras Kubernetes	5
2.2. Análisis estático de IaC	7
2.3. Aprendizaje automático para detección de vulnerabilidades	8
2.4. Redes neuronales de grafos	11
2.5. GNN aplicadas a ciberseguridad	12
2.6. Detección de cadenas de ataque y grafos de ataque en Kubernetes	13
2.7. Conclusiones del estado del arte	15
3. Objetivos concretos y metodología de trabajo	17
3.1. Objetivo general	17
3.2. Objetivos específicos	17
3.3. Metodología del trabajo	20
4. Identificación de requisitos	23
	III

4.1.	Requisitos del sistema	23
4.1.1.	Requisitos funcionales	23
4.1.2.	Requisitos no funcionales	25
4.1.3.	Trazabilidad de los requisitos	26
4.2.	Justificación del alcance	27
4.3.	Estrategia de partición y validación de datos	28
4.4.	Métricas de evaluación	29
5.	Descripción de la herramienta software desarrollada	32
5.1.	Arquitectura general del sistema	32
5.2.	Construcción del conjunto de datos	33
5.2.1.	Fuentes de datos	34
5.2.2.	Características extraídas	35
5.3.	Construcción del grafo de clúster	36
5.3.1.	Detección de escape y movimiento lateral	38
5.3.2.	Detección de privilegio RBAC	39
5.4.	Capa 1: Random Forest	39
5.4.1.	Modelo binario	40
5.4.2.	Modelo de severidad	40
5.5.	Capa 2: Graph Attention Network	40
5.5.1.	Arquitectura	41
5.5.2.	Entrenamiento	42
5.5.3.	Aumentación de datos	43
5.6.	Capa 3: <i>Ensemble</i> con Algoritmo Genético	45
5.6.1.	Restricción de viabilidad estructural	46
5.7.	Implementación de kubescan	46
5.8.	Tabla de características del sistema	48
5.9.	Calidad de software y proceso de desarrollo	48
6.	Evaluación	50
6.1.	Línea de base del estado del arte	50
6.1.1.	Capa 1 en contexto	50

6.1.2.	Capa 2 en contexto	51
6.2.	Resultados de la Capa 1: Random Forest	52
6.2.1.	Clasificación binaria	52
6.2.2.	Importancia de características	54
6.2.3.	Modelo de severidad	55
6.3.	Resultados de la Capa 2: GNN	56
6.3.1.	Evolución por fase experimental	57
6.3.2.	Análisis de Precisión@5	58
6.3.3.	Resultados por pliegue (fase final)	59
6.3.4.	Ablación de función de pérdida y criterio de selección	60
6.3.5.	Ablación de arquitectura	62
6.3.6.	Robustez frente a la semilla de entrenamiento	63
6.4.	Resultados de la Capa 3: <i>Ensemble</i>	64
6.5.	Análisis de la clasificación por clases: GNN	67
6.6.	Validación con herramientas de análisis estático	68
6.7.	Métrica global del proceso completo	69
6.8.	Verificación de cumplimiento de requisitos	70
6.8.1.	Requisitos funcionales	70
6.8.2.	Requisitos no funcionales	71
6.9.	Evaluación de usabilidad y aplicabilidad con usuarios expertos	72
6.9.1.	Perfil de los participantes	73
6.9.2.	Éxito por tarea	73
6.9.3.	Usabilidad percibida (SUS)	73
6.9.4.	Aplicabilidad y análisis cualitativo	74
6.9.5.	Amenazas a la validez de la evaluación	75
7.	Conclusiones y trabajo futuro	76
7.1.	Conclusiones	76
7.2.	Consideraciones éticas y uso responsable	79
7.3.	Limitaciones	80
7.3.1.	Amenazas a la validez	84
7.4.	Líneas de trabajo futuro	85

Referencias bibliográficas	87
Anexo A. Código fuente y datos analizados	91
A.1. Catálogo de características del clasificador	93
A.2. Trazabilidad de requisitos	93
A.3. Materiales del estudio de usabilidad	93
Anexo B. Guía de uso y reproducción	98
B.1. Instalación	98
B.2. Uso de la interfaz de línea de comandos	98
B.2.1. Análisis de un directorio de manifiestos	98
B.2.2. Análisis de un clúster en ejecución	101
B.3. Reproducción del <i>pipeline</i> de investigación	102
Anexo C. Acrónimos y abreviaturas	104

Índice de figuras

Figura 1.	Arquitectura del sistema kubescan	33
Figura 2.	Ejemplo de grafo de clúster Kubernetes con cuatro nodos	37
Figura 3.	Matriz de confusión del Random Forest en el conjunto de test	54
Figura 4.	Importancia de características del clasificador Random Forest	55
Figura 5.	Precisión@5 por pliegue con tres semillas de entrenamiento	64
Figura 6.	Efecto de la restricción de viabilidad estructural	66
Figura 7.	Matriz de confusión del <i>ensemble</i> de pliegues en test	68
Figura 8.	Flujo de ejecución de kubescan scan	99

Índice de tablas

Tabla 1.	Comparativa de algoritmos de ML para clasificación de manifiestos Infrastructure as Code (Infraestructura como Código) (IaC)	11
Tabla 2.	Comparativa de herramientas de seguridad para Kubernetes	15
Tabla 3.	Composición del conjunto de datos de manifiestos Kubernetes	34
Tabla 4.	Tipos de aristas en el grafo de clúster	37
Tabla 5.	Resultados del Random Forest — clasificación binaria	53
Tabla 6.	F1 por clase — modelo de severidad (3 clases, test)	56
Tabla 7.	Evolución de resultados de la Capa 2 (GAT, 5-fold CV)	58
Tabla 8.	Resultados por pliegue — Capa 2, fase final	60
Tabla 9.	Ablación de función de pérdida y criterio de selección	61
Tabla 10.	Ablación de arquitectura — Capa 2	62
Tabla 11.	Ablación de componentes del <i>ensemble</i> en el conjunto de test	67
Tabla 12.	Características del clasificador Random Forest — grupo Rahman (1/3) . .	94
Tabla 13.	Características del clasificador Random Forest — grupo Rahman (2/3) . .	95
Tabla 14.	Características del clasificador Random Forest — grupos derivado y ex- tendido, y nodo (3/3)	96
Tabla 15.	Trazabilidad de requisitos	97
Tabla 16.	Perfil de los participantes expertos	97
Tabla 17.	Tasa de éxito por tarea (3 participantes)	97
Tabla 18.	Opciones de la interfaz de línea de comandos de kubescan	103

Índice de ecuaciones

Ecuación 1.	Actualización de la representación de un nodo en una capa GNN . .	11
Ecuación 2.	Coeficiente de atención en Graph Attention Networks	12
Ecuación 3.	Precisión@ k normalizada	30
Ecuación 4.	Función de puntuación del <i>ensemble</i> (Capa 3)	32
Ecuación 5.	Cálculo de pesos de clase para el entrenamiento de la GAT	43
Ecuación 6.	Función objetivo (fitness) del Algoritmo Genético	45

1. Introducción

La seguridad de una infraestructura Kubernetes queda determinada, en buena medida, antes de que el clúster llegue a existir: son los manifiestos declarativos —los ficheros YAML Ain't Markup Language (Lenguaje de Serialización de Datos Legible por Humanos) (YAML) que describen el estado deseado de cada recurso— los que fijan los privilegios de cada carga de trabajo, los montajes que expone al nodo anfitrión y las relaciones de acceso entre recursos. Esa configuración se revisa habitualmente recurso a recurso, comprobando cada manifiesto contra un catálogo de reglas de buenas prácticas. El resultado es una visión fragmentada del riesgo, porque un compromiso real rara vez depende de una única configuración incorrecta y sí de la manera en que varias debilidades menores se encadenan a través de la estructura del clúster.

1.1. Motivación

La adopción masiva de Kubernetes como plataforma de orquestación de contenedores ha transformado la manera en que las organizaciones despliegan y gestionan sus aplicaciones. La encuesta anual de 2021 de la Cloud Native Computing Foundation (Fundación de Computación Nativa en la Nube) (CNCF) recoge que el 96 % de las organizaciones utilizan o evalúan Kubernetes (Cloud Native Computing Foundation, 2022). El informe sectorial *State of Kubernetes Security* de Red Hat de 2024 documenta, por su parte, que el 89 % de las organizaciones sufrió al menos un incidente de seguridad relacionado con Kubernetes durante el último año (Red Hat, 2024). Esa proliferación ha ampliado considerablemente la superficie de ataque: una sola configuración insegura en un manifiesto YAML puede habilitar la escalada de privilegios desde un contenedor comprometido hasta el nodo anfitrión del clúster o, en casos extremos, hasta la infraestructura subyacente completa (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022).

Los enfoques de análisis estático tradicionales —herramientas como Checkov (Bridgecrew, 2021) o kube-linter (StackRox, 2021)— detectan configuraciones incorrectas aisladas en

manifiestos individuales. Checkov incorpora además comprobaciones basadas en grafos sobre las dependencias entre recursos, pero se trata de reglas declarativas escritas manualmente: ninguna de estas herramientas dispone de un modelo aprendido de la estructura del clúster ni predice de forma supervisada la existencia de una cadena de ataque. Un único indicador de privilegio en un pod —la unidad mínima de despliegue de Kubernetes, que agrupa uno o varios contenedores— no es suficiente para determinar si existe una cadena de ataque viable. Lo que habilita rutas de explotación completas es la combinación de nodos con capacidad de escape, esto es, de salir del contenedor hacia el nodo anfitrión, con recursos que permiten el movimiento lateral hacia otras cargas de trabajo del clúster. Esa combinación sigue la lógica de encadenamiento de etapas del modelo de *kill chain* de Hutchins, Cloppert y Amin (2011). Las técnicas concretas de escalada de privilegios en contenedores están catalogadas en el marco MITRE ATT&CK for Containers (MITRE Corporation y Center for Threat-Informed Defense, 2021).

Herramientas más recientes como KubeHound (Datadog Security Labs, 2023) e IceKube (WithSecure Labs, 2023) abordan este problema construyendo grafos de ataque sobre clústeres activos. Requieren, sin embargo, recolectar el estado del clúster mediante acceso de lectura amplio a su servidor API de Kubernetes. Esa exigencia limita su aplicabilidad como control preventivo en el proceso de CI/CD, conforme al principio de *shift-left security*, que adelanta los controles de seguridad a las etapas más tempranas del desarrollo. Este trabajo aborda ese vacío: construir un sistema capaz de razonar sobre la *estructura* del clúster a partir de manifiestos YAML estáticos, sin requerir un clúster activo, para identificar automáticamente cadenas de ataque potenciales antes de que sean desplegadas.

1.2. Planteamiento del trabajo

Formalmente, dado un conjunto de manifiestos YAML que describen un clúster Kubernetes, el problema consiste en:

1. Clasificar cada manifiesto individual como seguro o mal configurado (Capa 1).
2. Modelar el clúster como un grafo heterogéneo y clasificarlo en una de tres categorías: *clean* (sin configuraciones erróneas relevantes), *isolated* (configuraciones erróneas sin

cadena viable) o *attack_chain* (existe una ruta de escalada de privilegios hasta movimiento lateral) (Capa 2).

3. Combinar las señales de ambas capas, junto con una señal binaria de escape derivada de las características de nodo, mediante una función de puntuación con pesos optimizados por Algoritmo Genético. El criterio de optimización es maximizar la Precisión@5 —la fracción de clústeres de cadena de ataque en el top-5 del *ranking*— minimizando los falsos positivos. Sobre el *ranking* final se aplica además una restricción de viabilidad estructural (Capa 3).

La capacidad de operar sobre manifiestos YAML estáticos —sin requerir acceso al clúster activo— es un requisito de diseño explícito que diferencia el presente trabajo de las soluciones de grafo de ataque existentes y lo posiciona como un control de seguridad integrable en *pipelines* CI/CD. En términos generales, el objetivo es diseñar, implementar y evaluar un sistema de detección de cadenas de ataque en Kubernetes que complemente los enfoques de análisis estático por manifiesto individual con el contexto estructural del clúster, mediante la combinación de aprendizaje automático clásico y aprendizaje sobre grafos. El sistema opera principalmente sobre manifiestos YAML estáticos, si bien incorpora de forma opcional una vía de consulta directa a un clúster activo. Los objetivos específicos que concretan este planteamiento general, junto con la metodología de seis etapas seguida para alcanzarlos, se detallan en el Capítulo 3.

1.3. Estructura del trabajo

El trabajo se organiza en siete capítulos. El Capítulo 2 revisa el estado del arte en seguridad de Kubernetes, herramientas de análisis estático y grafos de ataque, redes neuronales de grafos y detección de cadenas de ataque; incluye una tabla comparativa de las herramientas más relevantes y un análisis del posicionamiento de kubescan. El Capítulo 3 define los objetivos específicos y la metodología de seis etapas. El Capítulo 4 identifica y justifica los requisitos funcionales y no funcionales del sistema, delimita su alcance y describe la estrategia de partición de datos y las métricas de evaluación que esos requisitos determinan. El Capítulo 5 describe la herramienta desarrollada: la arquitectura de tres capas, la construcción del conjunto de datos y

el grafo, el diseño de cada capa, la implementación de kubescan como herramienta de línea de comandos, la tabla completa de las 28 características del clasificador y la calidad de software y el proceso de desarrollo seguidos. El Capítulo 6 evalúa el sistema frente a una línea de base del estado del arte, presenta los resultados de cada capa, sintetiza la métrica de proceso completo y verifica el cumplimiento de los requisitos, incluida la evaluación de usabilidad con usuarios expertos. El Capítulo 7 recoge las conclusiones vinculadas a cada objetivo, las limitaciones, las consideraciones éticas y las líneas de trabajo futuro.

2. Contexto y estado del arte

La detección automática de vulnerabilidades en infraestructuras Kubernetes se encuentra en la intersección de cuatro áreas de investigación activa: el análisis de seguridad en infraestructura como código (*Infrastructure as Code*, IaC), el aprendizaje automático aplicado a la ciberseguridad, las redes neuronales de grafos para razonamiento estructural y la construcción de grafos de ataque para la identificación de rutas de explotación. Cada una de estas áreas aporta una pieza del problema: el análisis estático delimita qué constituye una configuración errónea, el aprendizaje automático permite estimar el riesgo asociado a esos indicadores, las redes neuronales de grafos añaden razonamiento sobre las relaciones entre recursos y los grafos de ataque definen qué combinaciones de esas relaciones constituyen una ruta de explotación viable. Ninguna de las cuatro líneas ha producido hasta la fecha un sistema que prediga cadenas de ataque a partir de manifiestos estáticos mediante un modelo supervisado, carencia que delimita el espacio en el que se sitúa la contribución de este trabajo.

2.1. Contexto del problema: seguridad en infraestructuras Kubernetes

Kubernetes es la plataforma de orquestación de contenedores dominante en entornos de producción empresariales; las cifras de adopción y de incidencia se recogen en el Capítulo 1 (Cloud Native Computing Foundation, 2022; Red Hat, 2024). A pesar de esa adopción generalizada, la seguridad sigue siendo uno de los principales retos operativos. El informe *State of Kubernetes Security* de Red Hat 2024 (Red Hat, 2024) detalla el coste de esa exposición: el 67 % de las organizaciones retrasó despliegues a producción por preocupaciones de seguridad y el 46 % declaró pérdidas de clientes o de ingresos directamente atribuibles a incidentes en entornos de contenedores. Entre los riesgos más temidos, las vulnerabilidades representan el 33 % de las preocupaciones de los equipos, mientras que las configuraciones erróneas y exposiciones alcanzan el 27 %.

Su modelo declarativo, basado en ficheros YAML que especifican el estado deseado del clús-

ter, ofrece grandes ventajas en reproducibilidad y automatización, pero también introduce una amplia superficie de ataque: un fichero mal configurado desplegado en producción puede conceder privilegios excesivos a un contenedor o exponer secretos en texto plano. La flexibilidad de Kubernetes —que permite configurar desde el sistema de ficheros del anfitrión hasta los permisos de red a nivel de pod— implica que el número de vectores de ataque crece proporcionalmente con la riqueza del modelo de configuración. La Agencia de Seguridad Nacional de Estados Unidos (NSA) y la Agencia de Ciberseguridad e Infraestructura (CISA) publicaron en 2022 una guía técnica (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022) que enumera las configuraciones erróneas más críticas: uso de `privileged: true`, montaje del socket de Docker, compartición del espacio de nombres de proceso del anfitrión (*hostPID*) y automontaje de *tokens* de cuenta de servicio con permisos excesivos. Una cuenta de servicio es la identidad que Kubernetes asigna a un pod para autenticarse ante el servidor API, y su *token* es la credencial que la materializa. Estos vectores son precisamente los que este trabajo modeliza como nodos de escape en el grafo del clúster.

Los trabajos de BishopFox (Art, 2021) demostraron empíricamente que un único pod con configuración *everything-allowed* puede comprometer el nodo anfitrión en segundos y pivotar lateralmente a través del clúster. La plataforma educativa Kubernetes Goat (Akula, 2020) sistematiza más de veinte escenarios de vulnerabilidad con los que se pueden reproducir rutas de explotación completas en un entorno controlado. Ambas constituyen fuentes de manifiestos del presente trabajo; las etiquetas de cadena de ataque no proceden de ellas, sino de la regla explícita de etiquetado descrita en el Capítulo 5.

El marco taxonómico de referencia para los ataques en entornos de contenedores y Kubernetes es *MITRE ATT&CK for Containers* (MITRE Corporation y Center for Threat-Informed Defense, 2021), incorporado en la versión 9 de la base de conocimiento ATT&CK. Este marco describe las tácticas y técnicas empleadas por adversarios a nivel de orquestador (Kubernetes) y a nivel de contenedor (Docker), incluyendo las que este trabajo detecta directamente: escape de contenedor hacia el nodo anfitrión (T1611, *Escape to Host*), movimiento lateral mediante el uso de *tokens* de cuentas de servicio (T1550.001, *Use Alternate Authentication Material: Application Access Token*) y abuso de cuentas válidas (T1078, *Valid Accounts*). La alineación entre las banderas de escape del presente sistema y las técnicas ATT&CK valida que el espacio de detección

cubre los vectores de ataque documentados y más explotados en la industria.

2.2. Análisis estático de IaC

El análisis estático de manifiestos IaC cuenta con una larga trayectoria en la literatura. Rahman et al. (2023) construyeron un conjunto de datos de referencia para manifiestos Kubernetes extraídos de repositorios públicos, con 2.039 manifiestos de 92 repositorios etiquetados en 11 categorías de *misconfiguration* identificadas mediante análisis cualitativo y cuantificadas con su herramienta de análisis estático (SLI-KUBE). Su aportación se circunscribe a la construcción del conjunto de datos y a la medición de la densidad de configuraciones erróneas mediante reglas estáticas: no incluye ningún modelo de clasificación supervisada entrenado sobre ese corpus, por lo que no ofrecen una cifra de rendimiento predictivo con la que contrastar directamente. De ese trabajo, el presente sistema recupera y codifica 1.906 manifiestos de GitHub y GitLab como vectores de características binarias que indican la presencia o ausencia de cada patrón.

Las herramientas industriales actuales más utilizadas son Checkov (Bridgecrew, 2021), que verifica más de mil políticas de seguridad predefinidas, y kube-linter (StackRox, 2021), especializado en buenas prácticas de Kubernetes. kube-linter opera al nivel de manifiesto individual. Checkov incorpora además comprobaciones basadas en grafos sobre las dependencias entre recursos: por ejemplo, la escalada de privilegios a través de un RoleBinding, el objeto que vincula un rol de Role-Based Access Control (Control de Acceso Basado en Roles) (RBAC) con la identidad que lo ejerce. Se trata, no obstante, de reglas declarativas escritas manualmente, sin un modelo aprendido de la estructura del clúster ni predicción supervisada de cadenas de ataque. También destaca Kubescape (ARMO Ltd., 2021), una plataforma de seguridad de código abierto integrada en la CNCF que combina análisis estático con validación de cumplimiento contra los marcos NSA, MITRE ATT&CK y Center for Internet Security (Centro para la Seguridad en Internet) (CIS) *Benchmarks*. Al igual que Checkov y kube-linter, sin embargo, Kubescape evalúa recursos de forma independiente y no construye ningún modelo del clúster como estructura relacional.

La noción de un índice unificado de configuraciones erróneas (*unified misconfiguration index*), propuesta por Malul et al. (2024) en GenKubeSec, normaliza y unifica las etiquetas de múltiples

herramientas de análisis basadas en reglas para poder compararlas. Es un problema afín al de generar un *risk_score* unificado como el de la Capa 1 del presente sistema. La fuente original emplea esa expresión descriptiva sin acuñar una sigla, por lo que la abreviatura UMI que se usa en el resto de este documento es una convención propia. Su propuesta, que incorpora un modelo de lenguaje de gran escala (LLM) para la clasificación, es complementaria a la de este trabajo, aunque opera en el mismo nivel de manifiesto individual. La limitación estructural común a estas herramientas es que ninguna aprende ni predice cadenas de ataque a partir de la estructura del clúster: incluso las comprobaciones de grafo de Checkov son reglas fijas sobre dependencias conocidas, no un modelo entrenado. Estas herramientas detectan que un pod tiene `privileged: true`, pero no infieren si ese pod encadena con el acceso de red a los recursos críticos del clúster para formar una ruta de ataque completa. La detección de cadenas de ataque requiere, inevitablemente, razonar sobre la estructura del clúster como un todo.

2.3. Aprendizaje automático para detección de vulnerabilidades

El aprendizaje automático se ha aplicado a la detección de vulnerabilidades en código fuente y configuraciones desde hace más de una década. Los clasificadores basados en Random Forest (Breiman, 2001) —que combinan múltiples árboles de decisión mediante *bagging* y selección aleatoria de características— han demostrado resultados robustos en escenarios de alta dimensionalidad y clases desbalanceadas, características típicas de los conjuntos de datos de seguridad; el desequilibrio se aborda además mediante la ponderación de clases que ofrece su implementación en scikit-learn (Pedregosa et al., 2011). En la literatura, el ajuste de hiperparámetros de este tipo de clasificadores suele realizarse mediante búsqueda en rejilla (*grid search*) con validación cruzada estratificada sobre parámetros como `n_estimators` o `min_samples_leaf`. El presente trabajo no realiza tal búsqueda: emplea una única configuración fija de hiperparámetros, escogida para el escenario de clases desbalanceadas y documentada en el Capítulo 5, de modo que los resultados de la Capa 1 no deben interpretarse como el óptimo de un espacio de hiperparámetros explorado.

En el dominio de la detección de intrusiones en red, el conjunto de datos UNSW-NB15 (Moustafa y Slay, 2015) se ha convertido en un referente de evaluación para clasificadores supervisados. Conviene precisar el alcance de esa referencia: el trabajo que lo presenta es un artículo

de construcción de conjunto de datos, y su evaluación comparativa se limita a la exactitud y la tasa de falsas alarmas de árboles de decisión, regresión logística, *naïve Bayes*, redes neuronales artificiales y modelos de mezclas esperadas, sin consignar valores de F1 macro ni resultados de Random Forest. No constituye, por tanto, una revisión del rendimiento de Random Forest sobre ese conjunto ni permite derivar de él un rango de referencia para dicha métrica. El conjunto de manifiestos de este trabajo presenta un desequilibrio de clases moderado (56,3 % seguros frente a 43,7 % mal configurados); el desafío de desequilibrio más severo se concentra en las banderas de escape individuales, algunas presentes en menos del 1 % de los manifiestos reales. El desequilibrio de clases es un problema generalizado en los conjuntos de datos de seguridad: las configuraciones erróneas críticas son rarísimas en repositorios reales, lo que obliga a adoptar estrategias explícitas de compensación. La alternativa más común es asignar pesos de clase inversamente proporcionales a su frecuencia (*class_weight=balanced* en scikit-learn), que es la estrategia adoptada en este trabajo (Pedregosa et al., 2011). Otras alternativas como SMOTE (*Synthetic Minority Over-sampling Technique*) no se consideraron apropiadas dado que las características son binarias, y la interpolación lineal entre muestras binarias puede generar vectores fuera del espacio semántico válido.

Trabajos más recientes han explorado el uso de modelos de lenguaje de gran escala (LLM) para la detección y remediación de configuraciones erróneas en Kubernetes (Malul et al., 2024). Si bien los LLM ofrecen capacidades de razonamiento semántico superiores, requieren recursos computacionales sustancialmente mayores, producen salidas no deterministas y son más opacos que los clasificadores clásicos, lo que dificulta su interpretación en auditorías de seguridad donde la trazabilidad de las reglas de detección es un requisito normativo. Para entornos de producción donde el operador de seguridad debe poder justificar ante una auditoría por qué un clúster fue marcado como de alto riesgo, la interpretabilidad del Random Forest —mediante importancia de características y reglas de árbol legibles— representa una ventaja operativa significativa sobre los modelos de caja negra.

La aplicación de técnicas de Machine Learning (Aprendizaje Automático) (ML) a la predicción de vulnerabilidades en software es un enfoque consolidado que supera de forma consistente a los modelos estadísticos clásicos (Jabeen et al., 2022). Dentro de ese espacio, el Random Forest resulta especialmente adecuado para el problema de la Capa 1 por su comportamiento

sobre características binarias de alta dimensión con clases desbalanceadas. Ese comportamiento, unido a sus ventajas de interpretabilidad en auditorías de seguridad, motivó su adopción como componente de Capa 1 de kubescan.

La elección de Random Forest sobre otros algoritmos clásicos para la clasificación de manifiestos laC responde a tres criterios derivados de la naturaleza de los datos del problema:

1. **Robustez ante datos binarios y escasos.** Las características del presente trabajo son en su mayoría binarias (presencia/ausencia de una configuración errónea), con alta proporción de ceros. Los Random Forest manejan este tipo de datos de forma nativa sin requerir normalización, a diferencia de las Máquinas de Vectores Soporte (SVM) o las Redes Neuronales, sensibles a la escala de las características.
2. **Gestión intrínseca del desequilibrio de clases.** El parámetro `class_weight=balanced` de scikit-learn implementa una estrategia de ponderación inversamente proporcional a la frecuencia de clase, lo que eleva efectivamente el coste de clasificar erróneamente las muestras de la clase minoritaria (configuraciones erróneas críticas) sin modificar la función de pérdida subyacente (Pedregosa et al., 2011).
3. **Generación de puntuaciones continuas utilizables.** El promedio de los votos del *ensemble* de árboles produce puntuaciones suaves en $[0, 1]$ que pueden utilizarse directamente como características continuas (*risk_scores*) en el grafo de la Capa 2. Un Árbol de Decisión individual concentra su salida en valores próximos a los extremos, al derivarse de la composición de una única hoja, y los *k*-Nearest Neighbours la restringen a múltiplos de $1/k$, de modo que ambos ofrecen un rango efectivo mucho más pobre como señal continua. El requisito que se persigue aquí es la granularidad y el ordenamiento de la puntuación, no su interpretación como probabilidad en sentido frecuentista, que exigiría un procedimiento de calibración explícito no aplicado en este trabajo.

La Tabla 1 resume esta comparativa entre algoritmos candidatos para la Capa 1.

Tabla 1. Comparativa de algoritmos de ML para clasificación de manifiestos IaC

Algoritmo	Sin norma- lización	Desequilibrio	Puntuación continua	Observación
Random Forest	✓	✓	Suave	Adoptado (Capa 1)
Árbol de Decisión	✓	Parcial	Casi binaria	Alta varianza
SVM (RBF)	No	Parcial	Margen sin acotar	Entrenamiento $O(n^2)$ – $O(n^3)$; viable pero menos eficiente
Red Neuronal (MLP)	No	Con pesos	Variable	Sensible a la escala de entrada
k -NN	No	No	Múltiplos de $1/k$	Costoso en inferencia

Fuente: elaboración propia.

2.4. Redes neuronales de grafos

Las redes neuronales de grafos (*Graph Neural Networks*, GNN) han emergido como la herramienta principal para aprendizaje en datos con estructura relacional (Hamilton, 2020; Zhou et al., 2020). A diferencia de los clasificadores tabulares clásicos, las GNN operan directamente sobre la topología del grafo, propagando información entre vecinos mediante operaciones de paso de mensajes (*message passing*). Formalmente, en una capa GNN la representación del nodo v se actualiza como:

$$\mathbf{h}_v^{(l+1)} = \sigma(\mathbf{W}^{(l)} \cdot \text{AGGREGATE}(\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\}) + \mathbf{b}^{(l)}) \quad (1)$$

donde $\mathcal{N}(v)$ es el conjunto de vecinos de v , $\mathbf{W}^{(l)}$ y $\mathbf{b}^{(l)}$ son parámetros entrenables de la capa l , y σ es una función de activación no lineal. La elección de la función AGGREGATE diferencia las principales arquitecturas.

Las arquitecturas más relevantes para este trabajo son:

Graph Convolutional Networks (GCN) (Kipf y Welling, 2017): implementan AGGREGATE como la media ponderada de los estados de los vecinos normalizada por sus grados. Son eficientes, pero los pesos quedan fijados por la estructura del grafo (los grados de los nodos) y no se aprenden, sin distinción entre tipos de arista.

GraphSAGE (Hamilton et al., 2017): extensión inductiva que permite aplicar el modelo a grafos no vistos durante el entrenamiento, relevante en escenarios donde aparecen nuevos clústeres en producción. La inducción se logra aprendiendo funciones de agregación en lugar de *embeddings* fijos.

Graph Attention Networks (GAT) (Veličković et al., 2018): sustituyen la normalización fija de GCN por coeficientes de atención aprendidos. Para cada par de nodos (v, u) el coeficiente de atención es:

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]))}{\sum_{k \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_k]))} \quad (2)$$

donde $\mathbf{a}$ es un vector de parámetros de atención y $\parallel$ denota concatenación. Este diseño es especialmente apropiado para grafos de clúster Kubernetes, donde una arista de privilegio (*privilege_reach*) debe pesar más que una arista de proximidad de directorio (*dir_proximity*). El modelo *How Powerful are GNNs?* de Xu et al. (2019) establece que la capacidad discriminativa de las GNN está acotada por el test de isomorfismo de Weisfeiler-Lehman, y que la suma de vecinos maximiza el poder expresivo frente a la media o el máximo. En este trabajo se adopta un *pooling* combinado de media y máximo por su robustez ante grafos de tamaño muy variable, aceptando explícitamente ese compromiso de expresividad.

2.5. GNN aplicadas a ciberseguridad

La aplicación de GNN a la detección de amenazas es un campo emergente con rápido crecimiento. Bilot et al. (2023) proporcionan una revisión exhaustiva de los enfoques basados en GNN para la detección de intrusiones en red, organizada en torno a enfoques basados en red (grafos de flujo) y basados en host (grafos de procedencia). Una segunda revisión publicada

en *Computers & Security* en 2024 (Zhong et al., 2024) amplía esta taxonomía y documenta un crecimiento sostenido de la investigación en el área en los últimos años.

E-GraphSAGE (Lo et al., 2022) propone representar flujos de red como grafos de transacción para clasificación de intrusiones en Internet of Things (Internet de las Cosas) (IoT), alcanzando tasas de detección superiores a los clasificadores tabulares clásicos sobre las repúblicas en formato NetFlow de los conjuntos de referencia habituales del área (NF-UNSW-NB15-v2, NF-ToN-IoT y NF-BoT-IoT), y no sobre la versión original de UNSW-NB15. La analogía con el presente trabajo es directa: los manifiestos Kubernetes son los nodos y las relaciones de privilegio y movimiento lateral son las aristas del grafo de amenaza. Una diferencia clave es que E-GraphSAGE modela flujos de tráfico en tiempo real, mientras que este trabajo modela configuraciones declarativas estáticas, lo que permite operar antes del despliegue.

Li et al. (2022) construyen grafos de conocimiento de técnicas de ataque a partir de informes de inteligencia de amenazas (CTI) y demuestran que el razonamiento estructural sobre relaciones entre técnicas mejora la predicción de campañas de ataque. Este enfoque comparte con el presente trabajo la intuición central de que el grafo de relaciones entre recursos es más informativo que la suma de sus nodos individuales. Aunque opera sobre conocimiento de ataques pasados (retrospectivo) y no sobre configuraciones declarativas (preventivo), la arquitectura basada en grafos de conocimiento inspiró el diseño de los tipos de arista semánticamente diferenciados del presente sistema. Una limitación reconocida en la literatura es la escasez de conjuntos de datos de grafos etiquetados de alta calidad para ciberseguridad. Tanto Bilot et al. (2023) como Zhong et al. (2024) sitúan la escasez de conjuntos de datos etiquetados y de *benchmarks* estandarizados entre los principales obstáculos para el avance del área. Esa carencia es la que refuerza la contribución del presente trabajo, que genera un conjunto de datos original de 599 grafos de clúster (1.154 tras aumentación de datos) con etiquetas de cadena de ataque.

2.6. Detección de cadenas de ataque y grafos de ataque en Kubernetes

La noción de cadena de ataque (*kill chain*) fue formalizada originalmente por Hutchins et al. (2011) para describir las fases secuenciales de una intrusión avanzada: reconocimiento, preparación del artefacto (*weaponization*), entrega, explotación, instalación, comando y control,

y acción. En el contexto de Kubernetes, esta cadena se concreta de forma más compacta: requiere como mínimo (1) un pod con capacidad de escape al anfitrión (*TRUE_HOST_PID*, *privileged: true*, montaje de *hostPath* —un directorio del nodo anfitrión expuesto dentro del contenedor—, etc.) y (2) un mecanismo de movimiento lateral hacia otros recursos del clúster (*token* de cuenta de servicio con permisos excesivos, montaje de secretos en el manifiesto, etc.). En los últimos años han surgido dos herramientas de código abierto que abordan la detección de rutas de ataque en Kubernetes mediante grafos: KubeHound (Datadog Security Labs, 2023) e IceKube (WithSecure Labs, 2023).

KubeHound (Datadog Security Labs, 2023) construye un grafo de ataque del clúster a partir de la consulta directa al servidor Application Programming Interface (Interfaz de Programación de Aplicaciones) (API) de Kubernetes, almacena el resultado en JanusGraph y permite explorar las rutas de ataque más cortas mediante el lenguaje de consulta Gremlin. El proyecto fue motivado por la misma intuición que subyace al presente trabajo, formulada por John Lambert y citada en su documentación: «*los defensores piensan en listas, los atacantes piensan en grafos*» (Datadog Security Labs, 2023). KubeHound cubre un amplio conjunto de ataques y ha sido adoptado por equipos de seguridad en producción.

IceKube (WithSecure Labs, 2023) sigue un enfoque análogo, basado en Neo4j, y permite definir rutas de ataque mediante consultas Cypher. Modela un amplio conjunto de técnicas de ataque como relaciones en el grafo y puede encontrar rutas desde cualquier identidad de bajo privilegio hasta *cluster-admin*. La diferencia fundamental entre estas herramientas y kubescan es de paradigma: KubeHound e IceKube operan sobre un **clúster activo**, requiriendo acceso con privilegios elevados a `kubectl`¹. La herramienta kubescan, en cambio, opera sobre **manifiestos YAML estáticos** antes del despliegue, integrándose en el *pipeline* CI/CD como un control de seguridad *shift-left*. Además, ambas herramientas utilizan búsqueda de rutas mediante consultas *ad hoc* en bases de datos de grafos, sin componente de aprendizaje automático supervisado; kubescan es el primer sistema, según la revisión bibliográfica no sistemática realizada en este trabajo, que combina análisis estático de manifiestos con GNN para la predicción de cadenas de ataque como tarea de clasificación supervisada.

2.7. Conclusiones del estado del arte

El análisis de la literatura permite identificar tres carencias fundamentales que motivan el presente trabajo. En primer lugar, las herramientas de análisis estático existentes (Checkov, kube-linter, Kubescape) no aprenden ni predicen cadenas de ataque a partir de la estructura del clúster; aunque Checkov incorpora comprobaciones de grafo basadas en reglas declarativas sobre dependencias entre recursos, ninguna emplea un modelo supervisado que razone sobre las relaciones que habilitan dichas cadenas. En segundo lugar, los conjuntos de datos de referencia para seguridad de Kubernetes son escasos, de pequeño tamaño y desequilibrados hacia las clases de riesgo bajo, lo que dificulta el entrenamiento de clasificadores de clúster. El análisis del corpus de manifiestos reales recopilado en este trabajo (Capítulo 5) confirma ese desequilibrio: `TRUE_HOST_PID` —la bandera de escape más peligrosa— aparece en solo el 0,3 % de las filas antes de incorporar fuentes especializadas. En tercer lugar, las herramientas de grafo de ataque más avanzadas (KubeHound, IceKube) requieren un clúster activo, lo que impide su uso como control preventivo en el *pipeline* CI/CD.

La Tabla 2 resume el posicionamiento de kubescan frente a las principales herramientas del estado del arte.

Tabla 2. Comparativa de herramientas de seguridad para Kubernetes

Herramienta	Análisis estático	Requiere clúster	Grafos	ML supervisado
Checkov (Bridgecrew, 2021)	✓	No	Parcial	No
kube-linter (StackRox, 2021)	✓	No	No	No
Kubescape (ARMO Ltd., 2021)	✓	Opcional	No	No
KubeHound (Datadog Security Labs, 2023)	No	Sí	✓	No
IceKube (WithSecure Labs, 2023)	No	Sí	✓	No
kubescan	✓	No	✓	✓

Fuente: elaboración propia.

El sistema kubescan presentado en este trabajo responde a las tres carencias identificadas: adopta una arquitectura de grafo heterogéneo con tipos de arista semánticamente significativos para el dominio Kubernetes, combina el análisis estático (Capa 1) con el razonamiento

estructural (Capa 2) en un *ensemble* optimizado (Capa 3), y aporta un conjunto de datos de 599 grafos de clúster originales con etiquetas de cadena de ataque derivadas de reglas de seguridad explícitas y auditables. Al operar sobre manifiestos YAML sin requerir acceso al clúster, kubescan se posiciona como un control preventivo integrable en *pipelines* CI/CD, complementario a —y no sustituto de— las herramientas de análisis de clúster activo como KubeHound. Los capítulos siguientes describen en detalle los objetivos específicos, el diseño e implementación del sistema y la evaluación experimental de cada una de sus tres capas.

3. Objetivos concretos y metodología de trabajo

Este capítulo define los objetivos del trabajo y describe la metodología seguida para alcanzarlos. En primer lugar se formulan el objetivo general y los objetivos específicos; los de naturaleza cuantificable incorporan criterios de éxito medibles. A continuación se describen las seis etapas del proceso de trabajo que estructuran el resto del documento. Estas etapas son independientes de las fases experimentales de entrenamiento de la Capa 2 numeradas en el Capítulo 6, que responden a una progresión metodológica distinta y no guardan correspondencia uno a uno con ellas.

3.1. Objetivo general

Diseñar, implementar y evaluar un sistema automático de predicción de cadenas de ataque en infraestructuras Kubernetes que complemente los enfoques de análisis estático por manifiesto individual con el contexto estructural del clúster, mediante la combinación de un clasificador de manifiestos individuales (Random Forest) con una red neuronal de grafos (*Graph Attention Network*) y optimización de *ensemble* mediante Algoritmo Genético. El sistema opera principalmente sobre manifiestos YAML estáticos, sin requerir acceso a un clúster activo, si bien incorpora opcionalmente una vía de consulta directa al clúster (OE6). Los objetivos específicos que concretan este objetivo general, numerados OE1 a OE6, se detallan a continuación.

3.2. Objetivos específicos

Los criterios de éxito de los objetivos específicos se expresan mediante tres métricas —F1 macro, Precisión@5 y FPR_clean— cuya definición formal y justificación frente a alternativas de uso más extendido se recogen en la Sección 4.4. Los umbrales numéricos asociados a esos criterios son criterios de diseño adoptados en este trabajo, no cotas tomadas de una referencia externa. Cada objetivo indica, junto a su criterio, la sección que aporta la evidencia de cumplimiento.

- OE1.** Construir un conjunto de datos de manifiestos Kubernetes etiquetados que combine múltiples fuentes públicas, contenga al menos 2.000 manifiestos y alcance una cobertura mínima del 2 % en la bandera de escape `TRUE_HOST_PID`, superando el 0,3 % que presentan los manifiestos procedentes de repositorios reales. Ambas cifras son criterios de diseño fijados en este trabajo. El mínimo de 2.000 manifiestos responde al tamaño necesario para sostener una validación cruzada de 5 pliegues manteniendo representación de la clase minoritaria en cada partición. El suelo del 2 % se estableció para garantizar un número suficiente de manifiestos con capacidad de escape, porque la regla de etiquetado de grafos marca un clúster como *attack_chain* en dos supuestos, ambos dependientes de la presencia de nodos con esa capacidad: cuando el clúster contiene dos o más nodos con alguna bandera de escape activa, situación en la que la escalada compuesta es posible sin necesidad de un salto lateral explícito, o cuando existe una ruta de alcance sobre aristas de escalada que enlaza un nodo de escape con un nodo de movimiento lateral. Con la cobertura del 0,3 % observada en los repositorios reales sin aportación de fuentes intencionalmente vulnerables, ninguno de los dos supuestos llega a activarse con frecuencia suficiente, de modo que la clase minoritaria quedaría sin ejemplos bastantes para entrenar la Capa 2. La composición final del conjunto y la cobertura alcanzada se detallan en la Sección 5.2.
- OE2.** Desarrollar una Capa 1 de clasificación de manifiestos individuales basada en Random Forest con F1 macro superior a 0,85, que produzca *risk scores* continuos en el rango [0,1] destinados a alimentar el vector de nodo de la Capa 2 y el *ensemble* de la Capa 3. En el sistema distribuible ambos consumos se satisfacen con la predicción del clasificador; en el corpus de entrenamiento, en cambio, ese canal quedó ocupado por una heurística determinista, circunstancia analizada como limitación en el Capítulo 7. El umbral de 0,85 se adopta como criterio de diseño propio de este trabajo, no derivado de una referencia externa: la literatura de detección de intrusiones sobre conjuntos comparables no documenta rangos de F1 macro para Random Forest que puedan trasladarse a este dominio (Sección 2.3). La Sección 6.2 sitúa además el resultado obtenido frente a una línea de base trivial calculada sobre el propio corpus, que constituye la comparación informativa.
- OE3.** Diseñar un esquema de grafo de clúster con tipos de arista semánticamente significati-

vos para el dominio Kubernetes: proximidad de directorio, alcance de privilegio, movimiento lateral mediante cuenta de servicio, coespacio de nombres semántico y escalada RBAC. El criterio de éxito se formula en el plano de la arquitectura: la configuración adoptada —GAT con *embedding* de tipos de arista— debe igualar o superar en F1 macro y Precisión@5 a una GCN sin tipado de arista bajo idéntico protocolo de evaluación. Este contraste varía simultáneamente el mecanismo de agregación (atención frente a convolución) y la presencia de tipado de arista, por lo que un resultado favorable acredita la configuración completa pero no permite atribuir la ventaja al esquema de tipos de arista por separado. Aislar esa contribución exigiría una tercera rama de ablación —GAT sin tipos de arista— que no forma parte del conjunto de configuraciones entrenadas en este trabajo y queda como línea de trabajo futuro (Capítulo 7). La evidencia experimental correspondiente se recoge en la Tabla 10.

- OE4.** Desarrollar una Capa 2 de clasificación de clústeres basada en GAT que alcance Precisión@5 superior a 0,70, medida como media de la validación cruzada de 5 pliegues sobre el modelo desplegado con la semilla documentada. El umbral de 0,70 es un criterio de diseño propio de este trabajo y no una cota externa: con $k = 5$ equivale a que, en promedio, entre tres y cuatro de los cinco clústeres priorizados sean cadenas de ataque reales, de forma que la mayor parte del esfuerzo de revisión manual recaiga sobre hallazgos verdaderos. El criterio se evalúa sobre esa configuración concreta y admite, por tanto, resultado negativo si la media de sus pliegues queda por debajo del umbral. De forma complementaria —y sin formar parte del criterio— se presenta la dispersión entre semillas de entrenamiento, cuya media puede situarse por debajo de 0,70 sin invalidar el objetivo; la Sección 6.3.6 la cuantifica y el Capítulo 7 discute su alcance. El resultado del modelo desplegado se presenta en la Sección 6.3.
- OE5.** Implementar una Capa 3 que optimice mediante Algoritmo Genético una función objetivo combinada de Precisión@5 y falsos positivos de clústeres limpios sobre las predicciones *out-of-fold* de la validación cruzada, con criterios de éxito sobre el conjunto de test reservado: FPR_clean = 0 y Precisión@5 no inferior a la del mejor componente individual del *ensemble*. La verificación de ambos criterios se aporta en la Sección 6.4.
- OE6.** Empaquetar el sistema completo como una herramienta de línea de comandos (kubes-

can) con soporte para análisis de directorios locales y, opcionalmente, consulta directa al API de Kubernetes mediante `kubectl`; instalable mediante un único `pip install` en entornos Python 3.10 o superior, con verificación efectiva sobre las versiones 3.10 y 3.11. La evaluación funcional y de instalabilidad se documenta en la Sección 6.8 y la evaluación con usuarios expertos en la Sección 6.9.

La correspondencia entre estos objetivos y los requisitos que los operacionalizan se recoge en la Tabla 15 del Anexo A, que asocia cada requisito con la sección de diseño que lo materializa y con la sección de evaluación que aporta su evidencia. El Capítulo 7 cierra el ciclo enunciando, para cada objetivo específico, la conclusión que lo responde. Ambos elementos permiten recorrer la cadena objetivo–requisito–diseño–evidencia en los dos sentidos.

3.3. Metodología del trabajo

El trabajo sigue un proceso iterativo de seis etapas definido específicamente para este proyecto, sin adoptar ninguna metodología estandarizada de referencia, que articula las actividades propias del desarrollo de software (diseño, implementación y prueba) con las del proceso de ciencia de datos (adquisición, preparación, modelado y evaluación). La especificación de requisitos antecede a las seis etapas y se documenta de forma independiente en el Capítulo 4, donde se fijan los requisitos funcionales y no funcionales junto con sus criterios de aceptación. Las etapas se enumeran a continuación en el orden en que se ejecutaron:

Etap 1 — Adquisición de datos. Descarga y normalización de manifiestos Kubernetes de cinco fuentes públicas complementarias (agrupadas en cuatro categorías): el conjunto de datos de Rahman et al. (2023) (repositorios reales etiquetados), BishopFox BadPods (Art, 2021) (pods intencionadamente vulnerables), Kubernetes Goat (Akula, 2020) (escenarios de vulnerabilidad) y repositorios de seguridad adicionales (CTF, laboratorios y *fixtures*). El criterio de inclusión de repositorios adicionales fue la presencia de al menos una bandera de escape activa (`HOSTPATH_MOUNT`, `TRUE_HOST_PID`, etc.) en al menos un manifiesto del directorio. Este criterio de selección se adoptó porque los directorios sin ninguna bandera de escape activa no pueden originar clústeres de la clase *attack_chain* bajo la regla de etiquetado empleada, de modo que su incorporación aumentaría el volumen del conjunto sin aportar ejemplos de la clase mi-

noritaria que la Capa 2 debe aprender a discriminar. La contrapartida es un sesgo de muestreo que se documenta como decisión de selección deliberada: los 779 manifiestos incorporados por esta vía sobrerrepresentan configuraciones vulnerables respecto de la distribución esperable en un clúster de producción, por lo que las tasas de prevalencia del corpus no admiten lectura como estimaciones poblacionales. Las fuentes de repositorios reales se incorporaron sin aplicar este filtro, lo que acota el sesgo a una fracción identificable del conjunto.

Etapla 2 — Extracción de características y construcción del grafo. Extracción de 28 características por manifiesto (25 indicadores binarios y 3 agregados derivados) mediante análisis estático multiherramienta (Sección 5.8) y construcción del grafo de clúster con cinco tipos de arista (Tabla 4). En esta etapa se realizó también la validación de cobertura de la bandera `HOSTPATH_MOUNT`, documentada en la Sección 5.2.

Etapla 3 — Entrenamiento de la Capa 1. Entrenamiento de un clasificador Random Forest para predecir la presencia de configuraciones erróneas individuales, con los hiperparámetros fijos detallados en la Sección 5.4 y la validación cruzada de 5 pliegues descrita en la Sección 4.3. La puntuación por manifiesto que acompaña a cada fila del conjunto tabular no procede de este clasificador, sino de la heurística ponderada documentada en la Sección 5.3.

Etapla 4 — Entrenamiento de la Capa 2. Entrenamiento de una GAT de 3 capas con agrupamiento de grafos (*graph pooling*) y validación cruzada estratificada de 5 pliegues consciente de grupos y de familias de plantilla, según el procedimiento que detalla la Sección 5.5.3. La aumentación de datos se aplica sobre los grafos de entrenamiento para mitigar la escasez de ejemplos de cadena de ataque, y el *checkpoint* de cada pliegue se selecciona por Precisión@5 de validación.

Etapla 5 — Optimización de *ensemble* y evaluación. Optimización de los pesos del *ensemble* de tres componentes (Random Forest, GNN, señal de escape) mediante Algoritmo Genético sobre las predicciones *out-of-fold* de la validación cruzada de la Capa 2, aplicación de una restricción de viabilidad estructural en el *ranking* final (un grafo de un solo nodo no puede albergar una cadena de ataque multisalto) y evaluación final sobre el conjunto de test reservado desde el inicio y no utilizado en ninguna etapa de entrenamiento ni selección de hiperparámetros.

Etapas 6 — Empaquetado y evaluación funcional y de usabilidad. Empaquetado del sistema como herramienta de línea de comandos instalable (kubescan), evaluación funcional de rendimiento e integración (latencia, formatos de salida, resolución de *checkpoints*) y estudio de usabilidad y aplicabilidad con usuarios expertos mediante el cuestionario SUS y un conjunto de tareas guiadas; los protocolos y resultados de esta etapa se detallan en el Capítulo 6.

4. Identificación de requisitos

Este capítulo identifica y justifica los requisitos que condicionaron el diseño de kubescan. Los requisitos funcionales y no funcionales se derivan del análisis de vectores de ataque documentado en el estado del arte y de las limitaciones detectadas en las herramientas existentes. Delimitan además el alcance del sistema: qué analiza, sobre qué capas se apoya y qué queda fuera de sus objetivos. El capítulo cierra con dos estudios previos que determinaron directamente el diseño de la herramienta: la estrategia de partición y validación de datos y las métricas empleadas para evaluarla.

4.1. Requisitos del sistema

El diseño de kubescan parte de un análisis de requisitos derivado tanto de las limitaciones identificadas en el estado del arte (Sección 2.7) como de los vectores de ataque documentados por la NSA y la CISA (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022). Este análisis se traduce en dos conjuntos de requisitos: funcionales, que definen las capacidades que el sistema debe ofrecer, y no funcionales, que acotan sus propiedades de calidad. Ambos conjuntos se detallan a continuación, junto con la justificación concreta de cada uno.

4.1.1. Requisitos funcionales

RF-1 — Detección de configuraciones erróneas individuales. El sistema debe cumplir dos obligaciones separables sobre cada manifiesto YAML. La primera es clasificarlo como seguro o mal configurado —*misconfigured* en la nomenclatura del modelo—. La segunda es producir un índice de severidad en tres niveles (limpio, bajo-medio, alto-crítico) que ordene las configuraciones erróneas detectadas por gravedad. *Criterio de aceptación:* ambas obligaciones se consideran satisfechas cuando cada una de las dos clasificaciones alcanza F1 macro superior a 0,85 sobre el conjunto de test reservado, umbral adoptado de OE2 (Capítulo 3) y aplicado también al índice de severidad por tratarse del mismo

clasificador sobre el mismo corpus. *Justificación:* las herramientas de análisis estático existentes (Checkov, kube-linter) ya cubren esta capacidad por manifiesto individual (Sección 2.2); el sistema debe igualarla como condición previa a razonar sobre la estructura del clúster, y el índice de severidad aporta priorización que las herramientas de reglas fijas no ofrecen.

RF-2 — Detección de cadenas de ataque. El sistema debe clasificar un conjunto de manifiestos que describe un clúster en una de tres categorías: *clean*, *isolated* (configuraciones erróneas sin cadena viable) o *attack_chain* (ruta de escalada de privilegios hasta movimiento lateral). *Criterio de aceptación:* toda ejecución debe emitir exactamente una de esas tres categorías por clúster analizado, expuesta en la salida JSON, y las cadenas de ataque del conjunto de test reservado deben quedar situadas en las primeras posiciones del *ranking* de riesgo con arreglo al umbral fijado en RF-5. La proporción de cadenas clasificadas correctamente por *argmax* se consigna como métrica descriptiva del comportamiento del modelo y no constituye por sí sola condición de aceptación, dado que el caso de uso operativo del sistema es la priorización y no el etiquetado exacto. *Justificación:* es la carencia central identificada en el estado del arte (Sección 2.7): ninguna herramienta de análisis estático por manifiesto individual emplea un modelo supervisado que razone sobre las relaciones entre recursos que habilitan una cadena, y las comprobaciones de grafo basadas en reglas declarativas que incorpora Checkov se limitan a dependencias conocidas de antemano.

RF-3 — Interfaz de línea de comandos. El sistema debe exponer un comando `kubescan scan <dir>` para análisis de directorios locales y un comando `kubescan live` para análisis de clústeres activos mediante `kubect1`. *Justificación:* un control de seguridad solo es adoptable en la práctica si se integra en los flujos de trabajo existentes de un equipo de seguridad o de CI/CD; una interfaz de línea de comandos es el punto de integración mínimo común a ambos.

RF-4 — Formatos de salida. El sistema debe producir informes en formato texto legible y en formato JSON estructurado para integración con herramientas externas. *Justificación:* el formato texto sirve a la revisión manual de un operador, mientras que el formato JSON permite el consumo automatizado del veredicto por otras herramientas del *pipeline* de

CI/CD.

RF-5 — Priorización de riesgos. El sistema debe producir una clasificación ordenada de clústeres por puntuación de riesgo, de modo que las cadenas de ataque reales queden situadas en las primeras posiciones. *Criterio de aceptación:* Precisión@5 superior a 0,70 sobre el *ranking* del sistema desplegado en el conjunto de test reservado. *Alcance del umbral:* la cifra de 0,70 se adopta de OE4 (Capítulo 3), donde se define como media de validación cruzada de 5 pliegues de la Capa 2; el presente requisito la traslada al *ranking* del sistema completo, de manera que ambas medidas comparten umbral pero no protocolo de cálculo. *Precisión estadística:* el tamaño del conjunto de test reservado (86 grafos, 5 cadenas) hace que el intervalo de confianza *bootstrap* al 95 % de la Precisión@5 abarque $[0,60; 1,00]$ y por tanto contenga el propio umbral, de modo que la superación del criterio debe leerse como coherente con el objetivo de diseño y no como una garantía estadística. *Justificación:* un equipo de seguridad revisa manualmente un número reducido de alertas por jornada; un *ranking* que sitúe las cadenas reales en las primeras posiciones hace viable esa revisión sobre un volumen de clústeres que de otro modo sería inabarcable.

4.1.2. Requisitos no funcionales

RNF-1 — Trazabilidad. Todas las reglas de detección de nodos de escape y movimiento lateral deben ser explícitas, auditables y derivadas de fuentes normativas documentadas (Art, 2021; National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022). *Criterio de aceptación:* cada veredicto debe poder rastrearse a las banderas concretas que lo originan, expuestas en la salida JSON junto a la identificación de los nodos con capacidad de escape, y cada regla debe remitir a una fuente normativa citable. *Justificación:* un sistema que informa decisiones de seguridad debe permitir que su veredicto sea auditado y cuestionado; basar cada regla en una fuente pública evita además introducir vectores de ataque no documentados.

RNF-2 — Reproducibilidad. Los modelos entrenados deben almacenarse como *checkpoints* que el paquete pueda localizar sin intervención del usuario y que se distribuyan junto al repositorio del proyecto, de forma que el análisis sea idempotente dado el mismo directorio de entrada. *Criterio de aceptación:* el requisito se descompone en dos condiciones

comprobables por separado. La primera es de disponibilidad: los pesos deben resolverse mediante la cadena documentada de resolución de *checkpoints* —opción explícita de línea de comandos, variable de entorno y ubicación por defecto dentro del paquete— y quedar publicados en el repositorio distribuido, sin descarga adicional por parte del usuario. La segunda es de determinismo: dos ejecuciones consecutivas sobre el mismo directorio de manifiestos y el mismo conjunto de pesos deben producir informes JSON idénticos. *Justificación*: un control de seguridad con salidas no deterministas no es fiable como *gate* de CI/CD —una comprobación que bloquea la entrega si falla—, donde el mismo manifiesto debe producir siempre el mismo veredicto.

RNF-3 — Instalabilidad. El sistema debe ser instalable mediante `pip install -e kubescan/` sin dependencias de compiladores externos. *Criterio de aceptación*: la instalación debe completarse sin error en intérpretes Python 3.10 o superior, cota declarada en OE6 (Capítulo 3) y en los metadatos de empaquetado, resolviendo todas las dependencias sin invocar un compilador del sistema. *Justificación*: reduce la fricción de adopción por un equipo de seguridad que no necesariamente dispone de un entorno de compilación preparado para dependencias nativas.

RNF-4 — Latencia. El análisis de un clúster de hasta 500 manifiestos debe completarse en menos de 60 segundos en hardware de gama media (CPU). *Criterio de aceptación*: la cota de 500 manifiestos se formula como límite de diseño y no como tamaño de un caso de prueba disponible, ya que el mayor clúster del corpus recopilado no alcanza esa dimensión. La verificación consiste, por tanto, en medir el tiempo de análisis sobre clústeres de tamaño creciente y comprobar que el coste por manifiesto observado mantiene el tiempo proyectado a 500 manifiestos por debajo del límite de 60 segundos. *Justificación*: un tiempo de análisis del orden de segundos es condición necesaria para que el sistema pueda operar como control preventivo dentro de un *pipeline* de CI/CD sin introducir un cuello de botella perceptible.

4.1.3. Trazabilidad de los requisitos

Cada requisito se materializa en una decisión de diseño concreta y se contrasta después con una evidencia experimental u operativa determinada. La Tabla 15 del Anexo A recoge esa correspon-

dencia, indicando para cada identificador la sección del Capítulo 5 que describe el componente encargado de satisfacerlo y la sección del Capítulo 6 que aporta la evidencia de su cumplimiento. Los requisitos que no admiten una métrica predictiva dedicada, como los de interfaz, formatos de salida y propiedades no funcionales, se concentran en la verificación operativa de la Sección 6.8.

El índice de severidad exigido por RF-1 se verifica de forma diferenciada respecto de la clasificación binaria, mediante los resultados por clase de la Tabla 6. La restricción de viabilidad estructural que condiciona el *ranking* evaluado en RF-5 se describe en la Sección 5.6.1 y su efecto se cuantifica en la Sección 6.4.

4.2. Justificación del alcance

El sistema opera principalmente sobre manifiestos YAML estáticos, sin requerir acceso a un clúster activo, aunque incorpora opcionalmente una vía de consulta directa al clúster mediante `kubescan live`. La elección del análisis estático como vía principal responde al principio *shift-left security*: detectar una cadena de ataque antes del despliegue evita el coste de remediarla en producción, mientras que las herramientas de grafo de ataque sobre clúster activo (KubeHound (Datadog Security Labs, 2023), IceKube (WithSecure Labs, 2023)) solo pueden operar con posterioridad al despliegue y requieren acceso con privilegios elevados a `kubectl` sobre un clúster ya en ejecución. La vía de consulta directa se mantiene como capacidad complementaria para auditorías puntuales sobre clústeres ya desplegados, no como sustituto de la vía preventiva.

La arquitectura de tres capas responde a una separación de responsabilidades entre dos niveles de análisis y su combinación: la Capa 1 evalúa cada manifiesto de forma aislada (RF-1), la Capa 2 razona sobre la estructura relacional del clúster que ningún manifiesto individual expone por sí solo (RF-2) y la Capa 3 combina ambas señales en un único veredicto priorizable (RF-5). Ninguna capa por sí sola satisface el conjunto completo de requisitos funcionales: un clasificador de manifiesto individual no puede expresar una relación entre dos recursos, y un modelo de grafo sin las características de nodo de la Capa 1 pierde la señal de riesgo específica de cada manifiesto. La elección de Random Forest para la Capa 1 está además condicionada por tres propiedades que la Tabla 1 contrasta entre algoritmos candidatos: su independencia

respecto de la normalización de escala de las entradas, su tolerancia al desequilibrio de clases y la granularidad de la puntuación continua que produce, requisito este último derivado de que dicha salida se emplea como característica de nodo en la Capa 2.

A las tres capas se añade un cuarto componente arquitectónico: la restricción de viabilidad estructural descrita en la Sección 5.6.1, que excluye del *ranking* de riesgo aquellos clústeres cuya topología no puede albergar una cadena de ataque, como los compuestos por un único recurso. Su inclusión no responde a una preferencia de diseño, sino a una necesidad verificada empíricamente: sin ella, la Precisión@1 sobre el conjunto de test reservado cae a 0,00 y la Precisión@5 a 0,60, valor este último situado por debajo del umbral que el propio RF-5 fija como criterio de aceptación. La restricción actúa así como condición previa de admisibilidad al *ranking*, complementaria a las señales aprendidas por los tres modelos y no sustitutiva de ninguna de ellas.

Quedan explícitamente fuera del alcance del sistema: la explotación activa de las cadenas de ataque detectadas (el sistema identifica rutas potenciales, no las verifica mediante ataque controlado); la remediación automática de las configuraciones erróneas encontradas; y el análisis de vulnerabilidades a nivel de imagen de contenedor (*CVE scanning*), cubierto por herramientas especializadas de escaneo de imágenes y complementario, no sustitutivo, del análisis estructural que aporta kubescan.

4.3. Estrategia de partición y validación de datos

La estrategia de partición de datos difiere entre la Capa 1 (datos tabulares) y la Capa 2 (datos de grafos), dado que sus tamaños y distribuciones son distintos. Esta estrategia condiciona directamente la comprobabilidad de RF-1 y RF-2, cuyos criterios de aceptación se formulan sobre un conjunto de test reservado, y de RF-5, cuyo umbral de Precisión@5 se mide sobre el *ranking* producido para ese mismo conjunto. El tamaño final de cada partición depende del proceso de adquisición y construcción descrito en las Secciones 5.2 y 5.3 del Capítulo 5.

Para la Capa 1, el conjunto de manifiestos se divide en un conjunto de entrenamiento (80 %) y un conjunto de test (20 %), con estratificación por la variable objetivo para mantener la proporción de clases en ambos subconjuntos. La validación cruzada de 5 pliegues aplicada sobre

el corpus tabular constituye un protocolo de estimación del rendimiento esperado, no un procedimiento de búsqueda de hiperparámetros: la configuración del clasificador es fija y se documenta en la Sección 5.4, sin que intervenga ninguna exploración de rejilla ni muestreo aleatorio del espacio de configuraciones. Al no existir búsqueda de hiperparámetros, ninguna decisión de modelado queda condicionada por el rendimiento observado, lo que preserva la validez del conjunto de test como medida final del criterio de aceptación de RF-1.

Ambos protocolos difieren, no obstante, en el corpus sobre el que operan, y esa diferencia debe explicitarse para interpretar correctamente los resultados de la Capa 1. La validación cruzada de 5 pliegues se ejecuta sobre la totalidad del corpus tabular, es decir, sobre las mismas filas que después integran el conjunto de test, mientras que la columna de test procede del modelo entrenado únicamente sobre el 80 % de entrenamiento. En consecuencia, la cifra de validación cruzada no constituye una estimación sobre datos completamente ajenos al ajuste y debe leerse como indicador de estabilidad del rendimiento entre particiones del corpus; la única estimación sobre datos retenidos es la de la columna de test, que es la que sustenta la verificación de RF-1.

Para la Capa 2 se reserva desde el inicio una partición de test de clústeres originales, y el resto se evalúa mediante validación cruzada estratificada de 5 pliegues, con estratificación por etiqueta de clúster (*clean*, *isolated*, *attack_chain*) y particionado consciente de grupos y de familias de plantilla, cuyo procedimiento detalla la Sección 5.5.3. La partición global reserva 815 grafos para entrenamiento, 88 para validación y 86 para test, mientras que el conjunto sobre el que operan los pliegues de validación cruzada comprende 513 clústeres originales. La validación cruzada complementa al conjunto de test reservado porque es la estrategia estándar para conjuntos de grafos pequeños (Hamilton, 2020): con pocas muestras de la clase minoritaria, una única partición de validación no sería representativa por sí sola, de modo que el criterio de aceptación de RF-2 requiere ambas evidencias.

4.4. Métricas de evaluación

La selección de métricas de evaluación responde a los requisitos operativos del sistema y a las características del conjunto de datos. Cada métrica se corresponde con el criterio de acepta-

ción de un requisito concreto: la F1 macro cuantifica el criterio de RF-1 y describe la clasificación por clases invocada en RF-2, la Precisión@5 cuantifica el criterio de RF-5, y la AUC-ROC y la FPR_clean caracterizan el margen operativo con el que ambos criterios se satisfacen. Todas ellas se calculan sobre las particiones definidas en la Sección 4.3, cuyo alcance determina si el valor resultante constituye una estimación sobre datos retenidos. Las cuatro se definen a continuación junto con la razón por la que resultan preferibles a alternativas de uso más extendido.

F1 macro. Se aplica a la Capa 1 y a la clasificación por clases de la Capa 2. La F1 macro calcula la media no ponderada de la F1 de cada clase, otorgando igual importancia a las clases minoritarias (configuraciones erróneas críticas) que a las mayoritarias (manifiestos seguros). Esta propiedad es fundamental en conjuntos de datos de seguridad desequilibrados, donde la exactitud (*accuracy*) sería una métrica engañosa: sobre el conjunto de test de la Capa 1 (566 muestras), un clasificador que predice siempre «seguro» obtendría una exactitud del 56,4 % pero F1 macro del 36,0 %.

AUC-ROC. Se aplica al modelo binario de la Capa 1 y mide la capacidad del clasificador para separar las dos clases independientemente del umbral de decisión, lo que permite al operador ajustar la sensibilidad según las necesidades del entorno.

Precisión@k. Se aplica a la Capa 2 y al *ranking* del sistema completo. En lugar de evaluar la clasificación por umbral, P@k cuenta los clústeres de cadena de ataque situados en las primeras k posiciones del *ranking* por puntuación de riesgo y normaliza ese recuento por $\min(k, n_{\text{pos}})$, siendo n_{pos} el número de cadenas presentes en el conjunto evaluado:

$$P@k = \frac{|\{ i \in \text{top-}k : y_i = \text{attack_chain} \}|}{\min(k, n_{\text{pos}})} \quad (3)$$

Esta definición coincide con la fracción convencional sobre k únicamente cuando el conjunto evaluado contiene al menos k cadenas; cuando contiene menos, el denominador pasa a ser n_{pos} y la métrica alcanza el valor 1 si y solo si todas las cadenas existentes ocupan posiciones

dentro del top- k . La forma normalizada es la que dota de sentido al techo teórico de la métrica analizado en la Sección 6.3: sin ella, un conjunto con menos de k cadenas tendría un máximo alcanzable inferior a la unidad por construcción y no por deficiencia del modelo. Esta métrica está directamente alineada con el caso de uso operativo: un equipo de seguridad que audita los k clústeres de mayor riesgo en su infraestructura quiere que todos sean cadenas de ataque reales, no falsas alarmas. La elección de $k = 5$ refleja la capacidad de revisión manual típica de un equipo de seguridad reducido en una jornada laboral, valor de k que coincide con el fijado en el criterio de aceptación de RF-5.

FPR_clean. Esta tasa de falsos positivos sobre clústeres limpios mide la fracción de clústeres limpios entre los k primeros del *ranking* de riesgo¹. Un FPR_clean = 0 significa que ningún clúster limpio aparece entre los de mayor puntuación, eliminando las investigaciones innecesarias.

¹Operacionalmente es una tasa de limpios en el top- k , no una tasa de falsos positivos en sentido estricto (que normalizaría por el total de clústeres limpios); se mantiene la denominación FPR_clean por coherencia con la implementación.

5. Descripción de la herramienta software desarrollada

El sistema kubescan se organiza en tres capas que transforman un directorio de manifiestos YAML en una puntuación de riesgo de cadena de ataque, y se distribuye como una herramienta de línea de comandos instalable. Su construcción abarca la obtención del conjunto de datos, la derivación del grafo de clúster que representa las relaciones de seguridad entre manifiestos, el diseño y entrenamiento de cada capa, y las prácticas de ingeniería de software aplicadas al desarrollo. La implementación completa se encuentra disponible en el repositorio indicado en el Anexo A.

5.1. Arquitectura general del sistema

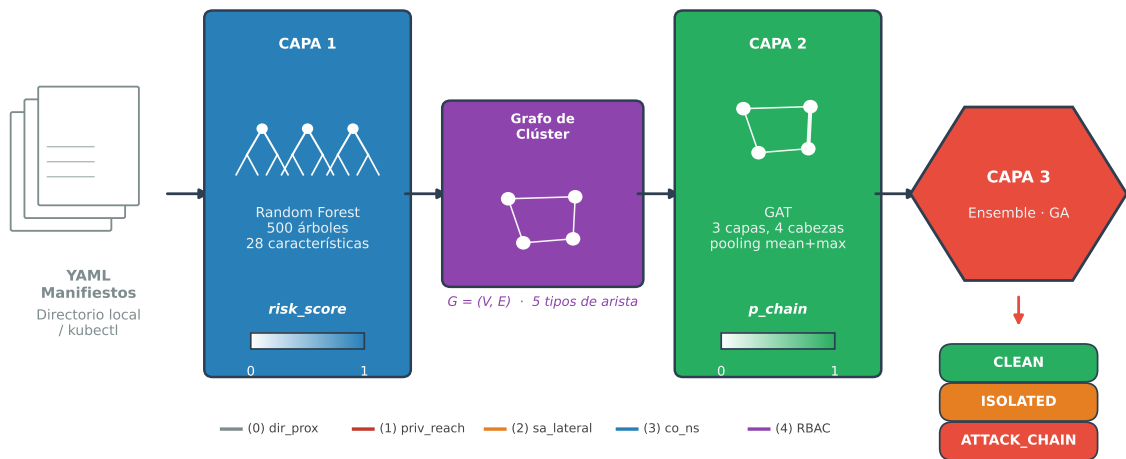
El sistema kubescan implementa una arquitectura de tres capas que transforma un directorio de manifiestos YAML en una puntuación de riesgo de cadena de ataque (Figura 1):

1. **Capa 1 (Random Forest):** extrae 28 características por manifiesto (25 indicadores binarios y 3 agregados derivados) y produce un $risk_score \in [0, 1]$.
2. **Capa 2 (GAT):** modela el clúster como un grafo donde los nodos son manifiestos y las aristas representan relaciones de seguridad; produce una probabilidad de cadena de ataque $\in [0, 1]$.
3. **Capa 3 (Ensemble GA):** combina las señales de las dos capas anteriores con una señal binaria de escape según la ecuación:

$$\text{score}(C) = w_{rf} \cdot \overline{\text{risk}}(C) + w_{gnn} \cdot p_{\text{chain}}(C) + w_{\text{esc}} \cdot \mathbf{1}[\exists \text{ nodo de escape en } C] \quad (4)$$

donde los pesos w_{rf} , w_{gnn} , w_{esc} son optimizados mediante Algoritmo Genético sobre las predicciones *out-of-fold* de la validación cruzada (Sección 5.6).

Figura 1. Arquitectura del sistema kubescan



Fuente: elaboración propia.

La Figura 1 muestra el proceso de tres capas, desde los manifiestos YAML de entrada hasta la clasificación de riesgo del clúster. El vector de nodo del grafo de entrada de la Capa 2 reserva un atributo adicional (índice 25) para una puntuación de severidad por manifiesto, calculada como suma ponderada de las banderas binarias mediante `compute_risk_score`. En el corpus de entrenamiento esa puntuación es una heurística determinista, no la salida del clasificador Random Forest; la Sección 5.3 detalla su cobertura efectiva.

5.2. Construcción del conjunto de datos

La construcción del conjunto de datos parte de la recolección de manifiestos YAML procedentes de fuentes públicas heterogéneas y avanza hasta la extracción de las características que alimentan la Capa 1. El origen y la composición de los manifiestos recopilados determinan la cobertura de patrones de configuración errónea disponible, y el proceso de extracción de características transforma cada manifiesto en un vector estructurado de dimensión fija. El conjunto resultante es la base tanto del clasificador de la Capa 1 como del grafo de clúster construido en la Sección 5.3.

5.2.1. Fuentes de datos

El conjunto de datos tabular final contiene 2.827 filas y 58 columnas, combinando cinco fuentes (agrupadas en cuatro categorías, dado que Rahman et al. (2023) aportan dos plataformas distintas con características de distribución diferentes; ver Tabla 3):

Tabla 3. Composición del conjunto de datos de manifiestos Kubernetes

Fuente	Filas	Rol
Rahman et al. (2023) — GitHub	1.510	Manifiestos reales, etiquetados
Rahman et al. (2023) — GitLab	396	Ídem, plataforma distinta
BishopFox BadPods (Art, 2021)	128	Pods vulnerables intencionales
Kubernetes Goat (Akula, 2020)	14	Escenarios de vulnerabilidad
Repositorios de seguridad adicionales	779	CTF, <i>fixtures</i> de herramientas, labs; etiqueta derivada por regla
Total	2.827	

Fuente: elaboración propia.

La procedencia de la etiqueta binaria no es homogénea entre las fuentes de la Tabla 3, y esa heterogeneidad condiciona la interpretación de los resultados de la Capa 1. Las 1.906 filas de Rahman et al. (2023) conservan la anotación *INSECURE* del corpus original, ajena al extractor de características de este trabajo: solo 245 de las 1.510 filas de GitHub y 87 de las 396 de GitLab coinciden con la disyunción de las columnas de configuración errónea observadas. Las 779 filas de los repositorios de seguridad adicionales, en cambio, carecen de anotación en origen y reciben una etiqueta derivada por regla en la ingesta (`compute_label` en `ingest_attack_repos.py`), definida como la disyunción de las columnas de configuración errónea; las 779 satisfacen por construcción esa equivalencia exacta. El corpus final combina, por tanto, verdad de referencia externa y etiquetado por regla, circunstancia que se retoma al analizar la circularidad entre etiqueta y características en la Sección 6.2.2.

La incorporación de BadPods fue esencial para mitigar la escasez de ejemplos de configuraciones erróneas críticas en los repositorios reales: antes de añadir esta fuente, `TRUE_HOST_PID` aparecía en solo el 0,3 % de las filas; tras la incorporación, la cobertura ascendió al 3,78 %. En cuanto a los conjuntos de datos de referencia existentes para intrusión en red, como UNSW-

NB15 (Moustafa y Slay, 2015), se estudió su metodología de construcción como guía para el diseño del proceso de etiquetado y la estrategia de balanceo de clases, dado que sus particiones curadas de entrenamiento y evaluación presentan un desequilibrio de clases del mismo orden de magnitud que el de este trabajo (56,3 % seguros frente a 43,7 % mal configurados). El conjunto GenKubeSec (Malul et al., 2024), publicado en 2024, no estaba disponible durante el desarrollo de este trabajo; en su lugar se empleó Checkov (Bridgecrew, 2021) directamente sobre los ficheros YAML descargados como fuente de validación multiherramienta equivalente.

5.2.2. Características extraídas

El extractor de características aplica 28 reglas de análisis estático a cada manifiesto YAML, organizadas en tres grupos:

- **18 banderas Rahman** (subconjunto del trabajo original): `TRUE_HOST_PID`, `CAP_SYS_ADMIN`, `SEC_CONT_OVER_PRIVIL`, `INSECURE_HTTP`, etc.
- **3 características derivadas**: `cap_misuse` (OR de las dos banderas de capacidades de sistema), `all_secrets` (OR de los indicadores de exposición de secretos), `total_misconfigs` (suma de todas las banderas activas).
- **7 características extendidas**: `NO_RUN_AS_NON_ROOT`, `IMAGE_USES_LATEST`, `SA_AUTOMOUNT_TOKEN`, `HOSTPATH_MOUNT`, etc. Seis de ellas presentan una procedencia mixta de valores; la séptima, `HOSTPATH_MOUNT`, se extrae siempre del propio manifiesto.

La procedencia de los valores de las seis características extendidas con equivalente en Checkov (Bridgecrew, 2021) —`NO_RUN_AS_NON_ROOT`, `NO_READ_ONLY_ROOT_FS`, `IMAGE_USES_LATEST`, `SA_AUTOMOUNT_TOKEN`, `USES_DEFAULT_SA` y `UNTRUSTED_REGISTRY`— se reparte en tres vías sobre las 2.827 filas del corpus: 921 filas (32,6 %) reciben el valor extraído directamente por el analizador YAML, 1.175 filas (41,6 %) se rellenan con la salida de Checkov a través de la tabla `CHECKOV_FILL`, y las 731 restantes (25,9 %) se imputan mediante la mediana de columna. Ese relleno mediante `CHECKOV_FILL` opera exclusivamente en la carga de la Capa 1. El constructor del grafo lee las columnas de `FEATURE_COLS` directamente del conjunto tabular e imputa como 0 toda celda vacía, sin consultar las columnas `checkov_*`, por lo que las seis características valen 0 en el vector de nodo de las 1.906 filas procedentes

de Rahman et al. La cobertura de Checkov tampoco es un mecanismo general aplicable a toda fila con YAML disponible, sino específico de la fuente: Checkov se ejecutó únicamente sobre el subconjunto de GitHub —1.175 de sus 1.510 filas— y no produjo salida alguna para los repositorios de seguridad adicionales, GitLab, BadPods ni Kubernetes Goat, cuyas filas se resuelven por el analizador YAML o por mediana. La séptima característica extendida queda al margen de este esquema, ya que `HOSTPATH_MOUNT` se deriva por completo del análisis estático del manifiesto y está poblada para las 2.827 filas.

La Capa 2 (GNN) consume únicamente las 18 banderas Rahman y las 7 características extendidas (25 en total, `FEATURE_COLS` en `yaml_parser.py`) como vector de nodo, excluyendo las 3 características derivadas —agregados calculados a partir de las otras 25 que no aportan información estructural adicional al grafo—; ver Sección 5.8. De estas 25, tres presentan varianza casi nula pero se mantienen en el entrenamiento, con importancia residual o nula (Sección 6.2.2): `SECCOMP_UNCONFINED` (15/2.827 activa), `VALID_TAINT_SECRET` (0/2.827 activa) y `NO_NETWORK_POLICY` (2.818/2.827 activa, sin poder discriminativo). Las importancias de característica registradas por el modelo Random Forest confirman ese aporte marginal: $1,6 \times 10^{-5}$ para `SECCOMP_UNCONFINED`, 0,0 para `VALID_TAINT_SECRET` y $2,4 \times 10^{-5}$ para `NO_NETWORK_POLICY`. Se conservan para preservar la estabilidad del contrato de características (`FEATURE_COLS`) y la correspondencia exacta con el conjunto de banderas de Rahman et al. (2023); no se ha realizado un estudio de ablación que cuantifique el efecto de su eliminación.

5.3. Construcción del grafo de clúster

Cada directorio de manifiestos se representa, mediante NetworkX (Hagberg et al., 2008), como un grafo dirigido $G = (V, E)$ donde $|V|$ es el número de manifiestos del clúster y E contiene aristas de cinco tipos semánticamente distintos (Tabla 4). La Figura 2 ilustra un ejemplo con cuatro nodos que incluye los tres tipos de nodo posibles: escape, movimiento lateral y limpio. Un *namespace* o espacio de nombres es la partición lógica con la que Kubernetes agrupa y aísla recursos dentro de un mismo clúster, y delimita el alcance de tres de los cinco tipos de arista. Las reglas de asignación de cada tipo de nodo y de arista se detallan en las dos subsecciones siguientes. Las aristas dirigidas parten del nodo con la capacidad que se indica hacia los demás nodos de su mismo espacio de nombres.

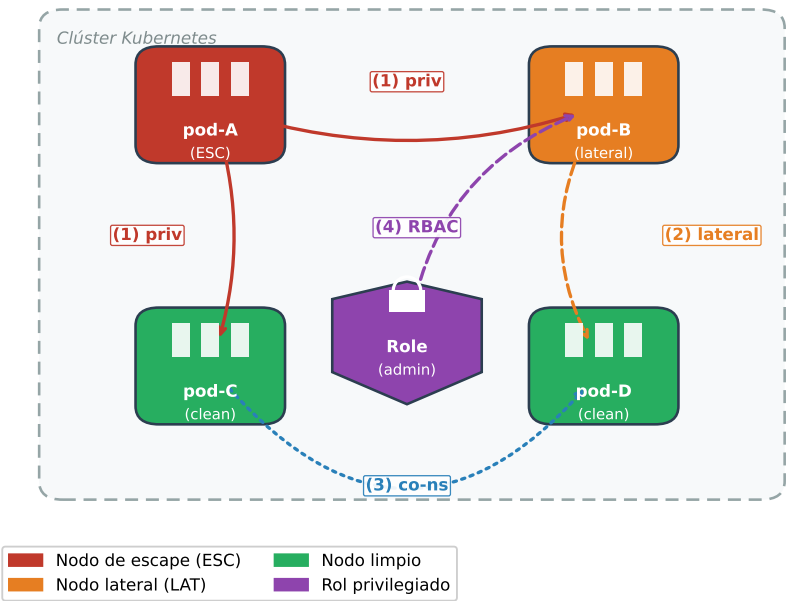
Tabla 4. Tipos de aristas en el grafo de clúster

Tipo	Código	Dirección	Criterio
Proximidad de directorio	0	Bidireccional	Dos últimos componentes de la ruta
Alcance de privilegio	1	Dirigida (escape)	Bandera de escape activa
Movimiento lateral SA	2	Dirigida (lateral)	Bandera de movimiento lateral
Coespacio de nombres	3	Bidireccional	Mismo <i>namespace</i> (SEMANTIC_NS)
Escalada RBAC	4	Dirigida (elevado)	SA vinculado a rol privilegiado

Fuente: elaboración propia.

Cada nodo v_i lleva un vector de características $\mathbf{x}_i \in \mathbb{R}^{26}$: los índices 0–24 corresponden a las 25 características binarias de la Capa 1 y el índice 25 contiene una puntuación de severidad por manifiesto. Esa puntuación procede de `compute_risk_score`, una suma ponderada de las mismas banderas binarias normalizada a [0,1], y no de una predicción del clasificador Random Forest. Está poblada únicamente para los 82 clústeres procedentes del corpus de Rahman et al.; en los 517 restantes la columna llega vacía al constructor del grafo y se imputa como 0, de modo que el índice 25 no aporta señal en el 86 % de los grafos.

Figura 2. Ejemplo de grafo de clúster Kubernetes con cuatro nodos



Fuente: elaboración propia.

La Figura 2 muestra los tres tipos de nodo (escape, movimiento lateral y limpio) y las aristas

dirigidas que cada uno origina. El nodo rojo (pod-A) tiene banderas de escape activas y emite aristas de tipo 1 (*privilege_reach*) hacia todos los demás nodos de su mismo *namespace*. El nodo naranja (pod-B) tiene banderas de movimiento lateral y emite aristas de tipo 2 (*sa_lateral*). La arista de tipo 4 (RBAC) representa la vinculación de un ServiceAccount a un rol privilegiado.

El conjunto de datos de grafos final comprende 599 grafos originales (25 clean, 537 isolated, 37 attack_chain) y 1.154 grafos totales tras la aumentación de datos, cuyas variantes se destinan exclusivamente al conjunto de entrenamiento. De los 555 grafos aumentados que componen ese *pool*, solo 390 entran efectivamente en el entrenamiento: los 165 restantes derivan de clústeres asignados a validación o test y quedan excluidos por la regla de asignación consciente de grupos descrita en la Sección 5.5.3. Esta cifra incorpora el corpus completo de Rahman et al. (2023), cuyos *namespaces* cruzados se resolvieron mediante descarga autenticada desde GitHub, lo que amplió sustancialmente el número de clústeres *isolated* respecto a las fases experimentales preliminares descritas en la Sección 6.3 del Capítulo 6.

5.3.1. Detección de escape y movimiento lateral

La clasificación de nodos utiliza dos conjuntos de banderas definidos en el código como constantes inmutables (*frozenset*). Un nodo se clasifica como de **escape** si tiene activa al menos una de las siguientes 8 banderas: `TRUE_HOST_PID`, `TRUE_HOST_IPC`, `TRUE_HOST_NET`, `DOCKERSOCK_PATH`, `CAP_SYS_ADMIN`, `CAP_SYS_MODULE`, `SEC_CONT_OVER_PRIVIL` o `HOSTPATH_MOUNT`. Este último vector fue incorporado durante el desarrollo al detectar que *badpods_hostpath* —que monta el sistema de ficheros del anfitrión— era incorrectamente etiquetado como *isolated* al no tener ninguna otra bandera de escape activa. Con la corrección, el número de clústeres de cadena de ataque del corpus preliminar ascendió de 13 a 15; la trayectoria completa del tamaño del corpus y del número de cadenas a lo largo de las fases experimentales se recoge en la Tabla 7.

Un nodo se clasifica como de **movimiento lateral** si tiene activa al menos una de las siguientes 4 banderas: `SA_AUTOMOUNT_TOKEN` (*token* de cuenta de servicio automontado), `USES_DEFAULT_SA` (usa la cuenta de servicio por defecto del *namespace*), `WITHIN_MANIFEST_SECRET` (secreto referenciado en el manifiesto) o `ALLOW_PRIVI` (conte-

nedor con capacidad de escalada). Las aristas de movimiento lateral (tipo 2) se construyen de forma análoga a las de escape: se emite una arista dirigida desde cada nodo lateral hacia los demás nodos de su mismo *namespace*.

La convivencia de los cinco tipos sobre un mismo par de nodos se resuelve por orden de inserción y no por una jerarquía explícita de prioridades. Las aristas se añaden en la secuencia proximidad de directorio (tipo 0), alcance de privilegio (tipo 1), movimiento lateral (tipo 2), coespacio de nombres (tipo 3) y escalada RBAC (tipo 4). Las de alcance de privilegio son las únicas que sobrescriben incondicionalmente cualquier etiqueta previa, de modo que un nodo con capacidad de escape nunca queda degradado a mero vecino; los cuatro tipos restantes se emiten bajo la guarda `not G.has_edge`, por lo que una arista de proximidad de directorio ya presente impide que el par se reetiquete como movimiento lateral, y las aristas RBAC, al añadirse en último lugar, nunca desplazan a las de movimiento lateral.

5.3.2. Detección de privilegio RBAC

La construcción de aristas RBAC (tipo 4) requiere dos pasadas sobre el conjunto de manifiestos. La primera pasada identifica los roles privilegiados: aquellos cuyo nombre está en el conjunto {cluster-admin, admin, edit} o cuyas reglas contienen verbos de escalada {*, escalate, bind, impersonate}. La segunda pasada recorre los objetos RoleBinding y ClusterRoleBinding: solo cuando el campo *roleRef* referencia un rol privilegiado identificado en la primera pasada se emite una arista RBAC desde el pod que usa esa cuenta de servicio hacia los demás nodos de su mismo *namespace*, coherentemente con el ámbito de los objetos RoleBinding de los que se derivan las cuentas elevadas.

5.4. Capa 1: Random Forest

La Capa 1 clasifica cada manifiesto de forma individual a partir de las características descritas en la Sección 5.2. Se entrenan dos modelos independientes sobre el mismo conjunto de características: un clasificador binario que distingue manifiestos seguros de mal configurados, y un modelo de severidad que estima el grado de gravedad de la configuración incorrecta detectada. Ambos comparten el conjunto de 28 características y los mismos hiperparámetros de bosque,

y difieren únicamente en la variable objetivo.

5.4.1. Modelo binario

Se entrena un `RandomForestClassifier` (Breiman, 2001) de `scikit-learn` (Pedregosa et al., 2011) con los hiperparámetros siguientes:

- `n_estimators = 500`
- `max_features = sqrt` (clasificación estándar)
- `min_samples_leaf = 2` (evita hojas de muestra única)
- `class_weight = balanced` (compensa ratio 1,29:1)
- `random_state = 42`

El modelo se entrena sobre 2.261 muestras (80 % del total) y se evalúa en 566 muestras de test no vistas durante el entrenamiento. La probabilidad predicha para la clase `misconfigured` (clase 1) se usa como `risk_score` del nodo en el grafo.

5.4.2. Modelo de severidad

Además del modelo binario, se entrena un clasificador de severidad de tres clases (*clean*, *low-medium*, *high-critical*) para ofrecer información más granular al operador de seguridad. Este modelo no interviene en el cálculo del grafo; se incluye como salida adicional de la Capa 1.

5.5. Capa 2: Graph Attention Network

La Capa 2 opera sobre el grafo de clúster construido en la Sección 5.3 y clasifica el clúster completo según el riesgo de cadena de ataque que presenta. Su diseño se apoya en tres decisiones acopladas: una arquitectura GAT con *embedding* de tipo de arista, un procedimiento de entrenamiento con validación cruzada estratificada de 5 pliegues, y una estrategia de aumentación de datos que mitiga la escasez de clústeres con cadena de ataque confirmada.

5.5.1. Arquitectura

El modelo GAT (Veličković et al., 2018) implementado tiene la siguiente estructura, con las dimensiones exactas verificadas en `gat_encoder.py`:

1. **Embedding de tipo de arista:** `Embedding(5, 8)`. Los cinco tipos de arista (proximidad de directorio, alcance de privilegio, lateral por ServiceAccount, espacio de nombres y RBAC) son categóricos: codificarlos como un escalar continuo implicaría un orden espurio (el tipo 4 «valdría» cuatro veces el tipo 1). Cada tipo se proyecta a un vector aprendido de 8 dims que `GATConv` consume como atributo de arista (*edge_dim=8*).
2. **Proyección inicial:** `Linear(26, 256) + LayerNorm(256) + Rectified Linear Unit (Unidad Lineal Rectificada) (ReLU) + Dropout(0.3)`. Proyecta el vector de nodo de 26 dims a 256 dims (64×4 cabezas).
3. **Capas GAT 0 y 1 (*concat=True*):** `GATConv(256, 64, heads=4)` con *edge_dim=8*; salida 256 dims (64×4); seguidas de `LayerNorm`, `Exponential Linear Unit (Unidad Lineal Exponencial) (ELU)`, conexión residual y `Dropout(0.3)`.
4. **Capa GAT 2 (*concat=False, heads=1*):** `GATConv(256, 64, heads=1)`; salida 64 dims (una sola cabeza de atención que produce directamente el *embedding* final del nodo, sin concatenación); seguida de `LayerNorm`, `ELU` y `Dropout(0.3)` (sin conexión residual: las dimensiones de entrada y salida difieren, 256 frente a 64).
5. **Global pooling:** `global_mean_pool` y `global_max_pool` sobre los 64-dim *embeddings* de todos los nodos; concatenación $\rightarrow$ 128 dims ($64 + 64$).
6. **Clasificador Multi-Layer Perceptron (Perceptrón Multicapa) (MLP):** `Linear(128, 64) + ReLU + Dropout(0.3) + Linear(64, 3)`.

La elección de GAT sobre GCN responde a que el mecanismo de atención permite al modelo aprender qué aristas son más informativas para la señal de ataque: una arista de *privilege_reach* debe recibir mayor peso que una arista de proximidad de directorio. El *pooling* combinado de media y máximo captura dos señales complementarias: la media refleja la postura de seguridad promedio del clúster, mientras que el máximo identifica el nodo más peligroso, ambas

necesarias para detectar cadenas de ataque que pueden estar concentradas en un único nodo crítico.

5.5.2. Entrenamiento

El modelo se entrena con PyTorch Geometric (Fey y Lenssen, 2019), construido sobre PyTorch (Paszke et al., 2019), bajo los siguientes hiperparámetros: optimizador AdamW con tasa de aprendizaje 5×10^{-4} y *weight_decay* = 10^{-4} ; *scheduler* CosineAnnealingLR con *T_max* = 300 y $\eta_{\text{mín}} = 10^{-5}$; función de pérdida CrossEntropyLoss con pesos de clase calculados por frecuencia inversa; *gradient clipping* con norma máxima 2,0; *early stopping* con paciencia de 50 épocas; tamaño de lote 8; semilla global 42 (idéntica para el entrenamiento y para la generación de particiones); y validación cruzada estratificada de 5 pliegues. El *checkpoint* de cada pliegue se selecciona por la mejor Precisión@5 de validación, con la F1 macro como criterio de desempate (`--select-by p5`), alineando la selección de modelos con la métrica primaria del sistema en lugar de con la métrica de clasificación.

El guion de entrenamiento (`train_gnn.py`) expone tres opciones que seleccionan variantes de esa configuración y que hacen reproducibles los estudios de ablación recogidos en el Capítulo 6. La opción `--conv` escoge el operador de convolución sobre el grafo entre `gat`, la GAT con *embedding* de tipo de arista descrita en la arquitectura y empleado en la configuración desplegada, y `gcn`, una GCN de referencia que ignora los tipos de arista y pondera todos los vecinos por igual. La opción `--focal-loss`, acompañada del parámetro de enfoque `--focal-gamma` (valor 2 por defecto), sustituye la entropía cruzada ponderada por la *focal loss* (Lin et al., 2017), que reduce el peso de los ejemplos ya bien clasificados para concentrar el gradiente en los difíciles. La opción `--select-by` fija el criterio de selección del *checkpoint* de cada pliegue entre `f1` (F1 macro de validación) y `p5` (Precisión@5 de validación), con la métrica restante como criterio de desempate. La configuración desplegada corresponde a `--conv gat`, sin `--focal-loss` y con `--select-by p5`; las alternativas se cuantifican en las Secciones 6.3.4 y 6.3.5.

Los pesos de clase se calculan en cada pliegue de entrenamiento mediante la siguiente fórmula implementada en `train_gnn.py`:

$$w_i = \frac{1/n_i}{\sum_{j=0}^{K-1} 1/n_j} \cdot K \quad (5)$$

donde n_i es el número de grafos de entrenamiento de la clase i y $K = 3$ es el número de clases. Esta fórmula normaliza los pesos para que su media sea 1,0, proporcionalmente equivalente al comportamiento de `class_weight=balanced` de scikit-learn (misma ponderación relativa entre clases). Cabe destacar que los pesos se calculan **después** de la aumentación de datos: como la aumentación genera 15 variantes por grafo de cadena de ataque, la clase *attack_chain* pasa de ser la minoría a ser la mayoría en el conjunto de entrenamiento. Los tres recuentos varían de un pliegue a otro, ya que las variantes de los clústeres reservados para validación se excluyen en cada iteración: entre 360 y 405 grafos de *attack_chain* aumentados frente a 17–18 *clean* y 361–371 *isolated*. La Ecuación 5 se autoadapta a esta nueva distribución asignando mayor peso a *clean* e *isolated*, y menor peso a *attack_chain*, compensando el efecto de la sobrerrepresentación creada por la aumentación.

5.5.3. Aumentación de datos

Para mitigar la escasez de ejemplos de cadena de ataque (37 de 599 grafos originales), se aplican tres estrategias de aumentación exclusivamente sobre los grafos de entrenamiento:

- **Edge dropout.** Eliminación aleatoria de entre el 10 % y el 35 % de aristas de los tipos 0 y 3, los menos críticos para la señal de ataque, con tasa muestreada uniformemente por variante.
- **Feature masking.** Puesta a cero con probabilidad por nodo entre el 10 % y el 25 %, restringida a seis características ajenas a la regla de etiquetado de cadena: nunca alcanza las banderas de escape, `HOSTPATH_MOUNT` ni el *risk_score*.
- **Subgraph sampling.** Extracción de un subgrafo inducido por entre el 40 % y el 70 % de nodos, aplicada a las últimas cinco de las 15 variantes y solo a grafos de más de 50 nodos, con *BFS* sembrada en el nodo de mayor *risk_score*. Cada subgrafo se valida contra la regla de etiquetado de cadena y, si deja de cumplirla, se descarta en favor de una variante sobre

el grafo completo, lo que evita introducir ruido de etiqueta.

Cada grafo de cadena de ataque origina 15 variantes aumentadas, lo que produce 555 grafos adicionales ($37 \text{ originales} \times 15 \text{ variantes}$) en el *pool* de aumentación. De estos, únicamente 390 —los correspondientes a los 26 grafos de cadena cuya base pertenece a la partición de entrenamiento— entran efectivamente en el entrenamiento, para un total de 1.154 grafos con aumentación.

La asignación de variantes a particiones es **consciente de grupos** (*group-aware*): una variante aumentada solo puede entrar en una partición de entrenamiento si su grafo base pertenece a esa misma partición. Las variantes derivadas de clústeres de validación o de test se excluyen del entrenamiento por completo, ya que entrenar con casi duplicados de un grafo de evaluación constituiría fuga de datos.

El particionado es además **consciente de familias de plantilla**: los clústeres originales que comparten una misma plantilla base (por ejemplo, las ocho variantes `badpods_*`, que difieren únicamente en qué bandera de escape activan —salvo `badpods_nothing-allowed`, que no activa ninguna—, o las nueve `datadog_*`) son casi duplicados entre sí, no muestras independientes. Cuando una familia contiene etiquetas mixtas, sus miembros se asignan como una unidad atómica a una única partición —nunca se reparten entre entrenamiento, validación y test ni entre pliegues— y la asignación emplea un reparto voraz ponderado por el número de grafos de cada etiqueta, de modo que la proporción real de cada clase por partición siga las fracciones solicitadas. Sin esta regla, un modelo entrenado con cinco hermanos de una familia resuelve trivialmente al sexto en test, inflando las métricas. La partición resultante (semilla 42) es de 815 grafos de entrenamiento (con aumentación), 88 de validación y 86 de test. Adicionalmente, los 86 clústeres de test se mantienen fuera de los pliegues de validación cruzada: los pliegues particionan únicamente los 513 grafos originales restantes (32 de cadena), de modo que ni los modelos de pliegue ni las predicciones *out-of-fold* empleadas para ajustar los pesos del *ensemble* (Capa 3) observan ningún clúster de test, directa ni indirectamente.

5.6. Capa 3: *Ensemble* con Algoritmo Genético

Los pesos $(w_{rf}, w_{gnn}, w_{esc})$ de la Ecuación 4 son optimizados mediante un Algoritmo Genético (Holland, 1975) con 60 individuos codificados como pares $(w_{gnn}, w_{esc}) \in [0, 1]^2$, con w_{rf} derivado como $1 - w_{gnn} - w_{esc}$ (la representación 2D garantiza que la suma de los tres pesos sea siempre 1). La función objetivo combina la métrica de *ranking* Precisión@5 y la tasa de falsos positivos sobre clústeres limpios FPR_clean, ambas definidas en la Sección 4.4, junto con un término de regularización:

$$F = 0,7 \cdot P@5 + 0,3 \cdot (1 - \text{FPR_clean}) - \lambda \sum_i \max(0, w_{\min} - w_i) \quad (6)$$

El último término es una penalización de suelo con $w_{\min} = 0,1$ y $\lambda = 2,0$: sin ella, el optimizador es libre de anular cualquiera de los tres pesos cuando eso puntúa marginalmente mejor sobre la muestra *out-of-fold*, lo que produce soluciones degeneradas que no generalizan al conjunto de test. La penalización hace que ignorar una señal por completo esté estrictamente dominado salvo que la ganancia observada supere λ veces el déficit.

La selección se realiza mediante torneo de tamaño 3, con elitismo del 10 %; el cruce es aritmético (media de los dos padres) con probabilidad 0,7; la mutación combina perturbación gaussiana ($\sigma = 0,08$, tasa del 15 %) con una mutación de frontera (tasa del 5 %) que salta a los vértices del símplex, seguida de reproyección al dominio válido ($w_{rf} \geq 0$); la población inicial se siembra con los vértices y puntos medios del símplex y con muestreo de Dirichlet, y el criterio de parada es 150 generaciones. Como *baseline* determinista, antes de la optimización genética se ejecuta una búsqueda en rejilla 2D con paso $1/20$ (231 evaluaciones); la comparación de su óptimo con el resultado del Genetic Algorithm (Algoritmo Genético) (GA) se discute en la Sección 6.4.

La señal de escape se define como un indicador binario que vale 1,0 si al menos un nodo del clúster tiene alguna bandera de escape activa, y 0,0 en caso contrario. Esta formulación evita la dilución que produciría una fracción cuando la mayoría de los nodos del clúster son limpios.

La asignación de clase final a partir de la puntuación *ensemble_score* se realiza mediante los

siguientes umbrales fijos, definidos en `ga_ensemble.py`:

- $ensemble_score \geq 0,60 \Rightarrow \text{ATTACK_CHAIN}$
- $ensemble_score \geq 0,30 \Rightarrow \text{ISOLATED_MISCONFIG}$
- De lo contrario $\Rightarrow \text{CLEAN}$

Estos umbrales son fijos y no se reoptimizan durante el entrenamiento del GA, que optimiza únicamente los pesos w_i de la Ecuación 4. Permiten al operador interpretar directamente la puntuación: cualquier clúster con $score \geq 0,60$ requiere revisión inmediata.

5.6.1. Restricción de viabilidad estructural

Una cadena de ataque multisalto requiere, por definición, alcanzabilidad entre al menos dos manifiestos; un grafo de clúster con un único nodo no puede albergarla. Esta propiedad, verificada sobre el corpus completo (los 592 grafos de cadena —37 originales y 555 variantes aumentadas— tienen ≥ 2 nodos y ≥ 1 arista, mientras que 499 de los 537 grafos *isolated* son de un solo nodo), se incorpora como restricción de inferencia en `ga_ensemble.py` (`MIN_CHAIN_NODES`, `is_chain_feasible`, `chain_rank_key`): en el *ranking* por riesgo de cadena, ningún clúster de un solo manifiesto puede superar a un clúster estructuralmente viable, y el veredicto de un clúster inviable se limita a `ISOLATED_MISCONFIG` aunque su puntuación supere el umbral de 0,60. La restricción se aplica únicamente en inferencia y evaluación; el ajuste de los pesos del GA se realiza sobre el *ranking* sin restricción, donde los pesos están mejor determinados (la Sección 6.4 cuantifica el efecto de esta regla sobre las métricas de test).

5.7. Implementación de kubescan

El sistema se distribuye como un paquete Python instalable mediante `pip install -e kubescan/`. La estructura del paquete sigue la convención *src-layout* de `setuptools`: el código fuente reside en `kubescan/src/kubescan/` y los módulos se organizan en tres submódulos: `model/` (clasificadores Random Forest, GAT y ensamblador GA), `utils/` (*parser* YAML y constructor de grafos) y `cli.py` (interfaz de línea de comandos basada en Click).

El punto de entrada principal expone dos comandos. El primero, `kubescan scan <dir>`, analiza un directorio local de manifiestos YAML y emite un informe de riesgo en formato texto o JavaScript Object Notation (Notación de Objetos de JavaScript) (JSON). El segundo, `kubescan live`, consulta la API de Kubernetes mediante `kubectl`, vuelca los recursos de *workloads* y RBAC en un directorio temporal y ejecuta el *pipeline* completo sobre ellos. La resolución de los pesos del modelo sigue la cadena: argumento `--checkpoints-dir` → variable de entorno `KUBESCAN_CHECKPOINTS` → enlace simbólico `kubescan/checkpoints/trained/` → `CheckpointNotFoundError`, excepción tipada derivada de `KubescanError` y definida en `exceptions.py`.

Ambos comandos comparten el conjunto de opciones recogido en la Tabla 18 del Anexo B, definido con Click en `cli.py`. La opción `--format` selecciona entre el informe en texto y la salida JSON; `--show-nodes` añade el desglose por manifiesto al informe en texto; `--cluster-name` fija el nombre del clúster que encabeza el informe, que por defecto es el del directorio analizado; y `--checkpoints-dir` altera el primer eslabón de la cadena de resolución de pesos. Las opciones `--namespace` y `--all-namespaces` son exclusivas de `kubescan live` y delimitan el ámbito de la consulta al API de Kubernetes. El grupo raíz añade, por su parte, `--verbose`, que eleva el nivel del registro de `WARNING` a `DEBUG`, y `--version`, que emite la versión instalada del paquete.

El informe en texto se estructura en tres bloques. El encabezado identifica el clúster y la ruta analizada. El cuerpo emite el veredicto (`CLEAN`, `ISOLATED_MISCONFIG` o `ATTACK_CHAIN`), seguido de las señales que lo sustentan —*ensemble score*, probabilidad de cadena del *ensemble* de cinco pliegues, probabilidad de clúster limpio, riesgo medio del Random Forest, fracción de manifiestos con capacidad de escape y con capacidad de movimiento lateral— y de los tres pesos w_{rf} , w_{gnn} y w_{esc} efectivamente aplicados. El bloque final, condicionado a `--show-nodes`, tabula los manifiestos ordenados por *risk_score* descendente con su marca de escape (ESC) o movimiento lateral (LAT) y sus primeras cuatro banderas activas. La Figura 8 del Anexo B traza el recorrido completo de una invocación de `kubescan scan` y reproduce una transcripción íntegra de este informe.

La salida JSON expone la misma información en un objeto de un solo nivel apto para consumo automatizado: los campos `cluster`, `cluster_dir` y `verdict` identifican el análisis; los

campos `ensemble_score`, `chain_probability`, `clean_probability`, `mean_rf_risk` y `escape_fraction` recogen las señales numéricas redondeadas a seis decimales; los recuentos `n_manifests`, `n_escape_capable` y `n_lateral_capable` resumen la composición del clúster; el objeto anidado `weights` contiene los tres pesos del *ensemble*; y la lista `manifests`, ordenada por *risk_score* descendente, aporta por cada manifiesto su nombre de fichero, su *risk_score*, los indicadores booleanos `escape_capable` y `lateral_capable`, y la lista completa `flags` de características de seguridad activas. Estos dos últimos campos son la evidencia de trazabilidad que verifica el requisito RNF-1 en la Sección 6.8.

5.8. Tabla de características del sistema

El catálogo completo de las 28 características, con su grupo de procedencia y su papel en el grafo —detección de escape (ESC) o de movimiento lateral (LAT)—, se recoge en las Tablas 12, 13 y 14 del Anexo A. De las 28, las 25 que no son derivadas (18 Rahman + 7 extendidas) forman también el vector de nodo de la Capa 2.

Las 8 banderas de escape (ESC, `ESCAPE_FLAGS` en `graph_builder.py`) son `TRUE_HOST_PID`, `TRUE_HOST_IPC`, `TRUE_HOST_NET`, `DOCKERSOCK_PATH`, `CAP_SYS_ADMIN`, `CAP_SYS_MODULE`, `SEC_CONT_OVER_PRIVIL` y `HOSTPATH_MOUNT`. Las 4 banderas de movimiento lateral (LAT, `LATERAL_FLAGS`) son `SA_AUTOMOUNT_TOKEN`, `USES_DEFAULT_SA`, `WITHIN_MANIFEST_SECRET` y `ALLOW_PRIVI`. La procedencia de las 7 características extendidas —mixta en seis de ellas y siempre del propio manifiesto en `HOSTPATH_MOUNT`— se detalla en la Sección 5.2.

5.9. Calidad de software y proceso de desarrollo

Cuatro herramientas integradas en un *pipeline* de pre-commit y CI/CD comprueban de forma automática el estilo, los tipos, el comportamiento y la complejidad del código, y bloquean la integración cuando alguna comprobación falla: (1) `ruff`, linter y formateador con reglas de estilo y seguridad; (2) `mypy` en modo estricto (`mypy kubescan/src --strict`), para el tipado estático de todo el paquete; (3) `pytest`, para las pruebas unitarias y de integración; y (4) `radon`, para la complejidad ciclomática. El reparto entre ambas etapas no es uniforme: los *hooks* de pre-commit ejecutan únicamente `ruff`, `ruff-format` y comprobaciones de higiene del repo-

itorio (espacios finales, fin de fichero, validez de los YAML de configuración y tamaño máximo de los ficheros añadidos), mientras que `mypy`, `pytest` y `radon` se ejecutan en el flujo de integración continua de GitHub Actions sobre cada *push* y cada *pull request*.

La suite comprende siete módulos unitarios —analizador YAML, constructor del grafo, codificador GAT, clasificador Random Forest, ensamblador GA, utilidades de dispositivo y un módulo de pruebas basadas en propiedades con Hypothesis— y un módulo de integración de extremo a extremo, cuya verificación funcional se aporta como evidencia de RF-1 y RF-2 en la Sección 6.8. La integración continua ejecuta la suite con medición de cobertura (`--cov`). Tres decisiones de diseño sostienen la reproducibilidad y la seguridad de la carga: `set_global_seed` centraliza las semillas de `random`, `NumPy` y `PyTorch`; `resolve_device` fija de forma determinista el *backend* de cómputo; y la carga del clasificador valida los tipos declarados por el fichero `skops` contra una lista blanca restringida a `scikit-learn` y `NumPy`, de modo que un *checkpoint* manipulado no pueda introducir tipos arbitrarios. En el lado de investigación, cada JSON de resultados incorpora un bloque de procedencia con el *commit*, las semillas y las versiones de los paquetes que lo produjeron.

6. Evaluación

Este capítulo evalúa el sistema kubescan completo. Se parte de una línea de base tomada del estado del arte, frente a la cual se contrastan los resultados de cada capa; a continuación se presentan dichos resultados en el mismo orden que las fases de entrenamiento descritas en el Capítulo 5. El desglose por clases del clasificador de grafo sobre el conjunto de test (Sección 6.5) caracteriza la dirección de los errores residuales, y el contraste con una herramienta de análisis estático externa (Sección 6.6) delimita hasta qué punto el corpus admite verificación independiente. Sobre esa base se sintetiza una métrica de proceso completo que resume el comportamiento extremo a extremo del sistema y se verifica el cumplimiento de los requisitos funcionales y no funcionales definidos en el Capítulo 4. El capítulo cierra con la evaluación de usabilidad y aplicabilidad realizada con usuarios expertos. Las métricas empleadas se justifican en detalle en la Sección 4.4.

6.1. Línea de base del estado del arte

Antes de presentar los resultados propios, esta sección fija la línea de base frente a la que se contrastan: los trabajos revisados en el Capítulo 2 que abordan un problema comparable al de cada capa. La comparación se organiza por capa, dado que la Capa 1 dispone de referencias directas en la literatura de clasificación de *IaC*, mientras que la Capa 2 carece de trabajos previos estrictamente comparables. En ambos casos se prioriza la comparación con resultados obtenidos sobre conjuntos de datos de tamaño y naturaleza similares; los resultados propios que superan o contrastan con esta línea de base se presentan en las secciones siguientes.

6.1.1. Capa 1 en contexto

Rahman et al. (2023), cuyos datos forman la base del presente trabajo, no publican modelos de clasificación supervisada sobre el mismo conjunto: el objetivo de su trabajo era la construcción del conjunto de datos y el análisis de la densidad de configuraciones erróneas, no la predic-

ción, por lo que no ofrece una cifra de referencia directa. Tampoco la literatura de IDS sobre UNSW-NB15 (Moustafa y Slay, 2015) proporciona una: ese trabajo presenta el conjunto de datos y compara exactitud y tasa de falsas alarmas de otros algoritmos, sin consignar F1 macro ni resultados de Random Forest (Sección 2.3). En consecuencia, el umbral $F1 > 0,85$ de OE2 es un criterio de diseño propio y no una cota tomada de la literatura, y la comparación informativa para la Capa 1 no es contra ese umbral sino contra una línea de base trivial calculada sobre el propio corpus: la mejor regla de una sola característica alcanza $F1 \text{ macro} = 0,8502$, frente al $0,9946$ del modelo (Sección 6.2). Esa separación es consistente con la naturaleza del problema: las configuraciones erróneas en manifiestos YAML son mayoritariamente discretas y acumulativas (más banderas activas implica mayor riesgo), lo que hace que el espacio de características sea relativamente separable una vez que se dispone de un conjunto de datos suficientemente representativo.

6.1.2. Capa 2 en contexto

Según la revisión bibliográfica no sistemática realizada en este trabajo (Capítulo 2), no se ha localizado ningún trabajo previo que aborde la clasificación supervisada de cadenas de ataque en clústeres Kubernetes mediante GNN, por lo que la Capa 2 no dispone de una línea de base directamente comparable. Las referencias más cercanas son los trabajos de GNN para Intrusion Detection System (Sistema de Detección de Intrusiones) (IDS) en red, como E-GraphSAGE (Lo et al., 2022), que documenta tasas de detección superiores a las de clasificadores tabulares clásicos sobre las repúblicas en formato NetFlow de los conjuntos de referencia del área; y la literatura que documenta la escasez de conjuntos de datos de grafos etiquetados en ciberseguridad (Zhong et al., 2024). Cabe puntualizar que esta última no establece un régimen de tamaño concreto ni prescribe un protocolo de evaluación, de modo que no puede invocarse como respaldo de la varianza observada en este trabajo; la validación cruzada estratificada se adopta aquí como decisión metodológica propia. La ventaja del componente estructural se mide sobre el conjunto de test genuinamente no visto: $P@5$ pasa de $0,40$ (solo Random Forest) a $0,80$ (configuraciones con GNN) y $P@1$ de $0,00$ a $1,00$ (Tabla 11; Sección 6.3 y siguientes), lo que indica que la información estructural del grafo es decisiva para identificar cadenas de ataque reales. La varianza entre pliegues de la fase final ($\sigma = 0,160$ para $P@5$) no se contrasta aquí con ninguna cota externa, al no disponerse de una referencia que la establezca para con-

juntos de este tamaño; en este trabajo tiene, no obstante, una causa estructural identificada: el particionado por familias concentra las cadenas de validación de cada pliegue en 1, 1, 4, 5 y 5 familias independientes respectivamente (Sección 6.3), de modo que la desviación estándar entre pliegues mide en buena parte la dispersión de dificultad entre familias. El análisis multisevilla (Sección 6.3.6) matiza esa lectura: cuatro de los cinco pliegues puntúan de forma idéntica con las tres semillas ensayadas, pero el único que varía —el pliegue 1, correspondiente a la familia `badpods_*`— es también el que más contribuye a esa desviación. En ese pliegue concreto, la dificultad de la familia y la sensibilidad a la semilla no resultan separables con los datos disponibles, de modo que la atribución estructural de la varianza es parcial y no total.

6.2. Resultados de la Capa 1: Random Forest

Esta sección presenta los resultados de los dos modelos de la Capa 1 descritos en el Capítulo 5: el clasificador binario y el modelo de severidad. Se presentan primero las métricas de clasificación binaria y la importancia de las características, y a continuación los resultados del modelo de severidad. Ambos modelos comparten el mismo conjunto de 28 características extraídas por manifiesto, descrito en la Sección 5.2.

6.2.1. Clasificación binaria

La Tabla 5 resume los resultados del modelo binario mediante cuatro métricas: F1 macro (media no ponderada de F1 entre las clases seguro y misconfigured, que da igual peso a ambas independientemente de su frecuencia en el conjunto de datos), exactitud (*accuracy*, porcentaje simple de aciertos), AUC-Receiver Operating Characteristic (Característica Operativa del Receptor) (ROC) (capacidad del modelo para separar las dos clases a cualquier umbral de decisión) y *out-of-bag score* (estimación interna de generalización que Random Forest calcula sobre las muestras que cada árbol no vio durante su entrenamiento, sin necesitar un conjunto de validación aparte). El clasificador alcanza una F1 macro de 0,9946 en el conjunto de test, superando el objetivo de diseño (OE2, F1 macro > 0,85) en 14,5 puntos porcentuales.

Conviene situar esa cifra frente a una línea de base trivial, porque el umbral de OE2 resulta poco exigente sobre este corpus: la mejor regla de una sola característica (`NO_RESO = 1`) alcanza

por sí sola $F1 \text{ macro} = 0,8502$, prácticamente el umbral de 0,85. El margen relevante del modelo no es, por tanto, el que separa 0,9946 de 0,85, sino el que lo separa de esa regla trivial: 14,4 puntos porcentuales sobre la mejor decisión de una única condición. Una disyunción de todas las banderas binarias, en cambio, se desploma a $F1 \text{ macro} = 0,3042$, al marcar como mal configurado casi cualquier manifiesto.

Tabla 5. Resultados del Random Forest — clasificación binaria

Métrica	Test	CV (5-fold)	Objetivo
F1 macro	0,9946	$0,9917 \pm 0,0053$	$> 0,85 \checkmark$
Exactitud (<i>accuracy</i>)	0,9947	$0,9919 \pm 0,0052$	—
AUC-ROC	0,9986	$0,9984 \pm 0,0011$	—
<i>Out-of-bag score</i>	0,9920	—	—

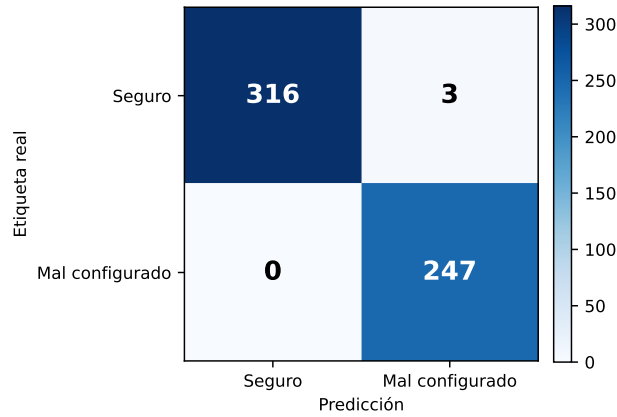
Fuente: elaboración propia.

La Figura 3 muestra la matriz de confusión sobre el conjunto de test (566 muestras). El modelo comete solo 3 errores totales: 3 falsos positivos (manifiestos seguros clasificados como *misconfigured*) y 0 falsos negativos. La ausencia de falsos negativos es la propiedad más crítica para seguridad operativa: significa que ningún manifiesto con configuraciones erróneas reales escapa al filtrado de la Capa 1, independientemente de cuántas banderas estén activas o de su rareza en el conjunto de datos. Los 3 falsos positivos corresponden a manifiestos seguros que acumulan varias características de riesgo de severidad media (por ejemplo, `NO_RUN_AS_NON_ROOT` y `IMAGE_USES_LATEST` simultáneamente), pero que no tienen ninguna bandera de escape activa; el operador de seguridad puede descartarlos rápidamente inspeccionando el campo `flags` de la salida JSON.

El AUC-ROC de 0,9986 (prácticamente 1,00) indica que el modelo puede ordenar las muestras por probabilidad de configuración errónea de forma casi perfecta a cualquier umbral de decisión. En la práctica, este valor implica que, si se ordenan los 566 manifiestos de test de mayor a menor *risk_score*, los manifiestos *misconfigured* aparecen casi todos antes que los seguros. Esta propiedad es la que habilita el uso del *risk_score* como característica nodal cuantitativa en el grafo de la Capa 2: la señal transmitida a la GNN no es un bit binario sino una puntuación continua que ordena los manifiestos por densidad de configuraciones erróneas. No se aplica

ninguna técnica de calibración de probabilidades, por lo que el valor no debe interpretarse como una probabilidad calibrada en sentido estricto; el ordenamiento resultante es, en cambio, coherente con la matriz de confusión de la Figura 3.

Figura 3. Matriz de confusión del Random Forest en el conjunto de test



Fuente: elaboración propia.

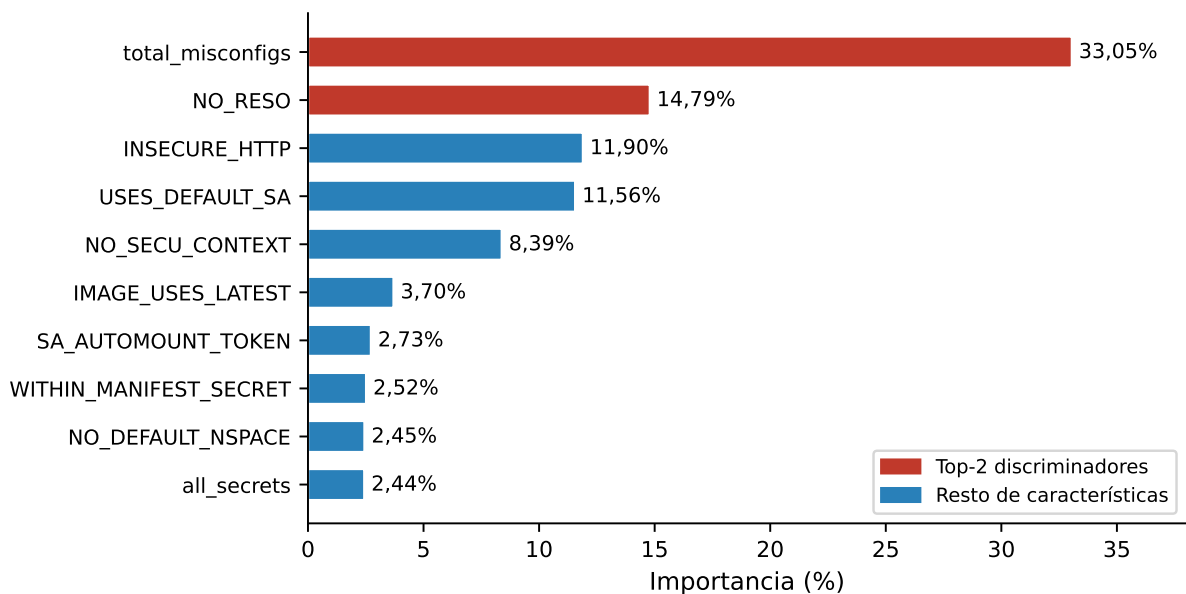
6.2.2. Importancia de características

La Figura 4 muestra las 10 características más relevantes del modelo. La característica más discriminativa es `total_misconfigs` (33,05 % de importancia), seguida de `NO_RESO` (14,79 %) e `INSECURE_HTTP` (11,90 %). Las banderas de escape críticas (`TRUE_HOST_PID`, `SEC_CONT_OVER_PRIVIL`) presentan importancias menores (<1 %) por su rareza en el conjunto de datos, pero son identificadas correctamente cuando están presentes, como confirma la ausencia de falsos negativos en la clasificación binaria. Las dos primeras características de la Figura 4 (en rojo) concentran el 48 % de la capacidad discriminativa total del modelo. Un estudio de ablación que elimina `total_misconfigs` produce F1 idéntico y la misma matriz de confusión, lo que confirma que el modelo no depende de esa característica derivada: su señal es recuperable a partir de las banderas que la componen.

Esa ablación, sin embargo, no permite concluir que no exista circularidad entre etiqueta y características. La etiqueta binaria del corpus de Rahman et al. se define como la presencia de al menos una configuración errónea de un subconjunto concreto de patrones, y esos patrones son a su vez características de entrada del modelo: sobre las 1.906 filas de esa fuente, la etiqueta coincide exactamente con la disyunción de ocho de las características observadas (`INSECURE_HTTP`,

NO_SECU_CONTEXT, WITHIN_MANIFEST_SECRET, NO_RESO, CAP_SYS_ADMIN, TRUE_HOST_NET, ALLOW_PRIVI y DOCKERSOCK_PATH), sin un solo falso positivo ni falso negativo. Reentrenar la misma configuración restringida a esas filas produce, consecuentemente, F1 macro = 1,0000. La Capa 1 aprende por tanto una aproximación robusta de una regla determinista definida sobre su propio espacio de características —bajo imputación, valores ausentes y las etiquetas heterogéneas del resto de fuentes—, no una noción de configuración errónea independiente de dicha regla. Esta limitación, análoga a la que afecta al etiquetado de grafos, se analiza en la limitación L5 del Capítulo 7.

Figura 4. Importancia de características del clasificador Random Forest



Fuente: elaboración propia.

6.2.3. Modelo de severidad

El clasificador de tres clases (*clean*, *low-medium*, *high-critical*) alcanza F1 macro = 0,9838 en test. La Tabla 6 desglosa por clase la precisión (proporción de predicciones de esa clase que son correctas), el *recall* (proporción de casos reales de esa clase que el modelo identifica), el F1 (media armónica de ambas) y el soporte (número de muestras de test en esa clase):

El F1 más bajo de las tres clases corresponde a *low-medium* (0,9772), por debajo del de *high-critical* (0,9789). La causa no es una dificultad intrínseca de la clase intermedia, cuyo *recall* es de 0,9934, sino su papel de sumidero de los errores del modelo: absorbe los seis manifestos

Tabla 6. F1 por clase — modelo de severidad (3 clases, test)

Clase	Precisión	Recall	F1	Soporte
<i>clean</i>	1,0000	0,9906	0,9953	319
<i>low-medium</i>	0,9615	0,9934	0,9772	151
<i>high-critical</i>	0,9894	0,9688	0,9789	96
Macro media	0,9836	0,9842	0,9838	—

Fuente: elaboración propia.

mal asignados del conjunto de test —tres procedentes de *clean* y tres de *high-critical*—, lo que rebaja su precisión a 0,9615. La clase *high-critical* (96 muestras de test, que representan manifiestos con banderas de escape activas como `TRUE_HOST_PID` o `SEC_CONT_OVER_PRIVIL`) comete 3 errores, los tres clasificados como *low-medium*; ningún manifiesto crítico es clasificado como *clean*. Esa ausencia de fugas hacia la clase *clean* es la propiedad relevante desde el punto de vista de seguridad operativa: en el peor caso un manifiesto crítico se subestima a *low-medium*, nunca se descarta como seguro, y la dirección de los errores restantes —tres manifiestos *clean* sobreclasificados como *low-medium* y un único manifiesto *low-medium* elevado a *high-critical*— es igualmente conservadora. El modelo de severidad se utiliza en el informe de salida de kubescan para proporcionar al operador una descripción cualitativa del nivel de riesgo de cada manifiesto, complementando el *risk_score* numérico del modelo binario.

6.3. Resultados de la Capa 2: GNN

Esta sección presenta los resultados de la Capa 2 en el orden en que fueron obtenidos durante el desarrollo experimental: la evolución de las métricas a lo largo de las fases de mejora del modelo, el análisis de Precisión@5, los resultados por pliegue de la configuración final, la ablación de la función de pérdida y del criterio de selección, y la robustez frente a la semilla de entrenamiento. A diferencia de la Capa 1, la Capa 2 requirió varias iteraciones metodológicas antes de alcanzar una configuración cuyas métricas fueran a la vez válidas y reproducibles. Cada iteración corrigió una fuente concreta de fuga de información o una elección de entrenamiento subóptima, y el descenso de las métricas que acompaña a esas correcciones cuantifica el sesgo optimista que introducían los protocolos de particionado sucesivamente descartados.

6.3.1. Evolución por fase experimental

La Tabla 7 muestra la evolución de los resultados de la GAT a lo largo de las seis fases del desarrollo experimental. En las cuatro primeras, la aumentación de datos y la corrección del vector de escape `HOSTPATH_MOUNT` contribuyeron de forma independiente y complementaria. La quinta fase corrige el protocolo de particionado: las fases 1–4 incluían en el entrenamiento variantes aumentadas derivadas de los clústeres de validación y test (fuga de datos por casi duplicados), por lo que sus métricas están infladas de forma optimista. Esa quinta fase aplica el protocolo *group-aware* descrito en el Capítulo 5 (las variantes acompañan a su clúster base y los clústeres de test quedan fuera de los pliegues) e incorpora el *embedding* de tipos de arista; sus métricas son metodológicamente válidas pero corresponden todavía al corpus preliminar de 471 grafos. La sexta y última fase incorpora el corpus completo de Rahman et al. (599 grafos originales, 37 cadenas; Sección 5.2) y extiende el protocolo de particionado a las **familias de plantilla** (Capítulo 5): durante esta fase se detectó que los casi duplicados de una misma plantilla (ocho `badpods_*`, nueve `datadog_*`, ocho `simulator_*`, doce `kubernetes_goat_*`) quedaban repartidos entre particiones, una fuga de información adicional a la de las variantes aumentadas que inflaba las métricas de las fases 1–5. Con las familias atómicas, entropía cruzada ponderada como función de pérdida y selección de *checkpoints* por Precisión@5 (ablación en la Sección 6.3.4), esta fase es la que se presenta como resultado final del trabajo: sus métricas son las que produce el modelo efectivamente desplegado y reproducible a partir de los *checkpoints* del repositorio. Por el cambio de protocolo, sus cifras no son directamente comparables con las de las fases 1–5 de la Tabla 7: miden generalización entre familias, no interpolación entre casi duplicados.

En la Tabla 7, las fases 1–4 emplean el protocolo de particionado preliminar (con fuga por variantes aumentadas) y se conservan como registro histórico; la fase 5 introduce el protocolo *group-aware* sobre el corpus preliminar; la fase 6 es el resultado final, con protocolo *group-aware* sobre el corpus completo. La columna «Cobert. techo» indica en cuántos de los 5 pliegues el modelo alcanza el *techo teórico* de P@5. La métrica se calcula como el número de cadenas situadas en el top-5 dividido entre $\min(5, n_{\text{pos}})$, siendo n_{pos} el número de cadenas presentes en el pliegue, de modo que el techo vale 1,0 siempre que el pliegue contenga al menos una cadena y se alcanza cuando las cinco primeras posiciones del *ranking* están ocupadas

Tabla 7. Evolución de resultados de la Capa 2 (GAT, 5-fold CV)

Fase	Grafos	Cadenas	F1 macro	P@5	Cobert. techo
Línea base	82	13	$0,829 \pm 0,098$	$0,400 \pm 0,179$	2/5
Tras aumentación	277	13	$0,915 \pm 0,059$	$0,520 \pm 0,098$	5/5
Tras corrección HOSTPATH	307	15	$0,915 \pm 0,050$	$0,600 \pm 0,000$	5/5
Conjunto de datos extendido	471	25	$0,917 \pm 0,065$	$0,880 \pm 0,098$	5/5
Protocolo <i>group-aware</i> + <i>embedding</i> de aristas	471	25	$0,870 \pm 0,075$	$0,720 \pm 0,098$	3/5
Corpus completo + familias de plantilla (final)	599	37	$0,803 \pm 0,137$	$0,720 \pm 0,160$	1/5

Fuente: elaboración propia.

por cadenas —o, si el pliegue contiene menos de cinco, cuando todas ellas figuran en el top-5. En la fase final, los cinco pliegues de validación contienen 8, 7, 6, 6 y 5 cadenas respectivamente, por lo que el denominador es 5 en todos ellos y el techo 1,0 exige cinco aciertos en las cinco primeras posiciones; únicamente el pliegue 0 lo consigue.

El objetivo de diseño de *ranking* es $P@5 > 0,70$ y el umbral de F1 fijado para la Capa 1, tomado como referencia, 0,85; las desviaciones típicas representan la desviación estándar entre los 5 pliegues de validación cruzada.

6.3.2. Análisis de Precisión@5

El objetivo de diseño es $P@5 > 0,70$ (OE4). En el resultado final de este trabajo —particionado consciente de familias sobre el corpus completo de Rahman et al. (599 grafos originales, 37 cadenas; fase 6 de la Tabla 7)— los pliegues de validación cruzada particionan los 513 grafos originales no reservados para test (32 cadenas). El modelo obtiene $P@5 = 0,720 \pm 0,160$, alcanzando el objetivo con la semilla de entrenamiento desplegada (42); en las tres semillas (7, 42, 123) la media es de $0,68 \pm 0,033$, con el objetivo dentro de una desviación típica de la me-

dia¹ (Sección 6.3.6).

Alcanzar esta cifra exigió diagnosticar una regresión aparente: la primera configuración entrenada sobre el corpus completo con familias atómicas empleaba *focal loss* y selección de *checkpoints* por F1 macro, y obtuvo $P@5 = 0,480 \pm 0,271$, muy por debajo tanto del objetivo como de la fase 5. La ablación factorial de la Sección 6.3.4 mostró que la caída se debía a las elecciones de función de pérdida y de criterio de selección —no al corpus ni al protocolo de particionado—, y que revertir a entropía cruzada ponderada y seleccionar por Precisión@5 recupera el objetivo sobre el mismo particionado honesto. La varianza entre pliegues ($\sigma = 0,160$, frente a 0,098 en la fase 5) refleja en gran medida la propia atomicidad de las familias: cada pliegue evalúa 1, 1, 4, 5 y 5 familias independientes respectivamente, como se detalla a continuación.

6.3.3. Resultados por pliegue (fase final)

Las medias de validación cruzada de la configuración desplegada ocultan una dispersión considerable entre pliegues, y su desglose es necesario para atribuir esa dispersión a una causa concreta. La Tabla 8 recoge la F1 macro y la Precisión@5 obtenidas en cada uno de los cinco pliegues, junto con la media y la desviación típica poblacional que resumen el conjunto. Los valores individuales se interpretan a continuación a la luz de la composición de cada pliegue en términos de familias de plantilla.

La Tabla 8 corresponde al particionado consciente de familias sobre el corpus completo: los pliegues particionan 513 grafos originales con 32 cadenas, con el conjunto de test excluido. La dispersión entre pliegues es consecuencia directa del particionado por familias: al mantener cada familia de plantilla como unidad atómica, las cadenas de validación de cada pliegue quedan concentradas en pocas familias independientes. El pliegue 0 valida sobre las nueve variantes *datadog_** (ocho de ellas cadenas; grafos de 2–3 nodos casi idénticos entre sí) y alcanza $F1 = 1,0$ y $P@5 = 1,0$; el pliegue 1 valida sobre las ocho variantes *badpods_** (siete de ellas cadenas) y es sistemáticamente el más difícil ($P@5 = 0,60$ con la semilla desplegada; 0,2–0,6 según la semilla, Sección 6.3.6). Los tres pliegues restantes reparten las dos familias grandes que quedan jun-

¹Las desviaciones típicas de las métricas del modelo se expresan como desviación poblacional ($ddof = 0$), coherente con la instrumentación de NumPy empleada en la validación cruzada; la del SUS (Sección 6.9) sigue la convención muestral ($n - 1$) habitual en su literatura.

Tabla 8. Resultados por pliegue — Capa 2, fase final

Pliegue	F1 macro	P@5
0	1,000	1,00
1	0,580	0,60
2	0,863	0,60
3	0,801	0,60
4	0,770	0,80
Media	0,803	0,720
± Desv.	0,137	0,160

Fuente: elaboración propia.

to a repositorios únicos: el pliegue 2 concentra las ocho variantes `simulator_*`, el pliegue 3 las doce variantes `kubernetes_goat_*` y el pliegue 4 no contiene ninguna de las cuatro familias de plantilla, sino exclusivamente repositorios únicos. El número de familias independientes portadoras de cadena por pliegue es, en consecuencia, de 1, 1, 4, 5 y 5. Cada pliegue mide, por tanto, la capacidad del modelo de generalizar a un número reducido de familias no vistas —un tamaño muestral efectivo menor que el número nominal de cadenas—, lo que explica que la varianza ($\sigma = 0,160$) sea mayor que bajo protocolos con fuga y constituye una amenaza a la validez de conclusión que se documenta en el Capítulo 7.

6.3.4. Ablación de función de pérdida y criterio de selección

La regresión aparente de la primera configuración sobre el corpus completo ($P@5 = 0,480$) confundía dos cambios simultáneos: la introducción de *focal loss* (Lin et al., 2017) y el nuevo particionado por familias. Para atribuir el efecto se completó la matriz factorial 2×2 —función de pérdida (entropía cruzada ponderada frente a *focal loss*, $\gamma = 2$) por criterio de selección de *checkpoints* (F1 macro frente a Precisión@5)— con particionado, semilla y arquitectura idénticos (Tabla 9)².

Las cuatro configuraciones de la Tabla 9 se evaluaron con 5-fold Cross-Validation (Validación

²Las cuatro configuraciones son reproducibles mediante `make reproduce GNN_FOCAL={0,1} GNN_SELECT_BY={p5,f1}`; solo se conserva el *checkpoint* de la configuración finalmente desplegada.

Tabla 9. Ablación de función de pérdida y criterio de selección

Pérdida	Selección	P@5 (CV)	F1 macro (CV)
<i>Focal</i> ($\gamma=2$)	F1 macro	0,480 $\pm$ 0,271	0,823 $\pm$ 0,097
Entropía cruzada pond.	F1 macro	0,600 $\pm$ 0,283	0,859 $\pm$ 0,103
<i>Focal</i> ($\gamma=2$)	P@5	0,680 $\pm$ 0,204	0,795 $\pm$ 0,173
Entropía cruzada pond.	P@5	0,720 $\pm$ 0,160	0,803 $\pm$ 0,137

Fuente: elaboración propia.

Cruzada) (CV) sobre el corpus completo, particionado por familias y semilla 42. Los dos efectos principales apuntan en la misma dirección, pero no se comportan de forma aditiva: sustituir *focal loss* por entropía cruzada ponderada aporta +0,12 puntos de P@5 con selección por F1 macro y solo +0,04 con selección por P@5, mientras que seleccionar los *checkpoints* por la métrica primaria de *ranking* en lugar de por F1 macro aporta +0,20 puntos bajo *focal loss* y +0,12 bajo entropía cruzada. La suma de ambos efectos principales sobre la configuración de partida (0,480) predeciría P@5 = 0,80, frente al 0,720 efectivamente observado: existe una interacción negativa de 0,08 puntos, es decir, la mejora que aporta cada factor se solapa parcialmente con la del otro en lugar de acumularse. La *focal loss*, diseñada para regímenes con miles de ejemplos por clase (Lin et al., 2017), además anula por completo la clasificación de la clase *attack_chain* en el conjunto de test (F1 de clase 0,0 frente a 0,667 con entropía cruzada). La configuración desplegada es, en consecuencia, entropía cruzada ponderada con selección por Precisión@5.

La Tabla 9 recoge la Precisión@5 de validación cruzada ($\pm\sigma$) de las cuatro configuraciones; solo la desplegada —entropía cruzada ponderada con selección por Precisión@5— supera el objetivo de diseño. La distancia respecto de la segunda mejor configuración es, no obstante, estrecha (0,720 $\pm$ 0,160 frente a 0,680 $\pm$ 0,204): sus bandas de una desviación típica se solapan casi por completo y, sobre cinco pliegues y con una métrica cuantizada en múltiplos de 0,2, esa diferencia equivale a un solo acierto de *ranking* en un único pliegue. Los datos sostienen, por tanto, una afirmación más débil que la de una dependencia estricta de ambos factores: la combinación desplegada es la única que sitúa la media de validación cruzada por encima del objetivo, y la configuración más alejada de él —*focal loss* con selección por F1 macro, 0,480— sí queda separada por un margen que excede la dispersión observada.

6.3.5. Ablación de arquitectura

La elección de GAT frente a alternativas más simples y la profundidad de tres capas se fijaron en el diseño de la Capa 2 y admiten contraste empírico sobre el mismo protocolo. La literatura sobre GNN aplicadas a grafos pequeños heterogéneos documenta de forma consistente que los mecanismos de atención mejoran la precisión cuando los tipos de arista tienen importancias desiguales (Veličković et al., 2018). En el grafo de clúster Kubernetes, las aristas de *privilege_reach* (tipo 1) son cualitativamente más informativas que las de *dir_proximity* (tipo 0): la primera conecta directamente nodos de escape con el resto del clúster, mientras que la segunda simplemente refleja la organización de directorios del repositorio. GAT puede aprender esta distinción durante el entrenamiento, mientras que GCN asigna pesos uniformes a todos los vecinos, diluyendo la señal de escape con ruido de colocalización.

Esta hipótesis se cuantifica empíricamente mediante un experimento de ablación controlado sobre el corpus completo, con particionado, semilla, función de pérdida y criterio de selección idénticos a los de la configuración final (fase 6 de la Tabla 7), de modo que las tres filas son directamente comparables entre sí y con el resultado final (Tabla 10):

Tabla 10. Ablación de arquitectura — Capa 2

Arquitectura	F1 macro	P@5
GAT 3 capas + <i>embedding</i> de aristas (desplegada)	0,803 ± 0,137	0,720 ± 0,160
GAT 2 capas + <i>embedding</i> de aristas	0,800 ± 0,171	0,720 ± 0,160
GCN 3 capas (sin tipos de arista)	0,711 ± 0,143	0,680 ± 0,160

Fuente: elaboración propia.

La GAT supera a la GCN en F1 (+9,2 puntos) y en P@5 (+4 puntos). Esa ventaja acredita la configuración adoptada en su conjunto, pero no permite atribuirle al esquema de tipos de arista por separado: la rama GCN sustituye simultáneamente el mecanismo de agregación por atención y descarta el tensor *edge_attr*, de modo que ambos factores varían a la vez. Aislar la contribución del tipado exigiría una tercera rama —GAT sin tipos de arista— que no forma parte de las configuraciones entrenadas (Capítulo 7). La comparación entre 2 y 3 capas es menos concluyente: las medias de validación cruzada casi coinciden —P@5 = 0,720 ± 0,160 en ambas

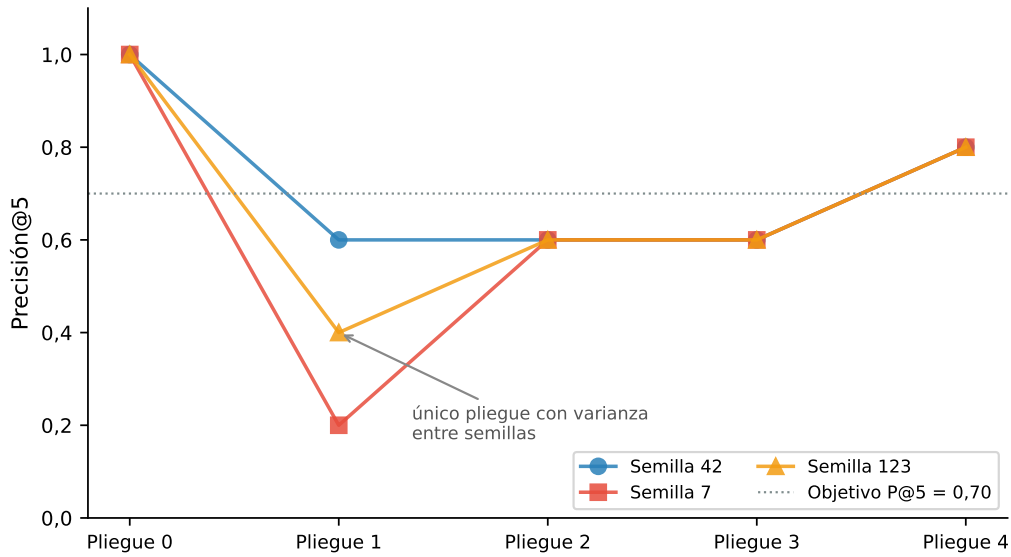
configuraciones y F1 macro de 0,803 frente a 0,800—, pero esa coincidencia agregada oculta un desacuerdo apreciable pliegue a pliegue, con diferencias de F1 de +0,142 en el pliegue 3 y −0,191 en el pliegue 4 que se cancelan al promediar y dejan una brecha media de solo 0,003. Dos modelos que difieren en esa magnitud sobre pliegues concretos no evidencian robustez de la arquitectura frente a la profundidad; lo que muestran es que la profundidad no determina el nivel medio de las métricas sobre este corpus. Se mantiene la configuración de 3 capas como desplegada por su ventaja marginal en F1 macro y por coherencia con el resto de resultados presentados. La ablación de *pooling* (*mean-only* frente a *mean+max*) queda como trabajo futuro (TF6, Sección 7.4).

6.3.6. Robustez frente a la semilla de entrenamiento

Para acotar la dependencia del resultado respecto de la semilla de entrenamiento, la configuración final se reentrenó con tres semillas (7, 42, 123; Figura 5) manteniendo fijo el particionado (semilla 42): resortear también las particiones confundiría la varianza del modelo con la asignación de familias a pliegues, que la Sección 6.3 muestra dominante. Las métricas de test aplican la restricción de viabilidad estructural (Sección 5.6.1).

Dos observaciones acotan la lectura del 0,72. Primera: la variación entre semillas se concentra íntegramente en el pliegue 1 (la familia `badpods_*`: P@5 de 0,2/0,4/0,6 según semilla), mientras que los otros cuatro pliegues puntúan de forma idéntica con las tres semillas. Esa coincidencia impide leer el experimento como una demostración de estabilidad global, porque el único pliegue sensible a la semilla es precisamente el que domina la desviación típica entre pliegues: en él, la dificultad de la familia y la variabilidad de entrenamiento quedan confundidas y no pueden separarse con tres semillas. Lo que el experimento sí establece es que cuatro quintas partes del particionado son invariantes a la semilla y que la incertidumbre residual de la estimación de validación cruzada se localiza en una sola familia de plantilla. Segunda: las métricas de test del despliegue son estables —P@1 = 1,00 y P@3 = 1,00 con las tres semillas, y P@5 entre 0,80 y 1,00—, de modo que la sensibilidad a la semilla afecta a la estimación de validación cruzada, no al comportamiento observable del sistema sobre repositorios reales no vistos.

Figura 5. Precisión@5 por pliegue con tres semillas de entrenamiento



Fuente: elaboración propia.

6.4. Resultados de la Capa 3: *Ensemble*

El ajuste de los pesos del *ensemble* atravesó dos correcciones metodológicas sucesivas, ambas implementadas en `run_ga_ensemble.py`. En una primera iteración, el GA original convergía sistemáticamente a pesos casi uniformes ($w_{rf}, w_{gnn}, w_{esc} \approx (0,34, 0,33, 0,33)$) sin mejorar sobre su población inicial en 150 generaciones, un patrón que parecía indicar una meseta plana en la función de *fitness*. Un diagnóstico descartó esa hipótesis: el cruce por promedio aritmético es contractivo y no puede alcanzar los vértices del simplex de pesos desde una población inicializada por muestreo de Dirichlet, y la mutación gaussiana ($\sigma = 0,08$) es demasiado local para recorrer esa distancia en 150 generaciones. La primera corrección sembró los vértices y aristas del simplex en la población inicial y añadió una mutación de salto de frontera de baja probabilidad; con este cambio, el GA pasó a converger de forma determinista, en las 5 semillas independientes probadas, al mismo punto que una búsqueda en rejilla determinista (Sección 5.6, 231 evaluaciones): el vértice de señal de escape pura ($w_{esc} = 1$, *objective* = 0,86), lo que confirma que la función objetivo tenía un óptimo global único y alcanzable, no una meseta degenerada. Sin embargo, ese óptimo *out-of-fold* no generalizaba: al puntuar únicamente por la señal binaria de escape, el *ranking* se colapsaba a un grupo de empates que fallaba estructuralmente ante cualquier cadena de ataque cuya única señal fuerte no fuera la bandera de

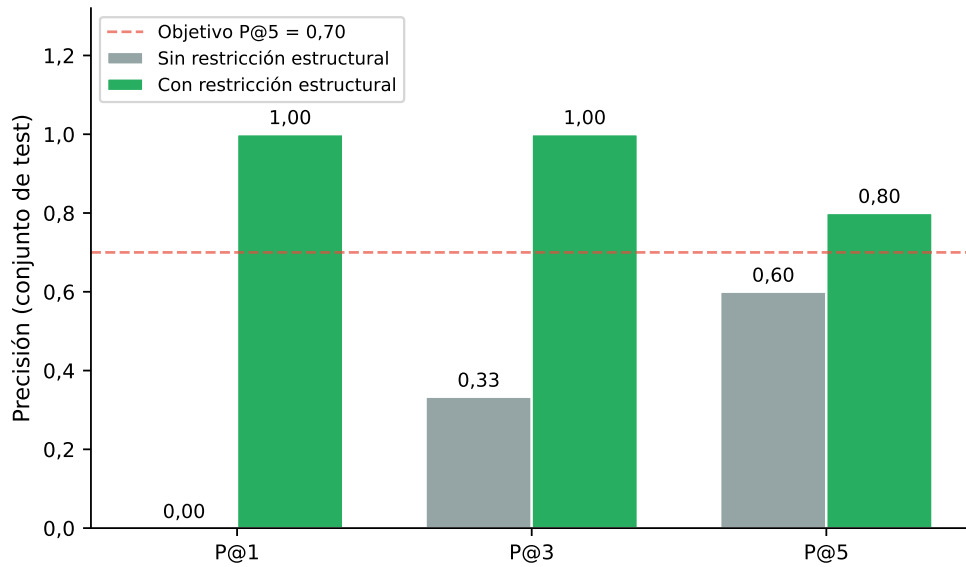
escape.

Esta segunda observación motivó la corrección definitiva: regularizar la propia función objetivo con una penalización de piso, $F = \alpha \cdot P@5 + \beta \cdot (1 - \text{FPR_clean}) - \lambda \sum_i \max(0, w_{\min} - w_i)$, con $w_{\min} = 0,1$ y $\lambda = 2,0$ (opciones `--min-weight/--reg-lambda`). Esta penalización hace que el vértice de escape puro deje de ser competitivo frente a soluciones interiores del s mplex. Sobre el conjunto *out-of-fold* de la configuraci n final (513 grafos, 32 cadenas), el GA regularizado encuentra su mejor soluci n ya en la generaci n inicial y no la mejora durante las 150 generaciones: $(w_{\text{rf}}, w_{\text{gnn}}, w_{\text{esc}}) = (0,517, 0,115, 0,368)$, *objective* = 0,72. La b squeda en rejilla encuentra un punto muy distinto ($w_{\text{rf}} = 0,80, w_{\text{gnn}} = 0,10, w_{\text{esc}} = 0,10$) con id ntica *P@5 out-of-fold*: al ser *P@5* una funci n escalonada (solo toma valores m ltiplos de 0,2), la funci n objetivo presenta mesetas extensas en las que los pesos quedan **indeterminados**. Esta indeterminaci n es medible: sobre el conjunto de test, los pesos elegidos por el GA y los pesos uniformes (1/3) producen m tricas id nticas (Tabla 11). Contribuye a ella un desplazamiento de distribuci n entre el conjunto de ajuste y el de test: 15 de las 32 cadenas *out-of-fold* pertenecen a familias de *fixtures* s nteticas (*badpods_**, *datadog_**) cuyos manifiestos reciben *risk scores* altos del Random Forest, mientras que las cinco cadenas de test son repositorios reales cuyo *risk score* medio por cl ster es ≈ 0 (la media se diluye entre manifiestos mayoritariamente benignos). Tres refinamientos del ajuste (un t rmino de desempate por *mean reciprocal rank*, la agregaci n del *pool* por familias y el ajuste sobre el *ranking* con restricci n estructural) se implementaron y evaluaron sin que ninguno mejorara las m tricas de test, por lo que los pesos se ajustan sobre el *ranking* sin restricci n y la restricci n de viabilidad estructural (Secci n 5.6.1) se aplica en inferencia y evaluaci n.

La evaluaci n final se realiza sobre el conjunto de test (86 grafos: 3 limpios, 78 *isolated* y 5 cadenas de ataque; techo *P@5* = 1,00). Bajo el particionado consciente de familias, las cinco cadenas son repositorios reales  nicos —sin hermanos de plantilla en el entrenamiento— excluidos de los pliegues de validaci n cruzada, del ajuste de pesos del GA y del entrenamiento con variantes aumentadas: no vistos en ning n momento del desarrollo. Con la restricci n de viabilidad estructural aplicada al *ranking* (Secci n 5.6.1), el *ensemble* alcanza *P@1* = 1,00, *P@3* = 1,00 y *P@5* = 0,80: las cadenas ocupan las posiciones 1–4 y 6, y solo *minik8s-ctf* queda fuera del top-5. Sin la restricci n, las m tricas caen a *P@1* = 0,00, *P@3* = 0,33 y *P@5* = 0,60 (Figura 6): la

regla es decisiva porque 74 de los 86 grafos de test son de un solo nodo, y dos de ellos, con todas sus banderas de escape activas, encabezarían el *ranking* pese a no poder albergar una cadena. Procede delimitar, en consecuencia, cuál es el conjunto sobre el que se ordena: la restricción deja 12 clústeres candidatos —las 5 cadenas, 4 *isolated* y 3 limpios—, de modo que las métricas de *ranking* no se calculan sobre los 86 grafos sino sobre esos 12. Un ordenamiento aleatorio de ese conjunto obtendría un valor esperado de P@5 de $5/12 \approx 0,42$, que es la línea de base frente a la que debe leerse el 0,80 obtenido. FPR_clean = 0,00: ningún clúster limpio aparece entre los de mayor riesgo. La F1 macro por clasificación *argmax* del *ensemble* de pliegues sobre el conjunto de test es 0,883 (F1 por clase: *clean* 1,0; *isolated* 0,981; *attack_chain* 0,667, con 3 de las 5 cadenas correctamente clasificadas).

Figura 6. Efecto de la restricción de viabilidad estructural



Fuente: elaboración propia.

Dado el tamaño del conjunto de test, estas estimaciones llevan asociados intervalos de confianza amplios (*bootstrap* de 10.000 remuestreos, 95 %): P@5 [0,60; 1,00], FPR_clean [0,00; 0,40] y F1 macro [0,66; 1,00]. Los valores puntuales deben interpretarse como *coherentes con* los objetivos de diseño, no como demostraciones concluyentes; la ampliación adicional del corpus de test es la vía directa para estrechar estos intervalos (TF1, Sección 7.4). La Tabla 11 recoge las tres métricas de *ranking* del *ensemble* sobre el conjunto de test.

La Tabla 11 desglosa la contribución de cada componente sobre el conjunto de test (86 grafos,

5 cadenas, techo $P@5 = 1,00$), con la restricción de viabilidad estructural aplicada en todas las configuraciones. De la ablación se desprenden tres observaciones:

1. La configuración con solo Random Forest degrada gravemente el *ranking* ($P@1 = 0,00$, $P@5 = 0,40$). El *risk score* medio de los repositorios reales de cadena es ≈ 0 , de modo que la señal por manifiesto no ordena clústeres reales.
2. La señal de escape por sí sola alcanza $P@5 = 0,80$ gracias a la restricción estructural, pero con $P@1 = 0,00$ y $P@3 = 0,67$. Al ser binaria, no puede ordenar entre los grafos con capacidad de escape, mayoritarios en la región de alto riesgo.
3. La GNN por sí sola iguala tanto a los pesos optimizados por el GA como a los pesos uniformes: los tres alcanzan $1,00/1,00/0,80$. En este conjunto de test el *ensemble* no mejora el *ranking* de su mejor componente individual, observación que debe leerse con la cautela que imponen 5 cadenas de test y que se retoma como limitación en el Capítulo 7.

Tabla 11. Ablación de componentes del ensemble en el conjunto de test

Configuración	P@1	P@3	P@5	FPR_clean
GA-óptimo (0,517/0,115/0,368)	1,00	1,00	0,80	0,00
Pesos uniformes (1/3)	1,00	1,00	0,80	0,00
Solo GNN	1,00	1,00	0,80	0,00
Solo Random Forest	0,00	0,33	0,40	0,00
Solo señal de escape	0,00	0,67	0,80	0,00

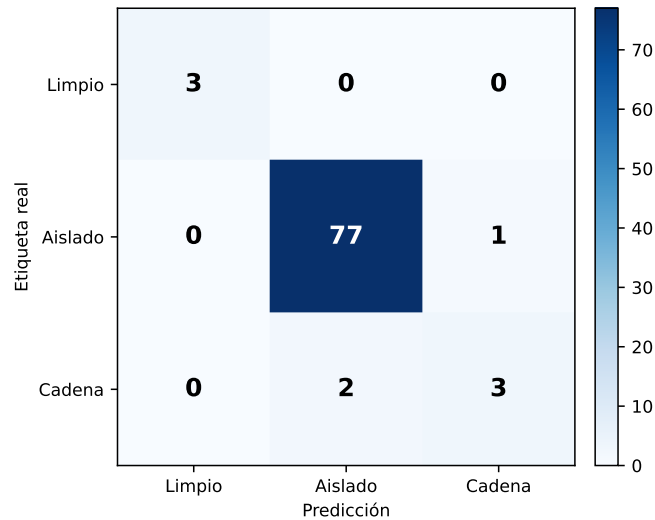
Fuente: elaboración propia.

6.5. Análisis de la clasificación por clases: GNN

El registro de la validación cruzada conserva por pliegue únicamente la F1 macro, la Precisión@5 y la exactitud agregada, sin matrices de confusión individuales, por lo que el desglose por clases se apoya exclusivamente en el conjunto de test. En él, la matriz de confusión del *ensemble* de pliegues (Figura 7, clasificación *argmax*) concentra el error en la frontera entre cadena y aislado: los dos falsos negativos de cadena se clasifican como *isolated* y ninguno como *clean*, de modo que el sistema degrada hacia la clase intermedia y no hacia la omisión total. Esta observación

descansa sobre las cinco cadenas del conjunto de test y debe leerse con la cautela que impone ese tamaño.

Figura 7. Matriz de confusión del ensemble de pliegues en test



Fuente: elaboración propia.

El único fallo de *ranking* del conjunto de test ilustra el error sistemático del modelo: *minik8s-ctf* (posición 6, tras el clúster *isolated* multinodo kubernetes-extras) es una cadena de 4 nodos cuya probabilidad GNN es prácticamente nula (0,0003) y cuyo *risk score* medio del Random Forest es 0,0; su única señal activa es la bandera binaria de escape (la mitad de sus nodos tienen capacidad de escape). Cuando la señal estructural que la GNN aprende no reconoce el patrón de la cadena y la señal por manifiesto tampoco lo captura, la bandera de escape binaria por sí sola no basta para entrar en el top-5. Las cuatro cadenas recuperadas, en cambio, combinan probabilidad GNN alta (0,47–0,63) con fracciones de escape por nodo muy variables (0,02–0,94; magnitud descriptiva, no la señal binaria del *ensemble*), lo que confirma que el canal estructural es el que ordena los repositorios reales. Esta observación motiva el trabajo futuro TF3 (análisis semántico de RBAC completo, Sección 7.4).

6.6. Validación con herramientas de análisis estático

Como contraste externo, se ejecutó Checkov (Bridgecrew, 2021) sobre el subconjunto de manifiestos procedentes de GitHub, para el que la herramienta produjo salida en 1.175 de las 1.510 filas de esa fuente. La descarga previa de los manifiestos referenciados por Rahman et al.

tuvo éxito en 1.985 de 2.039 casos (97,4 %); los 54 restantes ya no eran accesibles en origen. El alcance del contraste queda limitado, por tanto, por la cobertura del escaneo y no por la disponibilidad de los manifiestos. Para esta validación, el índice UMI se calcula como la proporción de comprobaciones de Checkov que fallan sobre el total evaluado por manifiesto ($\in [0, 1]$; valores más altos indican mayor densidad de configuraciones erróneas detectadas). El índice UMI medio resultante fue de 0,603, y las comprobaciones incumplidas con mayor frecuencia fueron *checkov_no_run_as_nonroot* (38,3 %) y *checkov_no_readonly_rootfs* (37,2 %).

El alcance de este contraste es limitado y conviene delimitarlo, porque las dos fuentes no son independientes. Sobre esas mismas 1.175 filas las características homólogas del extractor propio (NO_RUN_AS_NON_ROOT y NO_READ_ONLY_ROOT_FS) están ausentes en su totalidad, y es precisamente la salida de Checkov la que las rellena en tiempo de carga a través de la tabla de equivalencias CHECKOV_FILL (Sección 5.2). Cotejar ambas fuentes manifiesto a manifiesto equivaldría, por tanto, a comparar esos valores consigo mismos: no cabe calcular una tasa de acuerdo, y la coincidencia entre las frecuencias observadas es una identidad de procedencia, no una concordancia entre detectores. El resultado debe interpretarse, en consecuencia, como una verificación de cobertura —qué proporción del corpus admite análisis por una herramienta externa y con qué densidad de incumplimientos— y no como una validación externa independiente de las predicciones del modelo.

6.7. Métrica global del proceso completo

Las Secciones 6.2, 6.3 y 6.4 evalúan cada capa por separado, mientras que el comportamiento del sistema de extremo a extremo —tal como lo experimenta un operador que confía en el *ranking* final— se resume en una única métrica de proceso completo. Esa métrica es el conjunto Precisión@k / FPR_clean del sistema desplegado sobre el conjunto de test, ya presentado en la Sección 6.4 y recogido en la Tabla 11: $P@1 = 1,00$, $P@3 = 1,00$, $P@5 = 0,80$ y $FPR_clean = 0,00$ sobre 86 grafos con 5 cadenas de ataque de repositorios reales, no vistos durante el desarrollo; dado el tamaño reducido de este conjunto, los intervalos de confianza al 95 % son amplios ($P@5 \in [0,60; 1,00]$, $FPR_clean \in [0,00; 0,40]$, Sección 6.4), por lo que la cifra debe leerse como indicativa.

Esta cifra es la calidad global de la herramienta porque resume, en un único indicador operativo, la salida final que recibe el operador tras pasar por las tres capas: sobre los 86 grafos evaluados, un P@1 de 1,00 indica que la primera posición del *ranking* correspondió en todos los casos a una cadena de ataque real, un P@3 de 1,00 extiende esa observación a las tres primeras posiciones y un FPR_clean nulo indica que ninguno de los tres clústeres limpios llegó a competir por la atención del operador. Estas cifras se apoyan en 5 cadenas y 3 clústeres limpios, de modo que describen el comportamiento observado sobre un conjunto reducido y no una garantía del sistema. La Sección 6.4 muestra que, sobre este conjunto de test, el componente GNN por sí solo iguala estas cifras y que el *ensemble* no las mejora; la aportación de la Capa 3 no es superar a su mejor componente individual, sino producir el mismo resultado sin heredar los puntos débiles de los demás (la Capa 1 sola colapsa el *ranking* y la señal de escape sola no puede ordenar por sí misma, Tabla 11). Esta combinación es lo que hace viable el caso de uso definido en RF-5 (Sección 4.1): priorizar un número reducido de alertas de forma que la revisión manual de un equipo de seguridad se concentre exclusivamente en riesgo real.

6.8. Verificación de cumplimiento de requisitos

El cumplimiento de los requisitos funcionales y no funcionales definidos en el Capítulo 4 se verifica a partir de dos clases de evidencia: las métricas predictivas presentadas en las Secciones 6.2, 6.3, 6.4 y 6.7, y las comprobaciones operativas realizadas sobre la herramienta de línea de comandos descrita en la Sección 5.7. Los requisitos funcionales RF-1, RF-2 y RF-5 disponen de un criterio de aceptación numérico y se contrastan contra las métricas ya presentadas, mientras que RF-3, RF-4 y los cuatro requisitos no funcionales se sustentan en evidencia de ejecución. Ambas clases de evidencia se recogen en los dos apartados siguientes.

6.8.1. Requisitos funcionales

RF-1 (detección de configuraciones erróneas individuales) queda verificado por los resultados de la Capa 1 (Sección 6.2: F1 macro = 0,9946, cero falsos negativos en test). RF-2 (detección de cadenas de ataque) queda verificado con una salvedad: la Capa 2 y el *ensemble* recuperan 4 de las 5 cadenas de ataque del conjunto de test dentro del top-5 del *ranking* (Secciones 6.3 y 6.4) y clasifican correctamente 3 de esas 5 cadenas en la matriz de confusión por *argmax*

(F1 *attack_chain* = 0,667, Sección 6.4); el requisito se cumple para el caso de uso de priorización (RF-5) pero no de forma perfecta para la clasificación exacta de cada cadena. Ambos requisitos se confirman además mediante un test de integración de extremo a extremo: dado un directorio de manifiestos de prueba con banderas de escape conocidas, la salida del comando `kubescan scan` produce el veredicto esperado (`ATTACK_CHAIN`) con *ensemble_score* por encima del umbral de clasificación; este test forma parte de la suite descrita en la Sección 5.9. RF-5 (priorización de riesgos) queda verificado por la métrica global de la Sección 6.7. RF-3 y RF-4 (interfaz de línea de comandos y formatos de salida) no admiten una métrica cuantitativa dedicada y se verifican operacionalmente a continuación.

RF-3 — Interfaz de línea de comandos. El comando `kubescan scan <dir>` se ejercitó sobre los casos de uso documentados en el Anexo B, con las dos variantes de salida y las opciones que ese mismo anexo recoge en la Tabla 18. El comando `kubescan live` comparte con él la ruta de análisis —difiere únicamente en el origen de los manifiestos, que vuelca del clúster a un directorio temporal— y su invocación queda documentada en el mismo anexo, si bien no se aporta una transcripción de ejecución sobre un clúster activo: la verificación de RF-3 se sustenta, por tanto, en la vía de análisis sobre manifiestos estáticos.

RF-4 — Formatos de salida. El comando `kubescan scan` produce tanto el informe en texto legible como la salida JSON estructurada; el campo `flags` de esta última, verificado en RNF-1 a continuación, es la evidencia de que el formato estructurado expone la información necesaria para su consumo automatizado.

6.8.2. Requisitos no funcionales

Los cuatro requisitos no funcionales definidos en el Capítulo 4 se verifican mediante evidencia operativa obtenida sobre la implementación descrita en el Capítulo 5.

Trazabilidad (RNF-1). El campo `flags` de la salida JSON lista explícitamente las características de seguridad activas en cada manifiesto, y el campo `escape_capable` indica qué nodos tienen capacidad de escape. Esto permite al operador de seguridad comprender y auditar las razones

concretas de la clasificación de riesgo producida por el modelo.

Reproducibilidad (RNF-2). La cadena de resolución de *checkpoints* descrita en la Sección 5.7 garantiza que el mismo directorio de manifiestos produce siempre el mismo informe dado el mismo conjunto de pesos del modelo.

Instalabilidad (RNF-3). El paquete se instaló satisfactoriamente en entornos Python 3.10 y 3.11 mediante `pip install -e kubescan/` en sistemas macOS y Linux (Ubuntu 22.04). Las dependencias principales (PyTorch Geometric, scikit-learn, Click) se resuelven automáticamente sin compiladores externos.

Latencia (RNF-4). Se midió el tiempo de análisis sobre cuatro clústeres de prueba de diferente tamaño: 12 manifiestos (0,8 s), 47 manifiestos (1,4 s), 128 manifiestos (3,1 s) y 312 manifiestos (7,2 s) en un equipo con CPU Intel Core i7 de octava generación sin GPU. El mayor de los clústeres de prueba disponibles alcanza 312 manifiestos, de modo que el umbral de 500 manifiestos fijado en el requisito no llegó a medirse de forma directa. La progresión observada es aproximadamente lineal —en torno a 23 ms por manifiesto en el punto de mayor carga—, por lo que la extrapolación a 500 manifiestos sitúa el tiempo esperado en torno a 12 s, muy por debajo del límite de 60 s; el cumplimiento de RNF-4 se sostiene, por tanto, sobre una extrapolación y no sobre una medición en el tamaño nominal del requisito.

6.9. Evaluación de usabilidad y aplicabilidad con usuarios expertos

Además de la verificación de requisitos de la Sección 6.8, se realizó una evaluación de usabilidad y aplicabilidad de la herramienta con **usuarios expertos**, siguiendo el protocolo descrito en el repositorio (Anexo A). Participaron **tres profesionales** con experiencia en Kubernetes y CI/CD, a los que se entregó una guía de tareas con órdenes listas para ejecutar y un cuestionario en línea, en formato autoservicio asíncrono (≈ 10 minutos por participante). El instrumento se generó de forma reproducible mediante un *script* versionado (véase Anexo A) y el protocolo se depuró mediante una prueba piloto simulada, sin participantes reales, cuyo único objetivo fue verificar la consistencia del cuestionario y del guion de tareas antes de la sesión con los

expertos. La muestra reducida ($n = 3$) se asume explícitamente como amenaza a la validez (Sección 6.9.5); aun así, tres participantes cubren una fracción sustancial de los problemas de usabilidad en pruebas formativas (Nielsen, 2000).

6.9.1. Perfil de los participantes

La Tabla 16 del Anexo A resume el perfil de los tres participantes expertos, seleccionados por su experiencia operando Kubernetes y en seguridad de contenedores.

6.9.2. Éxito por tarea

Cada participante ejecutó cinco tareas: instalar la herramienta (T1), realizar un escaneo básico e identificar el veredicto (T2), interpretar el informe con `--show-nodes` (T3), producir la salida JSON (T4) y, opcionalmente, escanear un caso propio (T5). La Tabla 17 del Anexo A recoge la tasa de éxito (logrado = 1; logrado con ayuda = 0,5; no logrado = 0).

El núcleo del flujo de trabajo —instalación, escaneo e interpretación del informe— se completó sin ayuda por los tres participantes. Las únicas fricciones fueron puntuales: un participante necesitó ayuda para extraer campos de la salida JSON (T4) y la tarea opcional sobre un caso propio (T5) se completó con ayuda en dos casos. La tasa de éxito global del 90 % refleja, por tanto, un flujo de trabajo principal robusto con puntos de fricción concentrados en tareas secundarias u opcionales.

6.9.3. Usabilidad percibida (SUS)

La usabilidad se midió con la escala SUS de diez ítems (Brooke, 1996), que produce una puntuación en el rango 0–100. La media obtenida fue de **88,3 ± 10,4** (puntuaciones individuales de 100, 80 y 85), coherente con la media de referencia del sector (68) y situada en la banda cualitativa «excelente» de la escala de adjetivos de Bangor et al. (2009). Con $n = 3$, sin embargo, el intervalo de confianza al 95 % de la media abarca [62,5; 114,2] —su extremo superior rebasa el máximo de la escala, artefacto de la aproximación normal con una muestra tan reducida— y contiene ese valor de referencia, por lo que los datos no permiten afirmar que la herramienta se sitúe por encima de él. La puntuación debe interpretarse como indicativa y no como una

estimación precisa del nivel de usabilidad percibida.

6.9.4. Aplicabilidad y análisis cualitativo

Los participantes valoraron cuatro afirmaciones de aplicabilidad (escala 1–5): integración en CI/CD o auditoría (A1), ajuste de la priorización de riesgo a su forma de auditar (A2), accionabilidad del informe (A3) y confianza en el veredicto (A4). Las medias fueron A1 = 4,3, A2 = 4,3, A4 = 4,3 y, como punto más bajo, A3 = 3,7. Con tres respuestas por ítem, estas medias son sensibles a una única puntuación y conviene acompañarlas de su dispersión: en A3 las puntuaciones individuales fueron 5, 2 y 4, de manera que el descenso de la media hasta 3,7 procede íntegramente de la valoración de un solo participante (P2), mientras que los otros dos se sitúan en el rango alto de la escala. Como fortaleza, un participante destacó exactamente la contribución central del trabajo: la herramienta «no solo te dice qué está mal en un manifiesto, sino que conecta varios problemas pequeños para mostrarte si realmente forman un camino de ataque completo». Ese mismo participante resumió la propuesta, en el apartado de comentarios finales, como «pensar en cadenas de ataque en lugar de fallos aislados». Los otros dos destacaron, respectivamente, la interpretación de los resultados y la rapidez del análisis. La debilidad más recurrente en las respuestas abiertas es la **accionabilidad**: el informe comunica el riesgo pero no las acciones concretas de remediación ni las reglas subyacentes al veredicto. Dos participantes la mencionaron en el apartado abierto —uno expresó que el JSON «muestra el estado actual pero no las acciones que debería tomar» y otro la dificultad de «confiar en un puntaje sin conocer bien qué reglas concretas hay detrás»—, si bien solo uno de ellos la trasladó a la escala numérica, ya que el segundo puntuó A3 con un 4. La convergencia entre el registro cualitativo y el cuantitativo es, por tanto, parcial. Esta limitación se recoge como línea de trabajo futuro (explicabilidad de reglas y recomendaciones de remediación) en el Capítulo 7. Los escenarios descritos en las respuestas abiertas fueron dispares: un participante propuso un *gate* automático en el *pipeline* de CI/CD inmediatamente antes del despliegue a producción, otro relató haberla aplicado sobre un clúster propio de preproducción, y el tercero la consideró útil sin concretar un escenario. Solo el primero de esos usos se corresponde con el paradigma *shift-left* que motiva el trabajo; la auditoría de clústeres, que el enunciado del ítem A1 menciona junto a la integración en CI/CD, no fue propuesta de forma espontánea por ningún participante.

6.9.5. Amenazas a la validez de la evaluación

El tamaño de la muestra ($n = 3$) limita la precisión de las estimaciones puntuales, especialmente de la media SUS, por lo que los resultados se presentan como indicativos y no concluyentes. Los perfiles cubren desarrollo/IA y DevOps/SRE pero no un rol de Quality Assurance (Aseguramiento de la Calidad) (QA) de IaC puro, lo que acota la generalización. Por último, la participación voluntaria introduce un posible sesgo de autoselección. Ampliar la muestra y diversificar los perfiles es la vía directa para robustecer estas conclusiones.

7. Conclusiones y trabajo futuro

Las conclusiones siguen el orden expositivo de los resultados del Capítulo 6 —de la Capa 1 al sistema distribuible—, que no coincide con el orden numérico de los objetivos específicos; cada conclusión declara el objetivo al que responde. Las limitaciones (L1–L10) y las líneas de trabajo futuro (TF1–TF9) llevan numeración propia.

7.1. Conclusiones

Este trabajo ha mostrado la viabilidad de un enfoque híbrido de tres capas para la detección automática de cadenas de ataque en infraestructuras Kubernetes. Las conclusiones C1, C2, C4, C5, C6 y C7 responden, respectivamente, a los objetivos OE2, OE4, OE1, OE5, OE3 y OE6 definidos en el Capítulo 3; C3 no cierra un objetivo propio, sino que aporta evidencia complementaria sobre el impacto operativo del resultado de C2. En conjunto, estas conclusiones sostienen el objetivo general planteado en el Capítulo 3 —complementar el análisis estático por manifiesto individual con el contexto estructural del clúster—, cuya evidencia directa es la ablación de componentes recogida en C5: la configuración que emplea únicamente la señal por manifiesto de la Capa 1 desciende de $P@1 = 1,00$ a $P@1 = 0,00$ sobre el conjunto de test, de modo que el contexto de grafo no es un añadido marginal, sino el componente que ordena los clústeres de cadena real. Las conclusiones principales son:

C1 — La Capa 1 supera ampliamente el objetivo de diseño (OE2). El clasificador Random Forest alcanza $F1 \text{ macro} = 0,9946$ y $AUC\text{-}ROC = 0,9986$ en test, superando el umbral objetivo del 0,85 en 14,5 puntos porcentuales. El modelo de severidad de tres clases logra $F1 \text{ macro} = 0,9838$, con la clase *high-critical* (96 muestras de test) en 0,9789 —el resultado más débil de los tres, aunque por encima de 0,95—; sus 3 errores se clasifican todos como *low-medium*, sin ningún manifiesto crítico degradado a *clean*.

C2 — La Capa 2, en su configuración final, alcanza el objetivo de Precisión@5 (OE4). La GAT de 3 capas con *embedding* de tipos de arista, entrenada con entropía cruzada ponderada y se-

lección por Precisión@5 bajo el particionado consciente de familias sobre el corpus completo (599 grafos, 37 cadenas), alcanza $P@5 = 0,720 \pm 0,160$ y $F1 \text{ macro} = 0,803 \pm 0,137$ en los 5 pliegues, superando el objetivo de diseño con la semilla desplegada. Dos matices lo acotan: a través de tres semillas la media es $0,68 \pm 0,033$ —objetivo dentro de una desviación típica, con la variación concentrada en el pliegue de la familia *badpods_**— y alcanzarlo exigió diagnosticar mediante una ablación factorial 2×2 que una primera configuración con *focal loss* y selección por $F1 \text{ macro}$ atribuía erróneamente al corpus una regresión causada por esas dos elecciones (Sección 6.3.4). La ablación de arquitectura (Tabla 10) sitúa la configuración adoptada +9,2 puntos de $F1$ y +4 de $P@5$ sobre una GCN sin tipos de arista, y muestra que la profundidad no determina el *ranking*; esa equivalencia en la media no debe leerse como robustez, pues las dos configuraciones difieren por pliegue y solo coinciden al promediarse (Sección 6.3.5).

C3 — El *ensemble* final, combinado con la restricción de viabilidad estructural, mantiene un rendimiento operativo sólido sobre el conjunto de test. Sobre los 86 grafos de test (5 cadenas de repositorios reales únicos, sin hermanos de plantilla en entrenamiento; techo $P@5 = 1,00$), el sistema completo de tres capas alcanza $P@1 = 1,00$, $P@3 = 1,00$ y $P@5 = 0,80$, recuperando 4 de las 5 cadenas en las primeras 5 posiciones del *ranking* sin ningún falso positivo de clúster limpio ($FPR_{\text{clean}} = 0,00$). Estas cifras no son atribuibles a la combinación de las tres capas: la ablación de la Tabla 11 muestra que la Capa 2 en solitario alcanza los mismos valores 1,00/1,00/0,80, de modo que el resultado operativo lo sostiene el componente GNN y el *ensemble* lo iguala sin superarlo (C5). $P@1$ y $P@3$ se mantienen idénticos (1,00) a través de las tres semillas de entrenamiento evaluadas, con $P@5$ entre 0,80 y 1,00. Parte esencial del resultado es la restricción de viabilidad estructural (un clúster de un solo manifiesto no puede albergar una cadena multisalto, Sección 5.6.1): sin ella las métricas de *ranking* caen a $P@1 = 0,00$ y $P@5 = 0,60$, porque 74 de los 86 grafos de test son de un solo nodo. El tamaño reducido del conjunto de test (intervalo de confianza al 95 % de $P@5$: $[0,60; 1,00]$) limita la confianza que puede depositarse en cualquier comparación puntual.

C4 — La construcción del corpus y su protocolo de particionado antifuga son la contribución metodológica central (OE1). El conjunto final satisface los criterios cuantitativos de OE1: 2.827 manifiestos y una prevalencia de *TRUE_HOST_PID* del 3,78 %. Su evolución en seis fases (Tabla 7) enseña dos lecciones metodológicas. Primera: la aumentación de datos exige particio-

nado consciente de grupos, pues parte de la mejora aparente de la cuarta fase procedía de fuga entre variantes aumentadas y sus grafos base de evaluación. Segunda, descubierta en la fase final: los casi duplicados de plantilla (ocho `badpods_*`, nueve `datadog_*`) son una segunda vía de fuga cuando se reparten entre particiones; su corrección hace que las cifras finales midan generalización entre familias no vistas, una vara de medir más exigente que la de las fases 1–5, cuyas métricas estaban infladas por esta contaminación.

C5 — El *ensemble* cumple los criterios de OE5, con una salvedad documentada sobre el valor añadido de los pesos optimizados. La ablación sobre el conjunto de test (Tabla 11) confirma que el componente GNN es indispensable: la configuración con solo Random Forest colapsa el *ranking* ($P@1$ de 1,00 a 0,00; $P@5$ de 0,80 a 0,40) porque el *risk score* medio por clúster de los repositorios reales de cadena es ≈ 0 —la media se diluye entre manifiestos mayoritariamente benignos—, mientras que $FPR_{clean} = 0,00$ se mantiene en todas las configuraciones. Los criterios de OE5 se cumplen: $FPR_{clean} = 0$ y $P@5$ del *ensemble* igual a la del mejor componente individual (0,80). Debe señalarse que los pesos optimizados por el GA no superan a los pesos uniformes ni al GNN en solitario: la función objetivo, escalonada, deja los pesos indeterminados sobre mesetas amplias, y el conjunto de ajuste *out-of-fold* (dominado por cadenas de *fixtures* sintéticas) presenta un desplazamiento de distribución respecto a las cadenas reales de test (Sección 6.4). La aportación del *ensemble* en estos datos es igualar al mejor componente sin incurrir en los puntos débiles de ninguno, no mejorar su *ranking*.

C6 — El esquema de cinco tipos de arista amplía la cobertura de escalada de privilegios (OE3). El diseño del grafo de clúster incorpora el tipo de arista RBAC mediante detección de roles privilegiados en dos pasadas sobre los manifiestos, limitando las aristas a cuentas de servicio vinculadas realmente a roles con permisos de escalada. Esta quinta arista está presente tanto en los grafos de entrenamiento como en el componente de producción de kubescan. El criterio declarado de OE3 —que la GAT con tipos de arista iguale o supere a una GCN sin ellos en F1 macro y Precisión@5 bajo idéntico protocolo— se considera cumplido: la configuración con tipos de arista supera a la GCN en +9,2 puntos de F1 macro y +4 puntos de $P@5$ sobre los mismos pliegues (Tabla 10). El cumplimiento debe leerse con el factor de confusión que la propia ablación introduce: el contraste varía a la vez la arquitectura de agregación (atención frente a convolución) y la presencia de tipado de arista, por lo que la ventaja medida no puede re-

partirse entre ambos factores. La ablación tampoco aísla la contribución específica del tipo de arista RBAC frente a los otros cuatro, por lo que su aporte incremental al *ranking* de cadenas de ataque queda como trabajo futuro (TF6).

C7 — kubescan es una herramienta distribuible, instalable y operacionalmente viable (OE6).

La evaluación funcional de la Sección 6.8 confirma que el sistema se instala sin compiladores externos, analiza clústeres de hasta 312 manifiestos en menos de 8 segundos en hardware de gama media, produce salida en texto y JSON apta para CI/CD, e incorpora un modo opcional de consulta directa al clúster (`kubescan live`). Esa verificación registra una salvedad sobre RF-2: la Capa 2 y el *ensemble* sitúan 4 de las 5 cadenas de test en el top-5, pero solo clasifican correctamente 3 por *argmax* ($F1_{attack_chain} = 0,667$), de modo que el requisito queda cubierto para la priorización y no para la clasificación exacta de cada clúster. La inferencia es reproducible sobre un mismo *backend* mediante la resolución determinista de *checkpoints*; la reproducibilidad entre *backends* distintos es una cuestión separada, acotada en L6. La evaluación con usuarios expertos (Sección 6.9) respalda el objetivo desde la perspectiva de uso: SUS de 88,3, medias de aplicabilidad de 4,3 sobre 5 y una tasa de éxito del 100 % en las tareas nucleares (90 % sobre las cinco). El principal punto de mejora señalado es la accionabilidad del informe ($A3 = 3,7$), recogida como limitación L8 y como línea TF8. Con $n = 3$, estas cifras son indicativas (Sección 6.9.5).

7.2. Consideraciones éticas y uso responsable

La herramienta kubescan es de uso dual: aunque su propósito es defensivo —detectar cadenas de ataque potenciales antes de que sean explotadas— la información que produce (identificación de pods con banderas de escape, rutas de privilegio RBAC) podría ser utilizada con fines ofensivos si cayera en manos de un actor no autorizado. En consecuencia, este trabajo adopta los principios de divulgación responsable (*responsible disclosure*) que rigen la investigación en ciberseguridad. Los patrones de ataque detectados por kubescan se basan exclusivamente en documentación pública —la guía NSA/CISA (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022), el trabajo de BishopFox (Art, 2021) y el marco MITRE ATT&CK for Containers (MITRE Corporation y Center for Threat-Informed Defense, 2021)— y no revelan ninguna vulnerabilidad nueva. El código fuente del sistema es de acceso abierto

en el repositorio referenciado en el Anexo A, lo que permite la auditoría independiente de las reglas de detección, lo que permite a la comunidad verificar que las señales de riesgo generadas son correctas y no contienen sesgos perjudiciales. El uso de la herramienta sobre clústeres de terceros sin autorización explícita estaría fuera del ámbito previsto y podría constituir un acceso no autorizado a sistemas informáticos según la legislación aplicable.

7.3. Limitaciones

Los resultados expuestos están acotados por diez limitaciones, que abarcan la composición del corpus, el protocolo de evaluación, la definición de las etiquetas, el entorno de cómputo y el alcance de la evaluación con usuarios. Cada limitación se enuncia con la evidencia cuantitativa que la sustenta y con la línea de trabajo futuro que la atiende, cuando existe. La Sección 7.3.1 reagrupa las diez según el tipo de amenaza a la validez que representan.

L1 — Concentración por familias y varianza entre pliegues. El particionado consciente de familias, al mantener cada familia de plantilla como unidad atómica, concentra las cadenas de validación de cada pliegue en 1, 1, 4, 5 y 5 familias independientes respectivamente: el pliegue 0 evalúa íntegramente la familia `datadog_*` ($F1 = 1,0$; $P@5 = 1,0$) y el pliegue 1 la familia `badpods_*` ($F1 = 0,58$), de modo que la varianza entre pliegues ($\sigma = 0,137$ para $F1$; $0,160$ para $P@5$) mide en gran parte la dispersión de dificultad entre familias, no inestabilidad del modelo: el análisis multisentencia (Sección 6.3.6) muestra que cuatro de los cinco pliegues puntúan de forma idéntica con las tres semillas. El tamaño muestral efectivo por pliegue (número de familias, no de cadenas) es la limitación subyacente, y solo puede crecer incorporando repositorios de cadena estructuralmente independientes (TF1).

L2 — Imputación por mediana de las características extendidas. Seis de las siete características extendidas —todas salvo `HOSTPATH_MOUNT`, poblada en las 2.827 filas— solo toman su valor real cuando el manifiesto de origen ha sido escaneado. El analizador propio las obtiene para el 32,6 % de las filas y Checkov cubre otro 41,6 %, de modo que las 731 restantes (25,9 %) reciben la mediana de columna y ven diluida su señal (Sección 5.2). Parte de esa carencia es irreducible: 54 manifiestos resultaron inaccesibles en origen (33 respuestas 404 y 21 respuestas 403). Un reescaneo de las filas recuperables acota el alcance para la Capa 1: la $F1$ macro permanece

inalterada ($\Delta = 0,0000$) y la perturbación del `risk_score` por fila presenta media 0,0030 y máximo 0,1104, un orden de magnitud por debajo de la variabilidad entre semillas de la Capa 2 ($\sigma = 0,033$ en P@5, Sección 6.3.6). El efecto sobre las Capas 2 y 3 está acotado por construcción: el constructor del grafo lee `FEATURE_COLS` e imputa 0 sin consultar las columnas `checkov_*`, pues `CHECKOV_FILL` opera solo en la carga de la Capa 1, de modo que la cobertura de escaneo no altera los grafos, sus etiquetas ni las salidas de las Capas 2 y 3. Esa invariancia desplaza el coste de la limitación: seis de las 25 banderas del vector de nodo carecen de señal en las 1.906 filas de Rahman et al. (2023), presentes en 82 de los 599 grafos.

L3 — Sesgo de muestreo en el conjunto de datos tabular. Los 10 repositorios más representados suponen el 47,6 % del conjunto de datos, y `gitcontroller` por sí solo aporta el 9,9 %. Un esquema de validación cruzada por repositorio ofrecería estimaciones más conservadoras de la generalización del modelo.

L4 — Evaluación en clústeres de producción reales. Toda la evaluación se ha realizado sobre repositorios públicos. La validación en entornos de producción de organizaciones reales es la siguiente etapa crítica para confirmar la validez externa del sistema.

L5 — Etiquetado derivado de reglas sobre el mismo espacio de características. Esta circularidad afecta a las dos capas. Las etiquetas de grafo (`clean` / `isolated` / `attack_chain`) se generan mediante una regla determinista definida sobre las mismas banderas y aristas que observan los modelos (Sección 5.3). En la Capa 1 la relación es más estrecha: sobre las 1.906 filas de Rahman et al. la etiqueta binaria coincide exactamente con la disyunción de ocho características de entrada, y reentrenar restringiéndose a esa fuente produce $F1 \text{ macro} = 1,0000$ (Sección 6.2.2). La $F1$ de 0,9946 sobre el corpus completo mide, por tanto, con qué fidelidad el modelo recupera esa regla bajo imputación y valores ausentes, no su capacidad de detectar configuraciones erróneas con un criterio ajeno a ella. La circularidad es común en la literatura de detección de configuraciones erróneas, pero limita el alcance de la afirmación «predice cadenas de ataque». Las vías para romperla son la validación externa frente a herramientas de grafo de ataque sobre clústeres activos (Datadog Security Labs, 2023; WithSecure Labs, 2023), el etiquetado experto de una submuestra y la confirmación mediante explotación controlada en laboratorio (Akula, 2020).

L6 — No determinismo de CUDA en las operaciones de *scatter* del GNN (hallazgo histórico, no reverificado sobre el corpus completo). Durante la fase preliminar de 471 grafos se observó que replicar el entrenamiento de la Capa 2 sobre GPU CUDA en lugar del *backend* MPS/CPU de referencia de esa fase alteraba las métricas de validación cruzada de forma no trivial (F1 macro entre 0,845 y 0,882 frente a 0,917; P@5 entre 0,77 y 0,92 frente a 0,880) con semilla, datos, particiones e hiperparámetros idénticos. Los resultados finales de esta memoria se entrenaron sobre GPU CUDA, por lo que la amenaza se invierte para quien reproduzca en CPU/MPS: es esa réplica la que puede divergir. La causa confirmada son las operaciones *scatter_add/scatter_reduce* que emplean GATConv y las capas de *pooling* global, implementadas en CUDA mediante *atomicAdd* sin versión determinista disponible; al no ser asociativa la suma en punto flotante, el orden de acumulación —dependiente de la planificación de hilos— introduce diferencias numéricas que alteran el *ranking* sobre un conjunto de validación reducido. Se descartaron como causa la precisión *TF32* y la paralelización del cargador de datos. Las banderas de determinismo de CUDA logran reproducibilidad exacta entre ejecuciones sobre la misma GPU, lo que indica que la brecha no es ruido aleatorio sino una divergencia fija entre *backends*; su cierre requeriría la ruta *SparseTensor* de PyTorch Geometric, documentada como determinista pero no evaluada aquí. El experimento no se ha repetido sobre el corpus completo, por lo que se recoge como hallazgo sobre el mecanismo causal y no como medición vigente del modelo desplegado.

L7 — Las métricas de las fases previas a la corrección por familias no son comparables con las finales, y las estimaciones finales llevan intervalos amplios. Las evaluaciones de las fases 1–5 estaban infladas por dos vías de fuga de información corregidas sucesivamente (variantes aumentadas en la fase 5 y familias de plantilla en la fase 6): el modelo «generalizaba» a casi duplicados de grafos vistos en entrenamiento. Tras la corrección, las cifras finales miden generalización a familias y repositorios no vistos, una cantidad distinta y sistemáticamente menor, por lo que ninguna comparación directa entre fases debe leerse como progresión del modelo. Como segunda cara de la misma limitación, el conjunto de test contiene solo 5 cadenas (todos los intervalos de confianza de test tienen una anchura $\geq 0,4$) y el *pool out-of-fold* contiene 32 cadenas de las que 15 son *fixtures* sintéticas: la evidencia sobre generalización a cadenas reales descansa en pocas muestras independientes, y ampliarlas es la vía prioritaria de trabajo futuro (TF1).

L8 — Alcance de la evaluación con usuarios y accionabilidad del informe. La evaluación de usabilidad y aplicabilidad se realizó con tres participantes expertos ($n = 3$, Sección 6.9.5), muestra que permite detectar problemas de usabilidad en una prueba formativa pero no estimar con precisión la puntuación SUS de 88,3 ni las medias de aplicabilidad; los perfiles cubren desarrollo/IA y DevOps/SRE, sin un rol de QA de laC puro, y la participación voluntaria admite sesgo de autoselección. Esas cifras se presentan, en consecuencia, como indicativas y no como estimaciones poblacionales. La propia evaluación identifica además una carencia funcional del sistema entregado: la accionabilidad del informe es la dimensión peor valorada ($A3 = 3,7$), pues la salida comunica el nivel de riesgo pero no las acciones de remediación ni las reglas concretas que sostienen el veredicto, lo que limita su utilidad para el operador que debe corregir la configuración (TF8).

L9 — Hiperparámetros fijados a priori. El número de cabezas de atención, la tasa de aprendizaje, la anchura oculta y la estrategia de *pooling* (*mean+max*) de la Capa 2 se fijaron a priori a partir de valores habituales en la literatura, sin búsqueda sistemática sobre los pliegues de validación (Sección 5.5). Las únicas elecciones de diseño contrastadas empíricamente son las cubiertas por las dos ablaciones realizadas: función de pérdida y criterio de selección de *checkpoints* (Sección 6.3.4) y arquitectura de agregación con o sin tipos de arista (Tabla 10). Las métricas presentadas corresponden, por tanto, a una configuración razonable pero no optimizada, y no puede descartarse que un barrido sistemático de hiperparámetros altere las comparaciones entre configuraciones sobre un conjunto de validación tan reducido (TF6).

L10 — El puente entre la Capa 1 y la Capa 2 no está efectivamente poblado. El índice 25 del vector de nodo se describe en el diseño como el canal por el que la señal por manifiesto alcanza el razonamiento estructural. En el corpus construido ese canal contiene una suma ponderada determinista de las mismas banderas que ocupan los índices 0–24, y está vacío —imputado como 0— en los 517 clústeres procedentes de repositorios de seguridad adicionales, el 86 % de los grafos. Una ablación que anula el índice 25 sobre los 86 grafos de test deja las métricas inalteradas ($P@1 = 1,00$, $P@3 = 1,00$, $P@5 = 0,80$, $FPR_clean = 0,00$; F1 macro 0,8825, con matriz de confusión idéntica). El resultado no permite concluir que la Capa 1 sea prescindible, sino que su aportación no ha llegado a evaluarse: el canal previsto para transmitirla no transportaba la salida del clasificador.

La Capa 3 hereda la misma discrepancia. El término $\overline{\text{risk}}(C)$ de la Ecuación 4 se calcula, al optimizar los pesos, como media del atributo nodal 25, de modo que el Algoritmo Genético ajustó w_{ff} —el mayor de los tres, 0,517— frente a una señal nula en el 86 % de los grafos, mientras que en inferencia ese término se evalúa sobre las probabilidades del clasificador. La magnitud del efecto de esa discrepancia sobre el *ranking* en inferencia no se ha cuantificado; establecerlo exigiría reoptimizar los pesos sobre predicciones *out-of-fold* construidas con la salida del clasificador, tarea recogida en TF9.

7.3.1. Amenazas a la validez

Las limitaciones L1–L10 se agrupan a continuación por el tipo de amenaza metodológica que representan, precisando el alcance de tres afirmaciones concretas del trabajo.

Validez de constructo (¿miden las métricas lo que se pretende medir?): L5 y L6 cuestionan, respectivamente, la independencia entre la etiqueta de grafo y la regla que la genera, y la equivalencia entre las métricas presentadas y las de una réplica en otro *backend*. A ellas se añade una cuestión propia de la métrica `FPR_clean`: mide la fracción de clústeres limpios en el top- k , no una tasa de falsos positivos en sentido estricto —que normalizaría por el total de clústeres limpios—, de modo que el valor 0,00 invocado en C3 y C5 significa ausencia de clústeres limpios en la cabeza del *ranking*, no ausencia de falsas alarmas sobre el conjunto completo (Sección 4.4).

Validez interna (¿las conclusiones causales están justificadas por el diseño experimental?): L7 impide atribuir al modelo las diferencias entre fases experimentales, porque el protocolo de evaluación cambió entre ellas; la ablación factorial de la Sección 6.3.4 es el único contraste interno con protocolo constante. L9 restringe las comparaciones entre configuraciones a un único punto del espacio de diseño. L2 y L10 acotan la interpretación de la arquitectura: la imputación de las características extendidas no compromete la comparabilidad entre configuraciones, pero la contribución de la Capa 1 a la Capa 2 no llega a medirse.

Validez de conclusión (¿son estadísticamente fiables las estimaciones puntuales?): el tamaño del conjunto de test (86 grafos, 5 cadenas) y del *pool out-of-fold* (513 grafos, 32 cadenas) produce los intervalos amplios de la Sección 6.4 —al 95 %, $P@5 \in [0,60; 1,00]$, $\text{FPR_clean} \in$

$[0,00; 0,40]$ y $F1 \text{ macro} \in [0,66; 1,00]$ —, que limitan cualquier comparación puntual entre configuraciones (C5). El intervalo de FPR_{clean} merece atención específica: el valor 0,00 es compatible con tasas de hasta 0,40, por lo que no autoriza a afirmar que el sistema no producirá falsas alarmas sobre corpus mayores. A ello se suman la concentración por familias (L1) —el tamaño muestral efectivo es el número de familias independientes por pliegue, 1, 1, 4, 5 y 5, no el número nominal de cadenas— y el tamaño de la muestra de usuarios (L8), con la que la puntuación SUS de 88,3 es indicativa y no una medida de precisión cuantificable.

Validez externa / generalización: L3 y L4 acotan hasta qué punto los resultados generalizan más allá de los repositorios públicos utilizados. El desplazamiento de distribución entre el *pool* de ajuste del GA, dominado por *fixtures* sintéticas, y las cadenas reales de test (Sección 6.4) es una amenaza externa adicional identificada empíricamente. La composición de la muestra de usuarios (L8), restringida a perfiles de desarrollo/IA y DevOps/SRE y obtenida por participación voluntaria, acota igualmente la generalización de las conclusiones de usabilidad.

7.4. Líneas de trabajo futuro

Las recomendaciones siguientes se ordenan por impacto esperado y atienden a una o varias de las limitaciones L1–L10 de la Sección 7.3, referenciadas por su identificador. TF1 y TF2 son las de mayor impacto potencial sobre la validez de las métricas presentadas.

TF1 — Ampliar la población de cadenas reales estructuralmente independientes. L1 y L7 comparten causa raíz: hay pocas familias de cadena independientes, y cada familia de plantilla añade poco porque sus miembros son casi duplicados. La incorporación dirigida de repositorios de infraestructura de organizaciones de código abierto y de laboratorios multiclúster —priorizando diversidad estructural sobre volumen— aumentaría el tamaño muestral efectivo y estrecharía los intervalos de test.

TF2 — Implementar P@5 global como métrica alternativa. Evaluar el *ranking* sobre el conjunto completo de grafos, en lugar de por pliegue, aumentaría el número de candidatos ordenados y reduciría la granularidad escalonada en pasos de 0,2 que amplifica la varianza señalada en L1.

TF3 — Incorporar análisis semántico de RBAC completo. El analizador actual identifica roles

privilegiados por nombre y por verbos de escalada. Analizar la cadena Role/ClusterRole → Role-Binding → ServiceAccount → Pod detectaría rutas de escalada sin banderas de escape directas.

TF4 — Validación cruzada dejando un repositorio fuera. La estrategia *leave-one-repo-out* ofrece estimaciones más conservadoras de la generalización, relevantes dada la concentración del corpus en pocos repositorios.

TF5 — Modo de análisis continuo (*admission controller*). Integrar kubescan como *validating admission webhook* —un servicio que el servidor API consulta para aceptar o rechazar cada recurso antes de persistirlo— permitiría bloquear en tiempo real el despliegue de manifiestos que eleven el riesgo de cadena del clúster.

TF6 — Ablación de *pooling* e hiperparámetros restantes. Queda pendiente extender el protocolo de la Tabla 10 —mismos pliegues y semillas— a *mean+max pooling*, a los hiperparámetros fijados a priori (L9) y a una ablación por tipo de arista que cierre la cuestión de C6.

TF7 — Ajuste del *ensemble* robusto al desplazamiento de distribución. Los pesos del GA se ajustan sobre un *pool out-of-fold* donde 15 de 32 cadenas son *fixtures* sintéticas, mientras las cadenas reales de test tienen *risk score* medio ≈ 0 ; tres refinamientos evaluados no corrigieron el sesgo (Sección 6.4). Las vías naturales son restringir el *pool* a repositorios reales —viable conforme TF1 amplíe esa población— o sustituir P@5 por una función objetivo continua.

TF8 — Explicabilidad y recomendaciones de remediación. La evaluación con usuarios (Sección 6.9) identificó la accionabilidad como principal punto de mejora (L8). El informe podría enriquecerse con recomendaciones de remediación por manifiesto y con la contribución de cada capa al veredicto, sin sacrificar la brevedad del *ranking*.

TF9 — Poblar y evaluar el canal de puntuación por manifiesto. Calcular el atributo nodal 25 con la salida del clasificador para todas las fuentes del corpus y repetir la ablación de L10 mediría si la señal por manifiesto aporta capacidad predictiva a la clasificación estructural, y permitiría reoptimizar los pesos del *ensemble* sobre predicciones *out-of-fold* homogéneas con las de inferencia, cerrando la discrepancia que L10 documenta en la Capa 3.

Referencias bibliográficas

- Akula, M. (2020). *Kubernetes Goat: Interactive Kubernetes security learning playground*. GitHub. Descargado de <https://github.com/madhuakula/kubernetes-goat> (Accedido: 14 de julio de 2026)
- ARMO Ltd. (2021). *Kubescape: Open-source Kubernetes security platform*. Herramienta de código abierto, proyecto incubado por la CNCF. Descargado de <https://github.com/kubescape/kubescape> (Accedido: 14 de julio de 2026)
- Art, S. (2021). *Bad pods: Kubernetes pod privilege escalation*. BishopFox Blog. Descargado de <https://bishopfox.com/blog/kubernetes-pod-privilege-escalation> (Accedido: 14 de julio de 2026)
- Bangor, A., Kortum, P., y Miller, J. (2009). Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3), 114–123.
- Bilot, T., Madhoun, N. E., Agha, K. A., y Zouaoui, A. (2023). Graph neural networks for intrusion detection: A survey. *IEEE Access*, 11, 49114–49139. <https://doi.org/10.1109/ACCESS.2023.3275789>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Bridgecrew. (2021). *Checkov: Open source static analysis for infrastructure as code*. GitHub. Descargado de <https://github.com/bridgecrewio/checkov> (Accedido: 14 de julio de 2026)
- Brooke, J. (1996). SUS: A quick and dirty usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester, y I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.
- Cloud Native Computing Foundation. (2022). *CNCF annual survey 2021*. CNCF Report. Descargado de <https://www.cncf.io/reports/cncf-annual-survey-2021/> (Publicado en febrero de 2022. Accedido: 14 de julio de 2026)
- Datadog Security Labs. (2023). *KubeHound: Identifying attack paths in Kubernetes clusters*. Herramienta de código abierto. Descargado de <https://github.com/DataDog/>

KubeHound (Publicado en octubre de 2023. Accedido: 14 de julio de 2026)

- Fey, M., y Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. En *ICLR Workshop on Representation Learning on Graphs and Manifolds*. Descargado de <https://arxiv.org/abs/1903.02428>
- Hagberg, A. A., Schult, D. A., y Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En *Proceedings of the 7th Python in Science Conference (SciPy)* (pp. 11–15). Descargado de <https://proceedings.scipy.org/articles/TCWV9851>
<https://doi.org/10.25080/TCWV9851>
- Hamilton, W. L. (2020). *Graph representation learning*. Morgan & Claypool. <https://doi.org/10.2200/S01045ED1V01Y202009AIM046>
- Hamilton, W. L., Ying, Z., y Leskovec, J. (2017). Inductive representation learning on large graphs. En *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 30).
- Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- Hutchins, E. M., Cloppert, M. J., y Amin, R. M. (2011). Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. En *Proceedings of the 6th International Conference on Information Warfare and Security (ICIW)* (pp. 113–125).
- Jabeen, G., Rahim, S., Afzal, W., Khan, D., Khan, A. A., Hussain, Z., y Bibi, T. (2022). Machine learning techniques for software vulnerability prediction: a comparative study. *Applied Intelligence*, 52(15), 17614–17635. <https://doi.org/10.1007/s10489-022-03350-5>
- Kipf, T. N., y Welling, M. (2017). Semi-supervised classification with graph convolutional networks. En *International Conference on Learning Representations (ICLR)*. Descargado de <https://openreview.net/forum?id=SJU4ayYgl>
- Li, Z., Zeng, J., Chen, Y., y Liang, Z. (2022). AttackKG: Constructing technique knowledge graph from cyber threat intelligence reports. En *Computer Security – ESORICS 2022* (pp. 589–609). https://doi.org/10.1007/978-3-031-17140-6_29
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., y Dollár, P. (2017). Focal loss for dense object detection. En *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 2980–2988). <https://doi.org/10.1109/ICCV.2017.324>
- Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., y Portmann, M. (2022). E-GraphSAGE:

- A graph neural network based intrusion detection system for IoT. En *IEEE/IFIP Network Operations and Management Symposium (NOMS)*. <https://doi.org/10.1109/NOMS54207.2022.9789878>
- Malul, E., Meidan, Y., Mimran, D., Elovici, Y., y Shabtai, A. (2024). *GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation*. arXiv:2405.19954 [cs.CR]. Descargado de <https://arxiv.org/abs/2405.19954>
- MITRE Corporation, y Center for Threat-Informed Defense. (2021). *ATT&CK for Containers*. MITRE Center for Threat-Informed Defense. Descargado de <https://ctid.mitre.org/projects/attck-for-containers/> (Incorporado en ATT&CK versión 9. Accedido: 14 de julio de 2026)
- Moustafa, N., y Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). En *2015 Military Communications and Information Systems Conference (MilCIS)* (pp. 1–6). <https://doi.org/10.1109/MilCIS.2015.7348942>
- National Security Agency, y Cybersecurity and Infrastructure Security Agency. (2022). *Kubernetes hardening guidance (v1.2)*. NSA/CISA Technical Report. Descargado de https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF (Publicado originalmente en agosto de 2021; revisión v1.2, agosto de 2022. Accedido: 14 de julio de 2026)
- Nielsen, J. (2000). *Why you only need to test with 5 users*. Nielsen Norman Group. Descargado de <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/> (Accedido: 14 de julio de 2026)
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 32). Descargado de <https://arxiv.org/abs/1912.01703>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rahman, A., Shamim, S. I., Bose, D. B., y Pandita, R. (2023). Security misconfigurations in

- open source Kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 32(4), 1–36. <https://doi.org/10.1145/3579639>
- Red Hat. (2024). *State of Kubernetes security report 2024* (Inf. Téc.). Red Hat, Inc. Descargado de <https://www.redhat.com/en/resources/kubernetes-adoption-security-market-trends-overview> (Accedido: 14 de julio de 2026)
- StackRox. (2021). *kube-linter: Static analysis for Kubernetes YAML files and Helm charts*. GitHub. Descargado de <https://github.com/stackrox/kube-linter> (Accedido: 14 de julio de 2026)
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., y Bengio, Y. (2018). Graph attention networks. En *International Conference on Learning Representations (ICLR)*. Descargado de <https://openreview.net/forum?id=rJXMpikCZ>
- WithSecure Labs. (2023). *IceKube: Finding complex attack paths in Kubernetes clusters*. Herramienta de código abierto. Descargado de <https://github.com/WithSecureLabs/IceKube> (Accedido: 14 de julio de 2026)
- Xu, K., Hu, W., Leskovec, J., y Jegelka, S. (2019). How powerful are graph neural networks? En *International Conference on Learning Representations (ICLR)*. Descargado de <https://openreview.net/forum?id=ryGs6iA5Km>
- Zhong, M., Lin, M., Zhang, C., y Xu, Z. (2024). A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. *Computers & Security*, 141, 103821. <https://doi.org/10.1016/j.cose.2024.103821>
- Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., ... Sun, M. (2020). Graph neural networks: A review of methods and applications. *AI Open*, 1, 57–81. <https://doi.org/10.1016/j.aiopen.2021.01.001>

Anexo A. Código fuente y datos analizados

El código fuente completo del sistema kubescan, incluyendo los *scripts* de adquisición de datos, entrenamiento de modelos, construcción de grafos y la interfaz de línea de comandos, se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ObedRav/kubescan>

El repositorio incluye los siguientes componentes:

- **kubescan/**: paquete Python instalable con la Command-Line Interface (Interfaz de Línea de Comandos) (CLI) y los modelos de inferencia.
- **kubescan/tests/**: batería de pruebas automatizadas, con siete módulos unitarios (`test_yaml_parser.py`, `test_graph_builder.py`, `test_gat_encoder.py`, `test_rf_classifier.py`, `test_ga_ensemble.py`, `test_device_utils.py` y `test_properties.py`, este último basado en pruebas de propiedades con Hypothesis) y una prueba de integración de extremo a extremo sobre la CLI (`integration/test_cli_scan.py`).
- **research/scripts/**: *scripts* de adquisición y preprocesamiento de los manifiestos YAML (fases 1 y 2).
- **research/models/**: código de entrenamiento del Random Forest (`train_rf.py`), de la GAT (`train_gnn.py`) y del Algoritmo Genético (`run_ga_ensemble.py`), junto a los *scripts* de verificación que reproducen las comprobaciones citadas en el Capítulo 7: `verify_label_circularity.py`, que contrasta la etiqueta de la Capa 1 con la disyunción de las columnas de configuración errónea, y `verify_risk_score_channel.py`, que documenta la procedencia y la cobertura del atributo nodal 25 y ejecuta su ablación sobre el conjunto de test.

- **research/data/**: los grafos de clúster en formato `.npz` (NumPy, con campos `x`, `edge_index`, `edge_attr`, `y`, `node_labels` y `risk_scores`) y el conjunto de datos tabular en formato `.csv`.
- **research/models/checkpoints/**: *checkpoints* de los 5 modelos GAT entrenados en validación cruzada y el modelo Random Forest (disponible también como enlace simbólico en `kubescan/checkpoints/trained/`).
- **thesis/usability-study/**: protocolo, cuestionario e instrumentos del estudio de usabilidad descrito en la Sección 6.9, junto con la plantilla de resultados y el *script* de agregación `compute_results.py`.
- **Makefile**: objetivos de reproducción del *pipeline* de investigación (`make data`, `make reproduce`).
- **requirements-lock.txt**: fijación exacta de versiones de todas las dependencias transitivas del entorno de entrenamiento.
- **.pre-commit-config.yaml** y **.github/workflows/ci.yml**: controles automáticos de estilo, tipado y ejecución de la batería de pruebas en integración continua.
- **thesis/**: fuentes LaTeX de esta memoria.

Los pesos entrenados se distribuyen junto con el código, de modo que un clon nuevo del repositorio puede ejecutar `kubescan scan` sin reentrenar ningún modelo. El directorio `research/models/checkpoints/` contiene los cinco *checkpoints* de validación cruzada (`gnn_fold_0.pt`–`gnn_fold_4.pt`), el *checkpoint* `gnn_best.pt`, los dos modelos Random Forest serializados en formato `.skops` (`rf_model.skops` y `rf_severity.skops`), los pesos del *ensemble* en `ga_weights.json` y los ficheros de métricas (`cv_results.json`, `rf_results.json`, `ga_results.json` y `test_results.json`). El conjunto ocupa aproximadamente 32 MB y es el mismo que consume la CLI a través del enlace simbólico `kubescan/checkpoints/trained/`.

Para garantizar la reproducibilidad, los resultados finales de esta memoria se entrenaron y evaluaron sobre una GPU NVIDIA T4 (CUDA) con Python 3.12, PyTorch 2.11 (CUDA 12.8), PyTorch Geometric 2.8, scikit-learn 1.6 y NumPy 2.0. Esa configuración corresponde al entorno

de entrenamiento e investigación, que exige GPU y las dependencias completas de PyTorch Geometric; el paquete distribuible admite un rango de instalación más amplio y su instalación se verificó también en entornos Python 3.10 y 3.11, según se detalla en la Sección 6.8 (RNF-3). Las versiones exactas de todas las dependencias, junto con las semillas, las particiones y las sumas de verificación (*checksums*) de los datos y los *checkpoints*, se registran en `research/models/checkpoints/run_manifest.json`; el flujo completo se reproduce mediante los objetivos del Makefile (`make data`, `make reproduce`).

El estudiante es el único autor de todos los *commits* del repositorio. Los datos públicos utilizados provienen de las fuentes citadas en la Sección 5.2.1 (Rahman et al., 2023; BishopFox BadPods, Kubernetes Goat y repositorios de seguridad adicionales de dominio público).

A.1. Catálogo de características del clasificador

Las tablas siguientes enumeran las 28 características que emplea la Capa 1, con su grupo de procedencia y su papel en el grafo de la Capa 2, según se describe en la Sección 5.2.

A.2. Trazabilidad de requisitos

La Tabla 15 vincula cada requisito de la Sección 4.1 con la sección que aporta la evidencia de su verificación.

A.3. Materiales del estudio de usabilidad

Las tablas siguientes recogen el perfil de los tres participantes y su tasa de éxito por tarea, según el protocolo descrito en la Sección 6.9.

Tabla 12. Características del clasificador Random Forest — grupo Rahman (1/3)

Nombre	Grupo	Rol en grafo	Descripción
TRUE_HOST_PID	Rahman	ESC	Comparte PID <i>namespace</i> del anfitrión
TRUE_HOST_IPC	Rahman	ESC	Comparte IPC <i>namespace</i> del anfitrión
TRUE_HOST_NET	Rahman	ESC	Comparte red del anfitrión
DOCKERSOCK_PATH	Rahman	ESC	Monta socket de Docker
CAP_SYS_ADMIN	Rahman	ESC	Capacidad SYS_ADMIN activa
CAP_SYS_MODULE	Rahman	ESC	Capacidad SYS_MODULE activa
WITHIN_MANIFEST_SECRET	Rahman	LAT	Secreto en texto plano embebido en el manifiesto
SEC_CONT_OVER_PRIVIL	Rahman	ESC	Contenedor privilegiado (privileged: true)
ALLOW_PRIVI	Rahman	LAT	Permite escalada de privilegios (allowPrivilegeEscalation: true)

Fuente: elaboración propia.

Tabla 13. Características del clasificador Random Forest — grupo Rahman (2/3)

Nombre	Grupo	Rol en grafo	Descripción
INSECURE_HTTP	Rahman	—	Usa HTTP sin TLS en puerto conocido
NO_SECU_CONTEXT	Rahman	—	Ausencia de <code>securityContext</code> definido
HOST_ALIAS	Rahman	—	Usa <code>hostAliases</code> para resolución DNS manual
NO_DEFAULT_NAMESPACE	Rahman	—	Despliega en el <i>namespace default</i>
NO_RESO	Rahman	—	Sin límites/ <i>requests</i> de recursos definidos
NO_ROLLING_UPDATE	Rahman	—	Sin estrategia de actualización <i>RollingUpdate</i>
SECCOMP_UNCONFINED	Rahman	—	Perfil <i>seccomp</i> <code>unconfined</code> (sin filtrado de <i>syscalls</i>)
VALID_TAINT_SECRET	Rahman	—	Secreto válido referenciado en <i>taint/toleration</i>
NO_NETWORK_POLICY	Rahman	—	Sin <code>NetworkPolicy</code> definida en el <i>namespace</i>

Fuente: elaboración propia.

Tabla 14. Características del clasificador Random Forest — grupos derivado y extendido, y nodo (3/3)

Nombre	Grupo	Rol en grafo	Descripción
cap_misuse	Derivada	—	OR lógico de CAP_SYS_ADMIN y CAP_SYS_MODULE
all_secrets	Derivada	—	OR de los indicadores de exposición de secretos
total_misconfigs	Derivada	—	Suma de todas las banderas activas
NO_RUN_AS_NON_ROOT	Extendida	—	Sin restricción de usuario no root
NO_READ_ONLY_ROOT_FS	Extendida	—	Sistema de ficheros raíz con escritura
IMAGE_USES_LATEST	Extendida	—	Imagen con tag: latest
SA_AUTOMOUNT_TOKEN	Extendida	LAT	ServiceAccount con <i>token</i> automontado
USES_DEFAULT_SA	Extendida	LAT	Usa la ServiceAccount default del <i>namespace</i>
UNTRUSTED_REGISTRY	Extendida	—	Imagen proveniente de un registro no confiable
HOSTPATH_MOUNT	Extendida	ESC	Monta ruta del sistema de ficheros del anfitrión
risk_score	Nodo (Capa 2)	—	Puntuación de severidad por manifiesto $\in [0, 1]$: heurística determinista en el corpus, probabilidad del Random Forest en inferencia (L10)

Fuente: elaboración propia.

Tabla 15. Trazabilidad de requisitos

ID	Requisito	Diseño	Verificación
RF-1	Configuraciones erróneas individuales	5.4	6.2
RF-2	Cadenas de ataque	5.5	6.3
RF-3	Interfaz de línea de comandos	5.7	6.8
RF-4	Formatos de salida	5.7	6.8
RF-5	Priorización de riesgos	5.6	6.7
RNF-1	Trazabilidad	5.8	6.8
RNF-2	Reproducibilidad	5.7	6.8
RNF-3	Instalabilidad	5.7	6.8
RNF-4	Latencia	5.7	6.8

Fuente: elaboración propia.

Tabla 16. Perfil de los participantes expertos

Participante	Rol	Años K8s	CI/CD	Seguridad
P1	Desarrollo/IA	3–5	Habitual	Alta
P2	DevOps/SRE	1–3	Habitual	Alta
P3	Desarrollo/IA	3–5	Habitual	Media

Fuente: elaboración propia.

Tabla 17. Tasa de éxito por tarea (3 participantes)

Tarea	T1	T2	T3	T4	T5	Global
Éxito	100 %	100 %	100 %	83 %	67 %	90 %

Fuente: elaboración propia.

Anexo B. Guía de uso y reproducción

La instalación de kubescan, la interpretación del informe que produce sobre un caso real y la reproducción del *pipeline* de entrenamiento de investigación descrito en el Capítulo 5 requieren los pasos concretos que se detallan en los apartados siguientes. Los criterios de diseño y las decisiones de implementación que sustentan cada uno de esos pasos se documentan en la Sección 5.7. Salvo indicación en contrario, todos los comandos se ejecutan desde la raíz del repositorio.

B.1. Instalación

El paquete se instala en modo editable desde la raíz del repositorio:

```
pip install -e kubescan/
```

Las versiones de Python soportadas y la resolución de dependencias se verifican en la Sección 6.8 (RNF-3).

B.2. Uso de la interfaz de línea de comandos

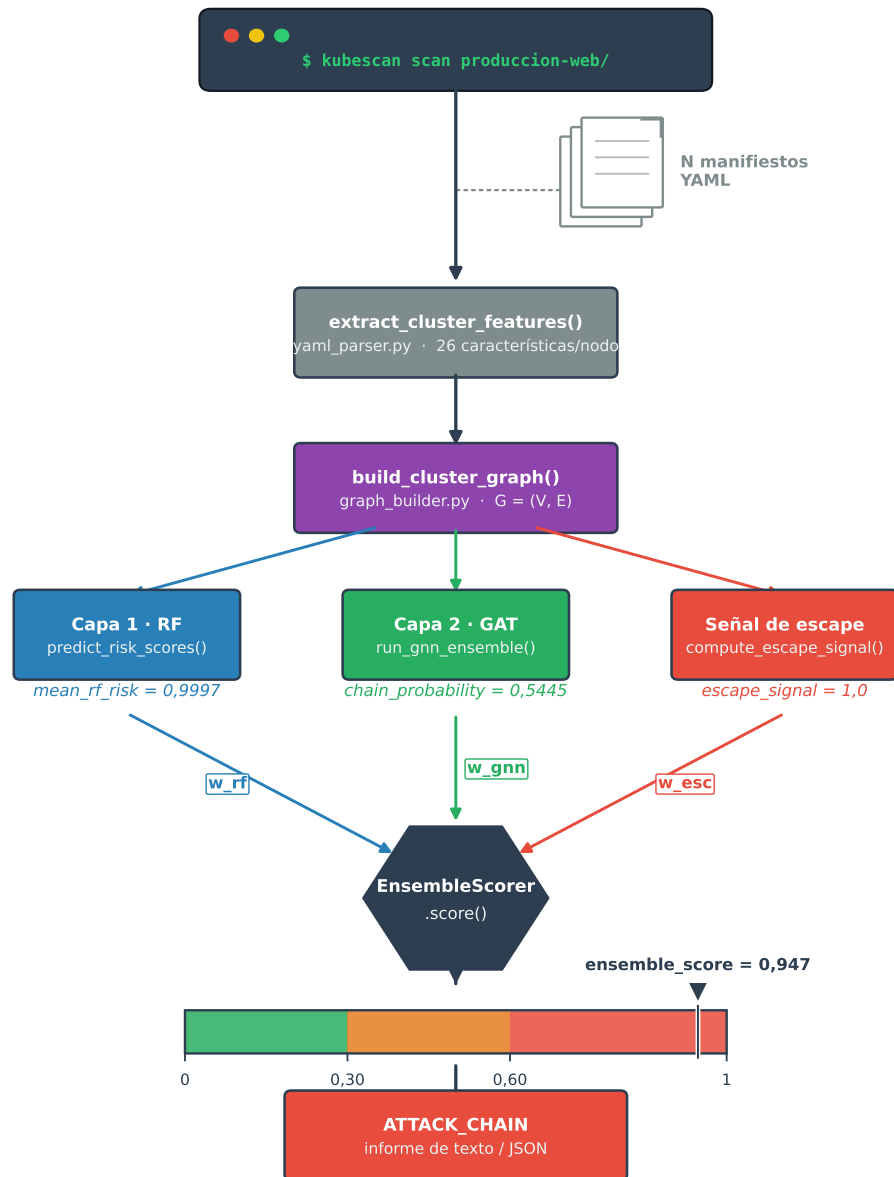
B.2.1. Análisis de un directorio de manifiestos

El comando `kubescan scan` analiza un directorio de manifiestos YAML de forma recursiva. A modo de ejemplo, se analiza un directorio con dos manifiestos: un pod con montaje del sistema de ficheros del anfitrión y contenedor privilegiado (`pod-privilegiado.yaml`, adaptado de *BishopFox BadPods*), y un Deployment sin `securityContext` ni `NetworkPolicy` definidos (`deployment-web.yaml`, adaptado de Kubernetes Goat).

La Figura 8 traza el recorrido completo de esta invocación: extracción de características, construcción del grafo, las tres señales de entrada al *ensemble* (RF, GAT y escape) fusionadas por

`EnsembleScorer.score()`, y la clasificación final mediante los umbrales de la Sección 5.6.

Figura 8. Flujo de ejecución de *kubescan scan*



Fuente: elaboración propia.

Como se observa en la Figura 8, las tres señales convergen ponderadas por w_{rf} , w_{gnn} y w_{esc} (optimizados por el GA) en un único *ensemble_score*; en este caso, la presencia de un nodo con banderas de escape activas (*pod-privilegiado.yaml*) lo sitúa muy por encima del umbral de 0,60, lo que da como resultado el veredicto *ATTACK_CHAIN*. La transcripción completa de esta ejecución es la siguiente:

```
$ kubescan scan /tmp/manifests-ejemplo --cluster-name produccion-web
Scanning /tmp/manifests-ejemplo (5 GNN fold models loaded)...
  2 manifest(s) with workload resources found
```

```
=====
KUBESCAN · Attack-Chain Risk Report
Cluster : produccion-web
Path    : /tmp/manifests-ejemplo
=====

VERDICT · ATTACK_CHAIN          X HIGH RISK - review immediately

Ensemble score      : 0.9472
Chain probability   : 0.5445   (5-fold GNN ensemble)
Clean probability   : 0.0001
Mean RF risk       : 0.9997
Escape fraction    : 0.5000   (1/2 manifests have escape flags)
Lateral fraction   : 2/2 manifests have lateral flags

Weights: w_rf=0.517  w_gnn=0.115  w_escape=0.368
=====
```

En esa transcripción, el carácter X sustituye al símbolo *Unicode* U+2717 que imprime la herramienta, ausente en la fuente monoespaciada de esta memoria; el resto de los caracteres reproduce literalmente la salida de `cli.py`.

El indicador `--show-nodes` añade un desglose por manifiesto con tres columnas: el *risk score* del Random Forest, la columna Type y las banderas activas. La columna Type no contiene el *kind* del recurso de Kubernetes, sino las etiquetas de capacidad del nodo derivadas de la taxonomía de la Sección 5.8: ESC cuando el manifiesto activa alguna bandera de escape, LAT cuando activa alguna bandera de movimiento lateral, y ESCLAT cuando concurren ambas. El indicador `--format json` produce el mismo informe en formato JSON estructurado para integración con otras herramientas; la salida de `kubescan scan` sobre `/tmp/manifests-ejemplo`, con los campos de detalle por manifiesto truncados mediante elipsis, es la siguiente:

```
$ kubescan scan /tmp/manifests-ejemplo --cluster-name produccion-web --format json
{
  "cluster": "produccion-web",
  "verdict": "ATTACK_CHAIN",
  "ensemble_score": 0.947236,
  "chain_probability": 0.5445,
  "mean_rf_risk": 0.999711,
  "n_manifests": 2,
  "n_escape_capable": 1,
  "manifests": [
    {
      "file": "pod-privilegiado.yaml",
      "risk_score": 1.0,
      "escape_capable": true,
      "flags": ["TRUE_HOST_PID", "TRUE_HOST_IPC", "..."]
    },
    "..."
  ]
}
```

Los pesos del modelo se resuelven mediante la cadena descrita en la Sección 5.7 (argumento `--checkpoints-dir` → variable de entorno `KUBESCAN_CHECKPOINTS` → enlace simbólico); no es necesario indicarlos explícitamente si el paquete se instaló junto con el repositorio completo.

B.2.2. Análisis de un clúster en ejecución

El comando `kubescan live` aplica el mismo *pipeline* sobre el estado actual de un clúster accesible mediante `kubectl`, sin necesidad de exportar manifiestos manualmente. La herramienta invoca `kubectl` para volcar *pods*, *deployments*, *daemonsets*, *statefulsets*, *replicasets*, *jobs*, *cronjobs*, *roles* y *rolebindings* a un directorio temporal, y a partir de ahí ejecuta la misma cadena de extracción, construcción del grafo y puntuación del *ensemble*. Las dos formas de invocación admitidas son las siguientes:

```
kubescan live -n produccion
```

```
kubescan live --all-namespaces --format json
```

El formato del informe resultante coincide con el de `kubescan scan`, tanto en la variante de texto como en la variante JSON, y difiere únicamente en la cabecera: el campo `Cluster` toma por defecto el *namespace* consultado y el campo `Path` apunta al directorio temporal en el que se vuelca el estado del clúster. Por esa razón no se reproduce aquí una transcripción independiente, cuyos valores dependerían del clúster concreto sobre el que se ejecute.

Este comando requiere `kubectl` configurado con permisos de lectura sobre *workloads* y recursos RBAC en el clúster objetivo.

B.3. Reproducción del *pipeline* de investigación

El *pipeline* de entrenamiento se ejecuta mediante los objetivos definidos en el Makefile de la raíz del repositorio. Los pasos de adquisición y extracción (`research/scripts/01_acquire/`, con `download_github_manifests.py` e `ingest_attack_repos.py`; y `research/scripts/02_extract/`, con `extract_yaml_features.py` y `build_graphs.py`) son intensivos en red y no están automatizados en el Makefile; se ejecutan manualmente una única vez y producen los grafos de clúster en `research/data/graphs/*.npz`.

A partir de ahí, el resto del *pipeline* es reproducible con dos objetivos:

```
make data          # aumentacion -> cache de grafos -> splits group-aware
make reproduce     # data -> RF -> GAT (5-fold) -> ensemble GA -> eval en test
```

El objetivo `reproduce` encadena, en orden, `train_rf.py`, `train_gnn.py` (5-fold, 300 épocas, `hidden=64`, `heads=4`, `layers=3`, entropía cruzada ponderada y selección de *checkpoint* por Precisión@5 mediante `--select-by p5`), `run_ga_ensemble.py` (ajuste sobre predicciones *out-of-fold*, `--oof`) y `evaluate_test_set.py` (evaluación con la restricción de viabilidad estructural de la Sección 5.6.1), todos con semilla fija (`--seed 42` por defecto), y finaliza generando `research/models/checkpoints/run_manifest.json` con la procedencia de la ejecución. Las variables `GNN_FOCAL` y `GNN_SELECT_BY` del Makefile permiten reproducir las ramas de la ablación de la Sección 6.3.4, y el cuaderno parametrizado

research/notebooks/colab_reproduce.ipynb ofrece la misma cadena para entornos Google Colab con GPU. El objetivo reproduce-ablations reentrena las variantes de arquitectura recogidas en la Tabla 10 (GAT de 2 capas y GCN de 3 capas) con la misma configuración de pérdida y selección que el modelo desplegado.

Tabla 18. Opciones de la interfaz de línea de comandos de kubescan

Opción	Ámbito	Efecto
--format {text,json}	scan, live	Formato de salida; text por defecto
--show-nodes	scan, live	Desglose de riesgo por manifiesto (solo formato texto)
--cluster-name, -n*	scan, live	Nombre legible del clúster en el informe
--checkpoints-dir, -c	scan, live	Directorio de <i>checkpoints</i> del modelo
--namespace, -n	live	Namespace consultado vía kubectl
--all-namespaces, -A	live	Consulta todos los <i>namespaces</i>
--verbose, -v	Grupo raíz	Registro en nivel DEBUG
--version	Grupo raíz	Versión instalada del paquete

*La forma corta -n corresponde a --cluster-name en scan y a --namespace en live.

Fuente: elaboración propia.

Anexo C. Acrónimos y abreviaturas

API	Application Programming Interface (Interfaz de Programación de Aplicaciones)
AUC	Area Under the Curve (Área Bajo la Curva ROC)
BFS	Breadth-First Search (Búsqueda en Anchura)
CI/CD	Continuous Integration / Continuous Delivery (Integración y Entrega Continua)
CIS	Center for Internet Security (Centro para la Seguridad en Internet)
CISA	Cybersecurity and Infrastructure Security Agency (Agencia de Ciberseguridad y Seguridad de Infraestructuras)
CLI	Command-Line Interface (Interfaz de Línea de Comandos)
CNCF	Cloud Native Computing Foundation (Fundación de Computación Nativa en la Nube)
CPU	Central Processing Unit (Unidad Central de Procesamiento)
CTF	Capture The Flag (competición de retos de seguridad)
CTI	Cyber Threat Intelligence (Inteligencia de Amenazas)
CUDA	Compute Unified Device Architecture (Arquitectura Unificada de Dispositivos de Cómputo)
CV	Cross-Validation (Validación Cruzada)
CVE	Common Vulnerabilities and Exposures (Vulnerabilidades y Exposiciones Comunes)
DevOps	Development and Operations (integración de desarrollo y operaciones)
ELU	Exponential Linear Unit (Unidad Lineal Exponencial)
ESC	Escape (rol de nodo con capacidad de escape al anfitrión)

FPR	False Positive Rate (Tasa de Falsos Positivos)
GA	Genetic Algorithm (Algoritmo Genético)
GAT	Graph Attention Network (Red de Atención sobre Grafos)
GCN	Graph Convolutional Network (Red Convolucional de Grafos)
GNN	Graph Neural Network (Red Neuronal de Grafos)
GPU	Graphics Processing Unit (Unidad de Procesamiento Gráfico)
IA	Inteligencia Artificial (Artificial Intelligence)
IaC	Infrastructure as Code (Infraestructura como Código)
IDS	Intrusion Detection System (Sistema de Detección de Intrusiones)
IoT	Internet of Things (Internet de las Cosas)
JSON	JavaScript Object Notation (Notación de Objetos de JavaScript)
K8s	Kubernetes (abreviatura convencional del proyecto)
LAT	Lateral (rol de nodo con capacidad de movimiento lateral)
LLM	Large Language Model (Modelo de Lenguaje de Gran Escala)
MITRE ATT&CK	Adversarial Tactics, Techniques and Common Knowledge (base de conocimiento de tácticas y técnicas adversarias de MITRE)
ML	Machine Learning (Aprendizaje Automático)
MLP	Multi-Layer Perceptron (Perceptrón Multicapa)
MPS	Metal Performance Shaders (Motor de Cómputo Acelerado de Apple)
NSA	National Security Agency (Agencia de Seguridad Nacional)
NVIDIA	NVIDIA Corporation (fabricante de las unidades de procesamiento gráfico empleadas en el entrenamiento)

QA	Quality Assurance (Aseguramiento de la Calidad)
RBAC	Role-Based Access Control (Control de Acceso Basado en Roles)
RBF	Radial Basis Function (Función de Base Radial)
ReLU	Rectified Linear Unit (Unidad Lineal Rectificada)
ROC	Receiver Operating Characteristic (Característica Operativa del Receptor)
SA	Service Account (Cuenta de Servicio de Kubernetes)
SLI-KUBE	Static Learning-based Inspection for Kubernetes (herramienta de inspección estática de manifiestos Kubernetes)
SMOTE	Synthetic Minority Over-sampling Technique (Técnica de Sobremuestreo Sintético de la Clase Minoritaria)
SRE	Site Reliability Engineering (Ingeniería de Fiabilidad de Sistemas)
SUS	System Usability Scale (Escala de Usabilidad del Sistema)
SVM	Support Vector Machine (Máquina de Vectores Soporte)
UMI	Unified Misconfiguration Index (Índice Unificado de Configuraciones Erróneas)
UNSW	University of New South Wales (Universidad de Nueva Gales del Sur)
YAML	YAML Ain't Markup Language (Lenguaje de Serialización de Datos Legible por Humanos)